



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

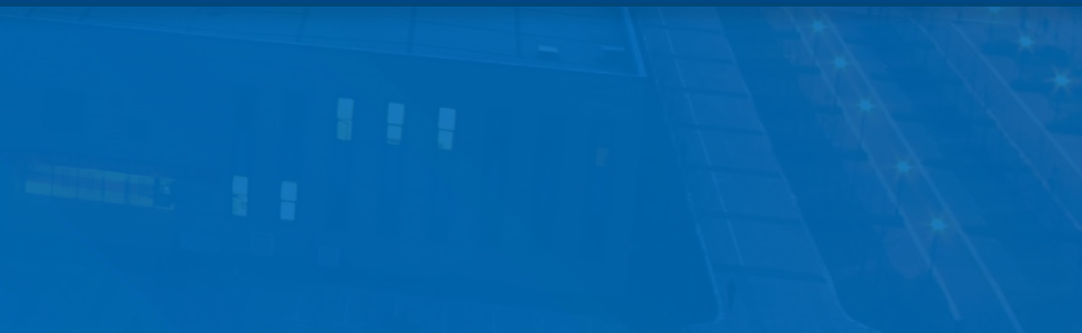
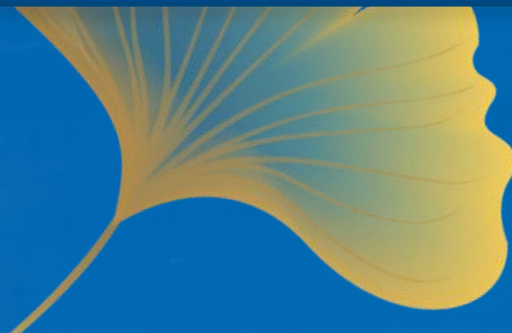
1902 2022

A small, stylized illustration of a traditional Chinese building, likely a part of Nanjing University's campus, positioned between the years 1902 and 2022.

微服务架构

Microservices Architecture

李杉杉





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

CONTENT

目录

01

微服务架构基础知识

02

微服务设计核心模式





南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

01

微服务架构基础

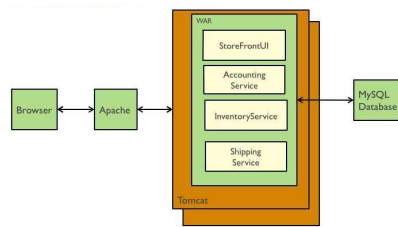
- 主流架构风格
- 发展历程
- 主流定义
- 主要特性
- 架构风格对比



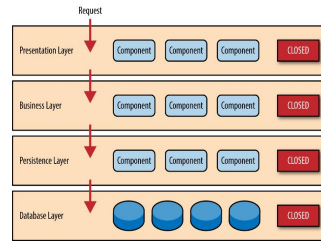
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



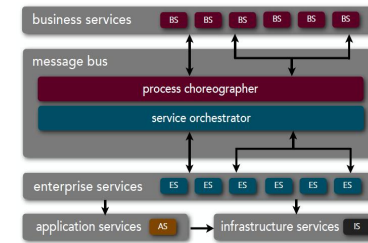
主流架构风格



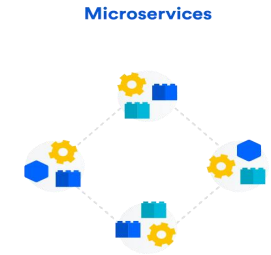
单体架构



分层架构



SOA架构



微服务架构



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || 单体架构

➤ 单体（Monolithic）应用的全部功能被集成在一起作为一个单一的单元。

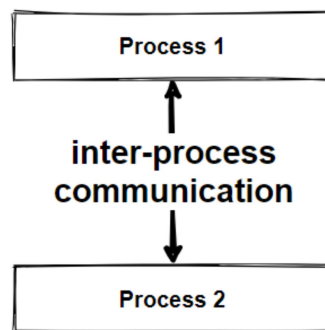
- Java Web应用程序（WAR）文件
- GoLang语言，交付形态为单一可执行文件
- Ruby或Node.js，交付形态为目录和子目录下各种源码
- 初始化、配置、监控、管理等功能

➤ 通信方式

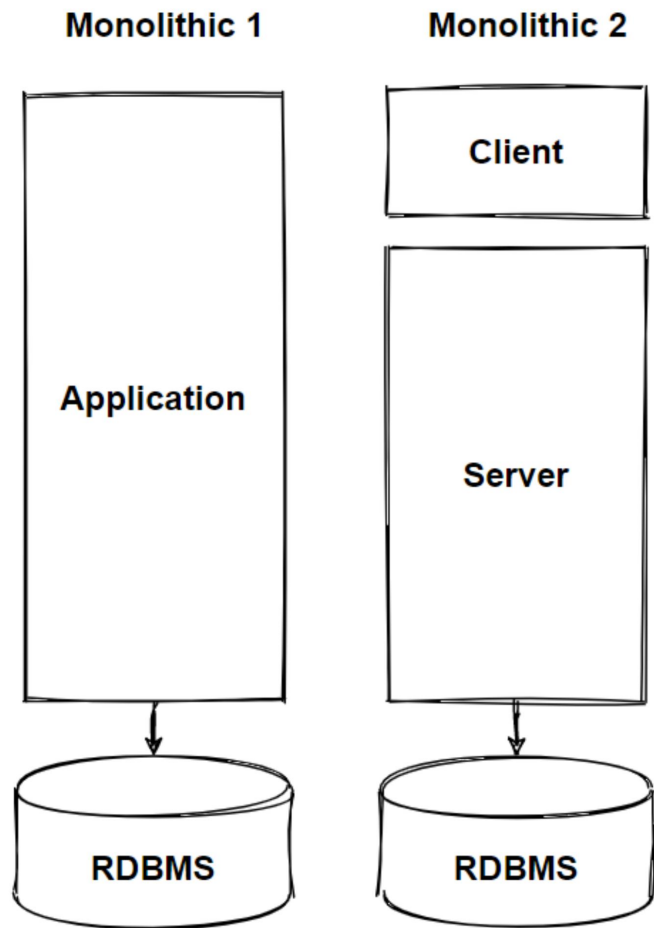
- 进程间通信
- 方法间调用
- 无需网络调用

➤ 事务管理

- 单一数据库
- 所有事务单一上下文
- 事务提交和回滚操作简单
- 一个方法中处理



```
function place_order()  
do_payment  
decrease_stock  
send_shipment  
generate_bill  
update_order
```





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || 单体架构

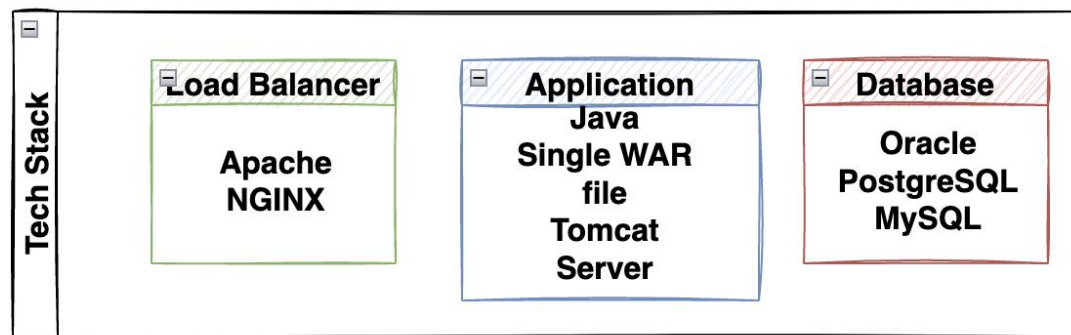
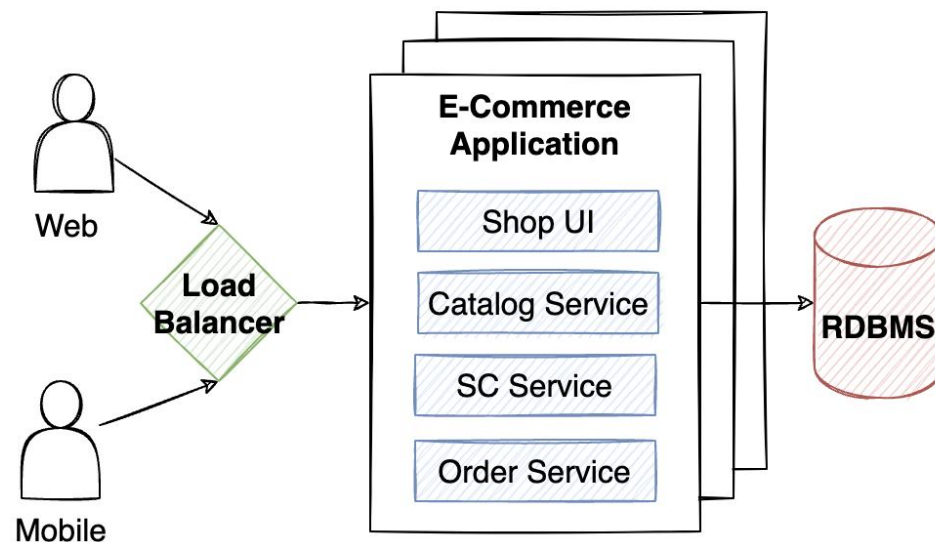
单体架构示例1（某电商应用）

➤ 功能性需求

- 浏览商品
- 查找和筛选商品
- 查看折扣
- 加购物车
- 下单
- 查看历史订单
-

➤ 非功能性需求

- 可伸缩性
- 性能
-





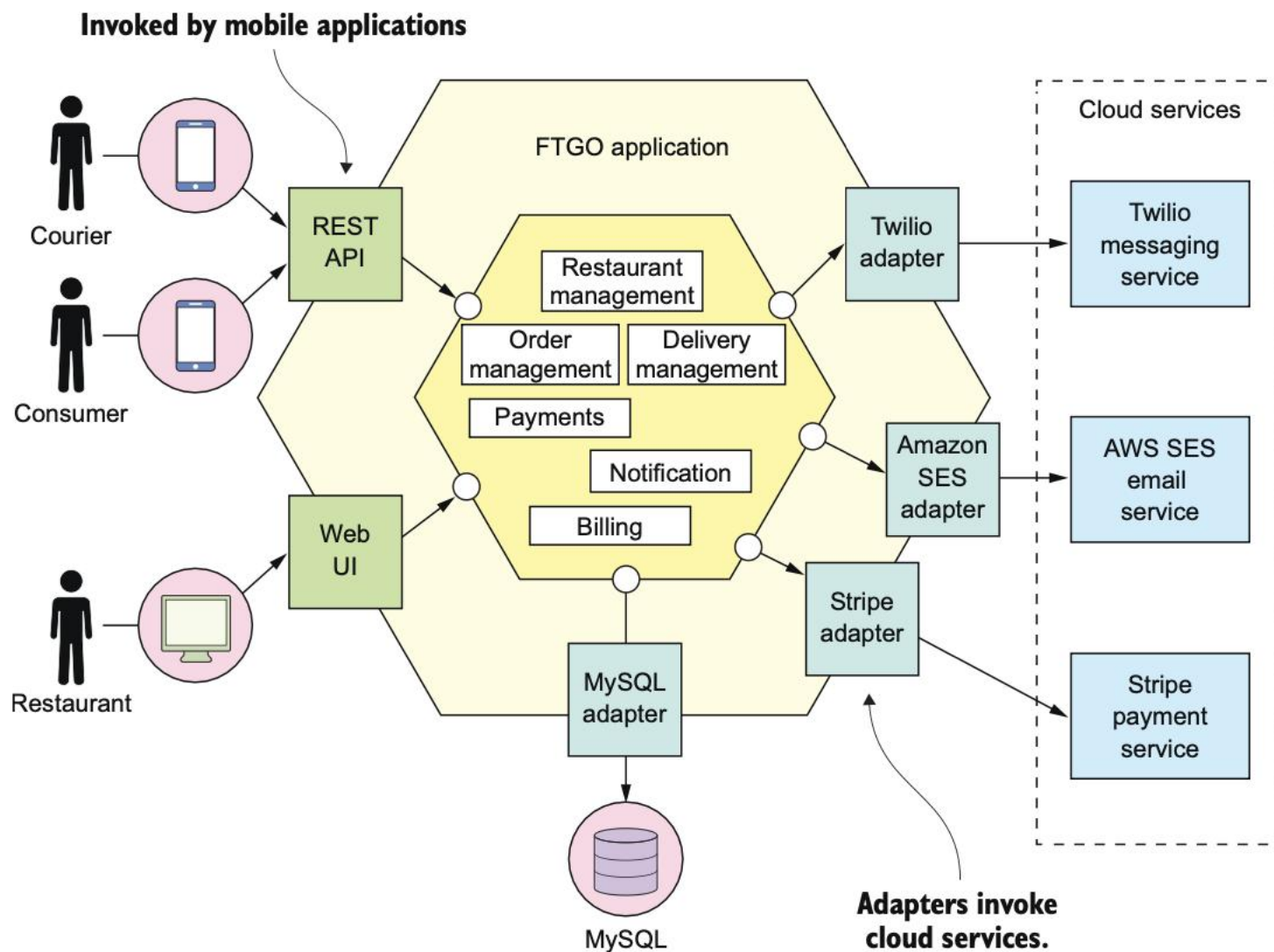
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || 单体架构

单体架构示例2（FTGO应用）

- 订餐应用
- 分层模块化企业级Java应用
- 核心是业务逻辑组件
 - 订单管理
 - 配送管理
 - 餐馆管理等
- 业务逻辑外是实现用户界面的适配器和与外部系统的接口
 - Stripe管理支付
 - Twilio实现消息传递
 - Amazon SES发送电子邮件
 - MySQL存储数据





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

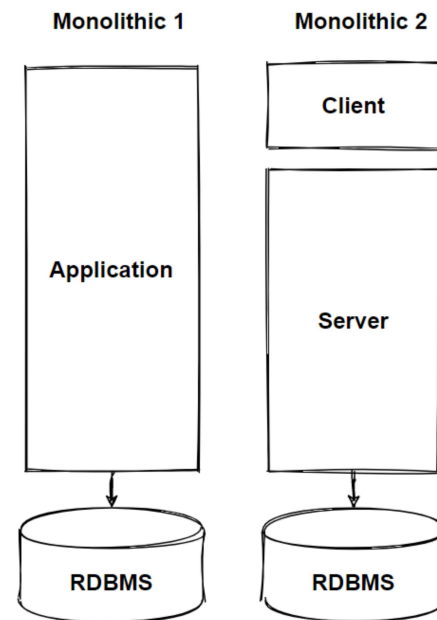


主流架构风格 || 单体架构

➤ 单体架构的好处

- 易于开发 (Development)
 - 一个应用 (不复杂)
- 易于修改 (Modifiability)
 - 代码和数据库模式
- 易于测试 (Testability)
 - 端到端测试、UI测试
- 易于部署 (Deployability)
 - WAR文件复制
- 易于伸缩 (Scalability)
 - 多个实例负载均衡

Concurrent Users	Requests/second	Latency (Expected)
2K	0.5K	
20K	12K	
100K	80K	<= 2 sec
500K	300K	?





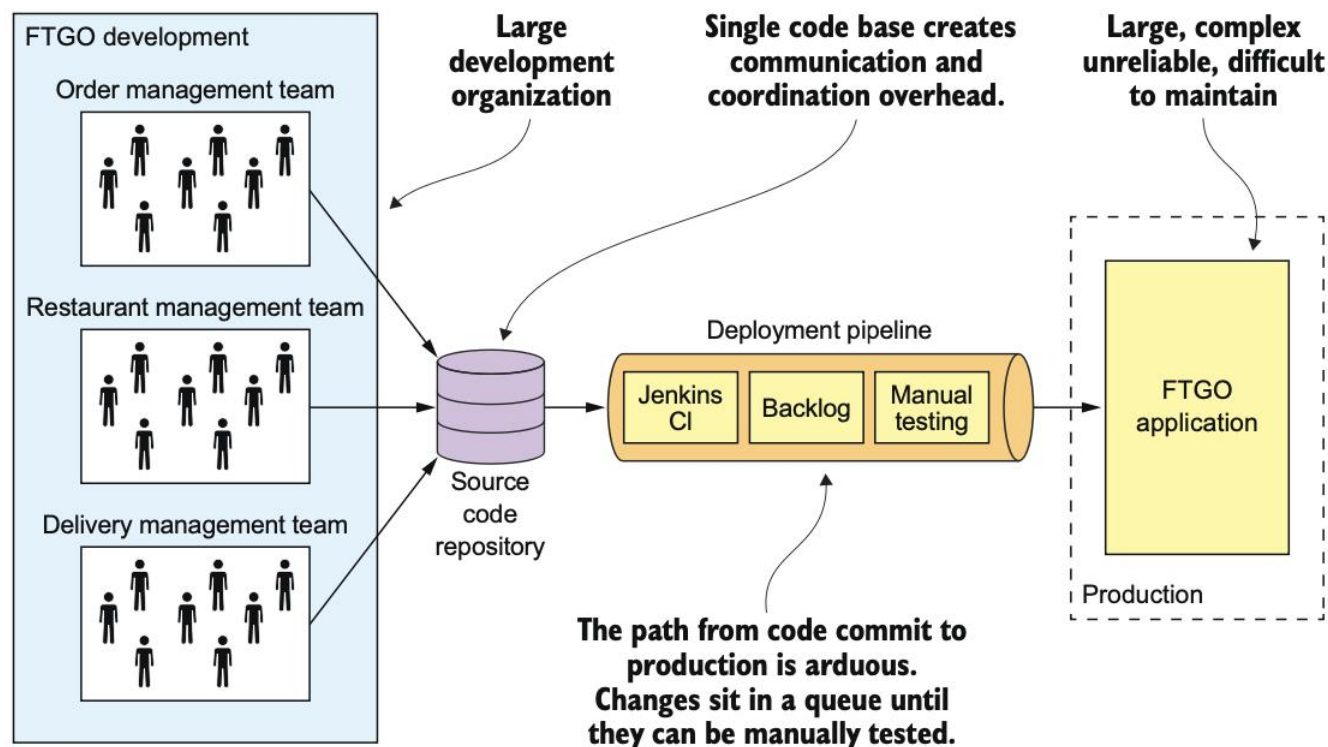
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || 单体架构

➤ 单体架构的问题

- 过度复杂性
 - 理解、数以千计JAR文件和百万行代码
 - 修复bug和实现新功能易出错、脏泥球
- 开发速度缓慢
 - IDE编辑-构建-启动运行
- 部署周期长且易出错
 - 微小变更同一代码库提交、构建
 - 变更测试（持续集成测试套件、手动）
- 难以扩展
 - 资源冲突
- 可靠性差
 - 无法全面测试
 - 缺少故障隔离
- 过期技术栈





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

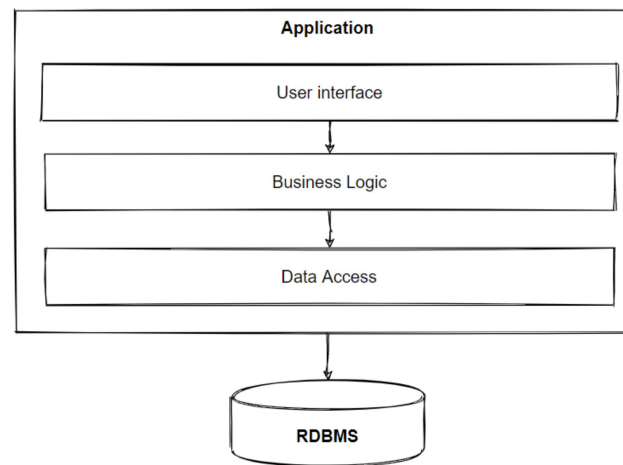


主流架构风格 || 分层架构

➤ 分层架构是对复杂系统进行抽象和分层、结构化设计的架构方案

➤ 垂直架构（结构简单，易于组织开发、测试和维护）

- 表现层、业务层、（持久层）、数据层
- 关注点分离原则
 - 软件元素的独特性
 - 责任分离
 - 高内聚、低耦合
- SOLID原则
 - 单一职责原则（Single Responsibility Principle）
 - 开闭原则（Open/Closed Principle）
 - 里氏替换原则（Liskov Substitution Principle）
 - 接口隔离原则（Interface Segregation Principle）
 - 依赖倒置原则（Dependency Inversion Principle）



Separation of Concerns



Separation of Concerns
(only, from a different point of view)





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || 分层架构

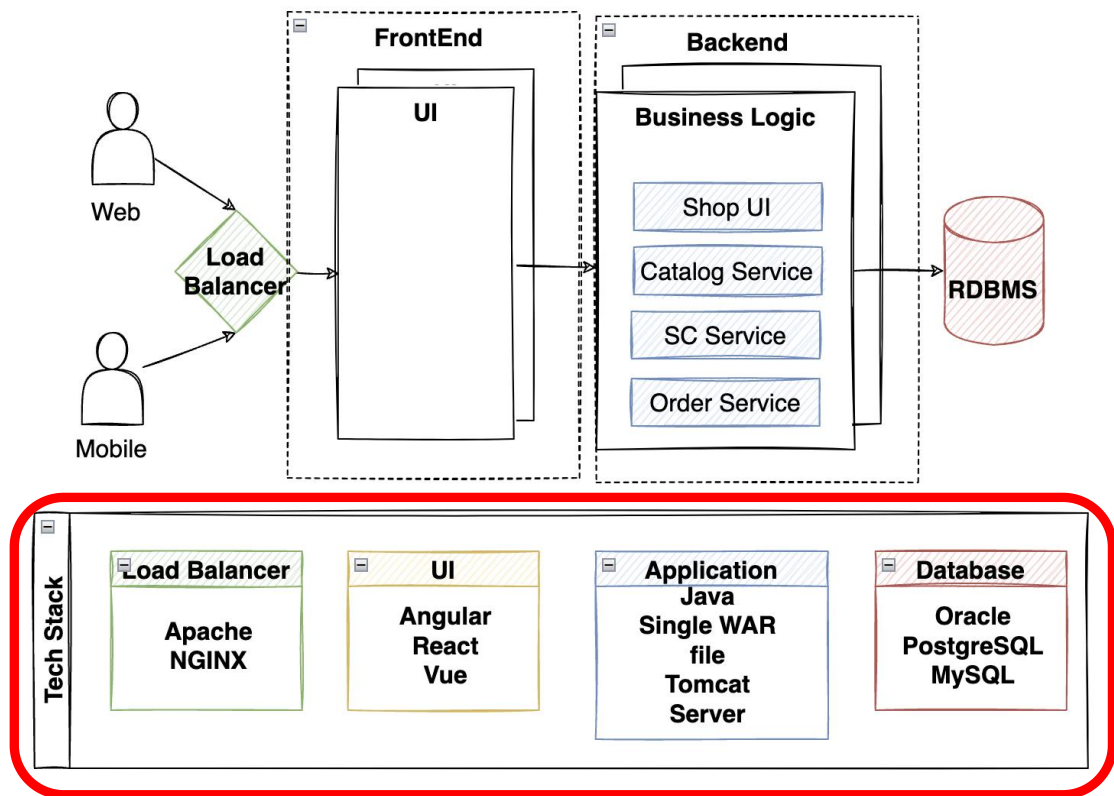
分层架构示例（某电商应用）

➤ 功能性需求

- 浏览商品
- 查找和筛选商品
- 查看折扣
- 加购物车
- 下单
- 查看历史订单
-

➤ 非功能性需求

- 可伸缩性
- 性能
- **可维护性**
-



- 业务增长，更多、更高的非功能性需求，可用性、可靠性等
- 单向层级调用性能代价
- 紧耦合重复造轮子，堕落成无序的网状结构



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || SOA架构

➤ 面向服务架构（SOA）

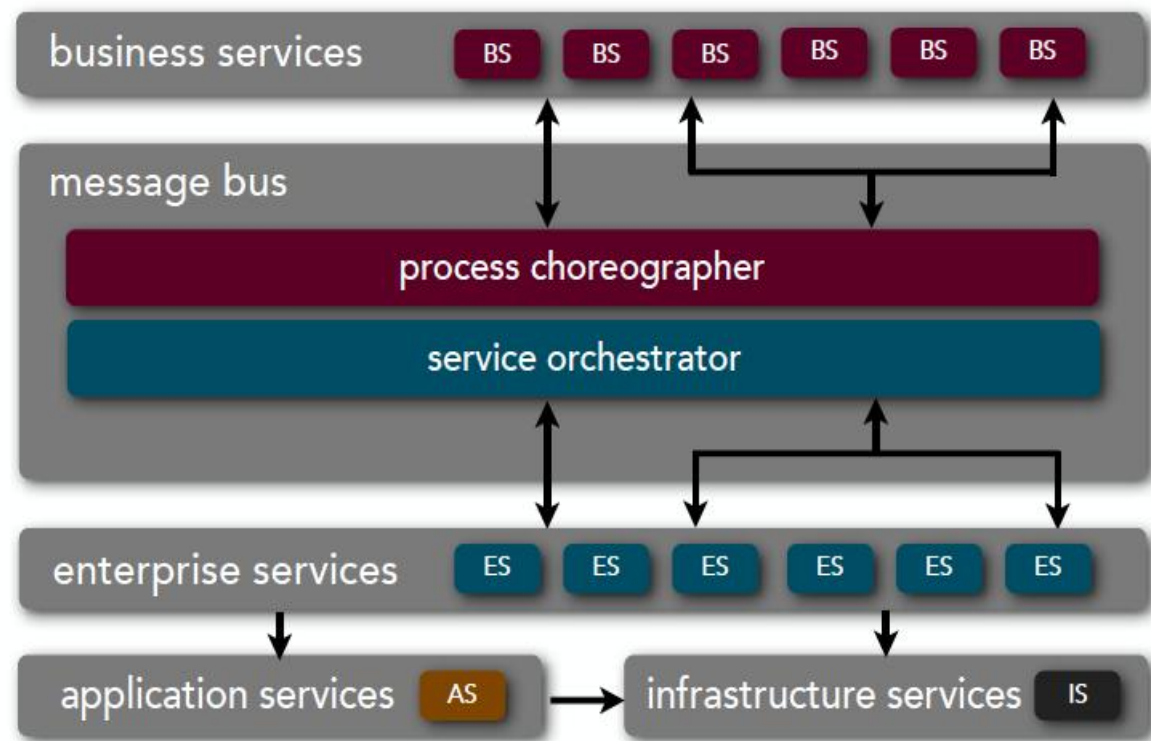
是一个**分布式**组件的集合，这些组件为其它组件**提供**服务（provider），或者**消费**其它组件所提供的服务（consumer），而无需知道其它组件的实现细节。

➤ 企业服务总线（ESB）

为服务间的**相互调用**提供支持环境，**路由**服务间的消息，并对消息和数据进行必要的**转换**。

➤ 服务编排引擎（Orchestration Engine）

可以根据预先定义的**脚本**对服务消费者与服务提供者之间的交互进行**指挥**。





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



主流架构风格 || SOA架构

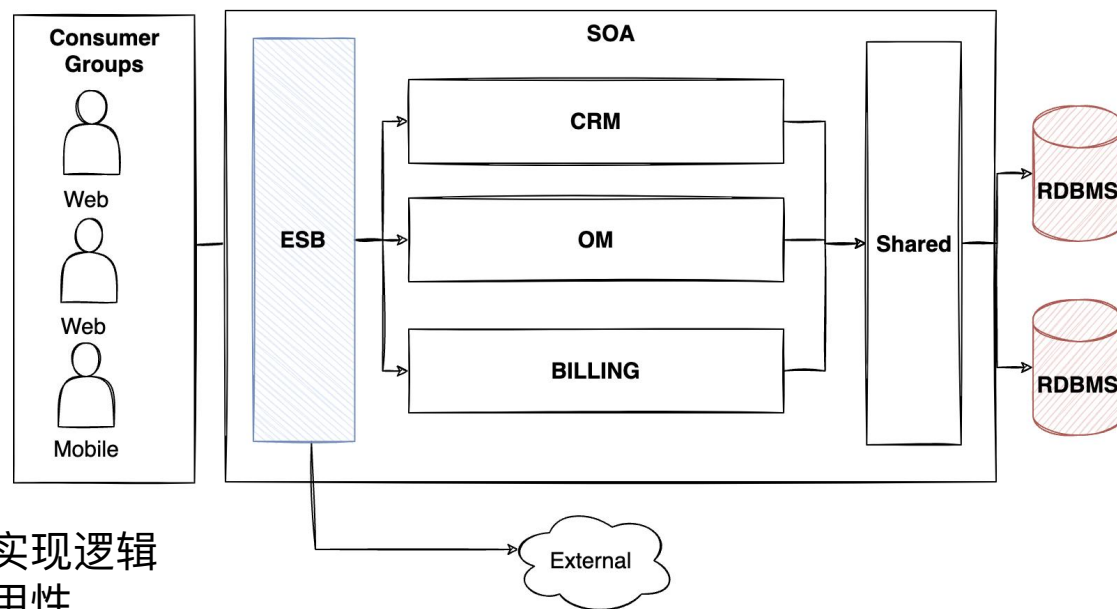
• 面向服务架构的特点：

- + 服务自身**高内聚**、服务间**松耦合**，最小化开发维护中的相互影响
- + 良好的**互操作性**，符合开放标准
- + 模块化，**高重用性**
- + 服务动态**识别**、**注册**、**调用**
- 系统**复杂性**提高
- 难以**测试**验证
- 各独立服务的**演化**不可控
- 中间件易成为**性能**瓶颈

• 面向服务架构实现原则：

- 服务**解耦**：服务之间的关系最小化，只是互相知道接口
- 服务**契约**：服务按照描述文档所定义的服务契约行事
- 服务**封装**：除了服务契约所描述内容，服务将对外部隐藏实现逻辑
- 服务**重用**：将逻辑分布在不同的服务中，以提高服务的重用性
- 服务**组合**：一组服务可以协调工作，组合起来形成定制组合业务需求
- 服务**自治**：服务对所封装的逻辑具有控制权
- 服务**无状态**：服务将一个活动所需保存的资讯最小化

SOA架构示例（某电商应用）

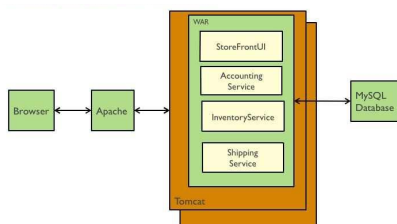




南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

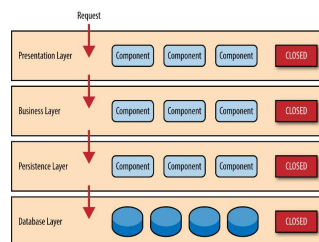


主流架构风格



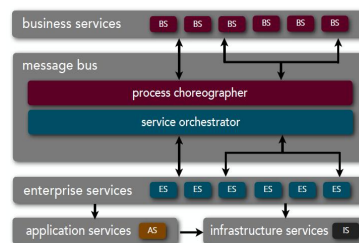
单体架构

- 最简单的架构风格
- 完全封闭
- 代码统一部署
- 紧耦合“大泥球”
- 业务发展初期



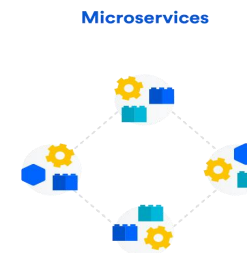
分层架构

- 抽象和分层
- 结构化思考
- MVC三层架构
- 紧耦合“重复造轮子”
- 业务快速增长



SOA架构

- 面向服务架构
- 重量级消息通信机制
- ESB企业服务总线
- 共享数据库
- 集中式服务解决方案



微服务架构

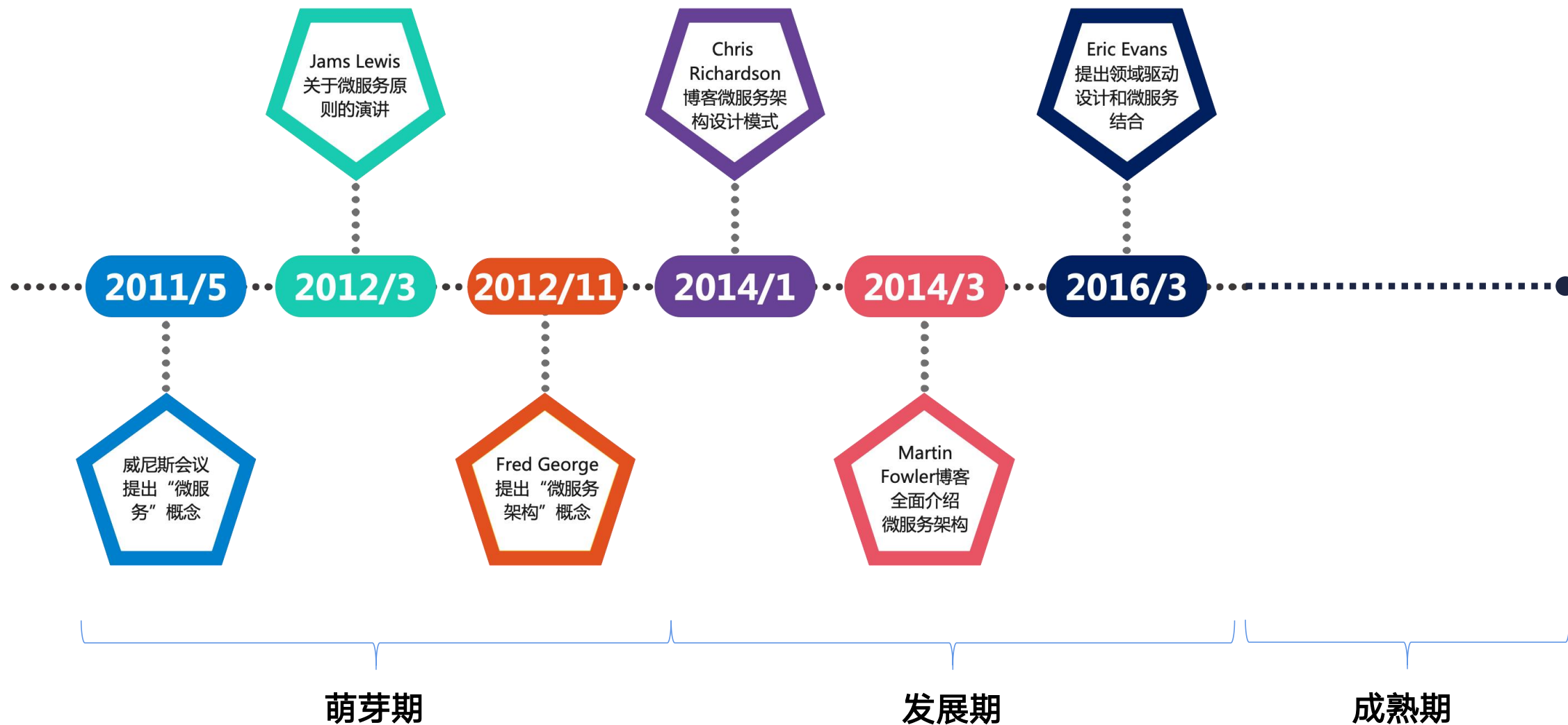
- 服务解耦组件化
- 轻量级通信机制
- 非共享数据库
- 独立部署运行
- 独立扩展伸缩
- 去中心化协同管理



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 发展历程

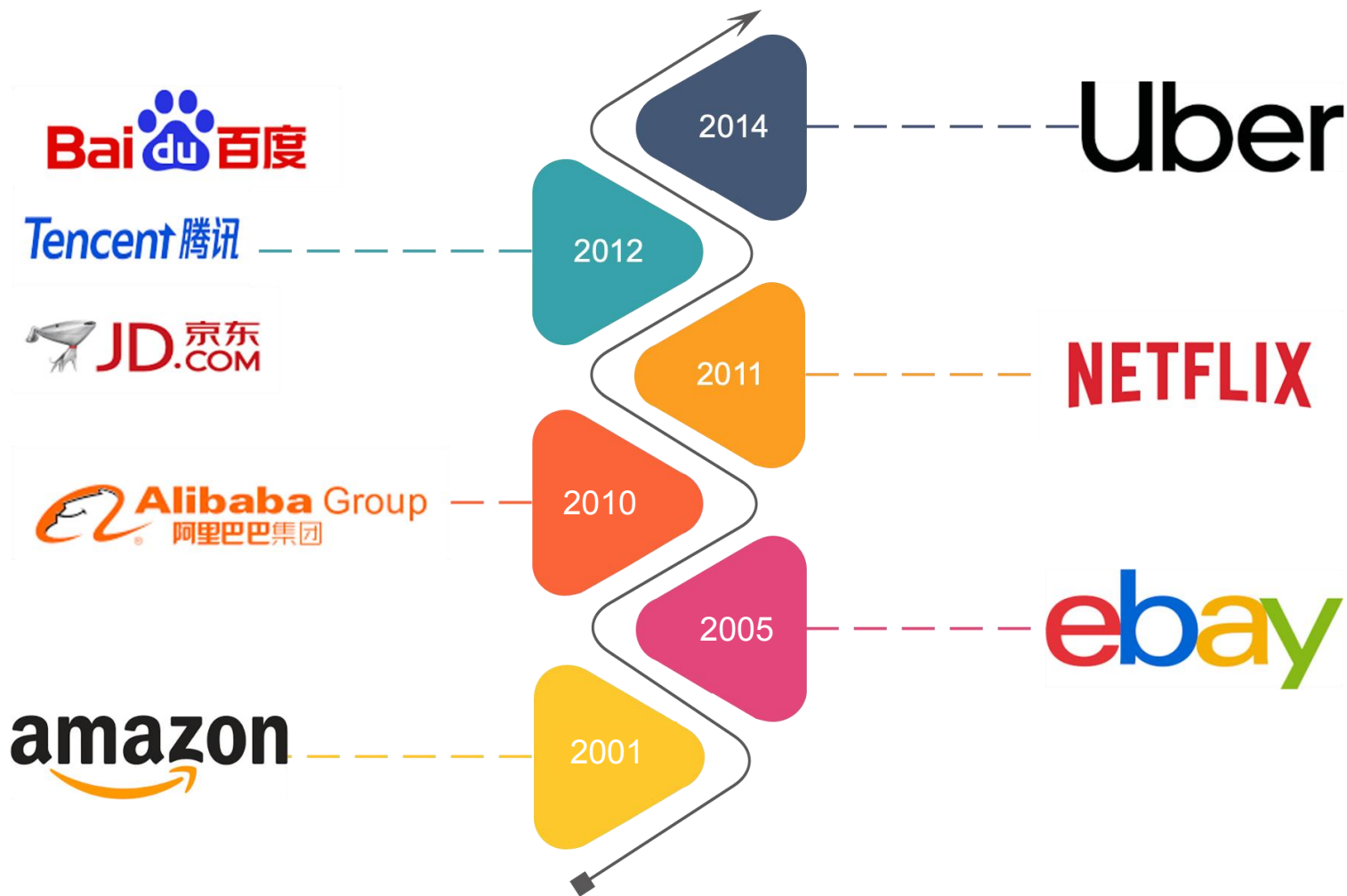




南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 发展历程





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 发展历程

O'REILLY®

TEAMS ▾ INDIVIDUALS FEATURES ▾ BLOG CONTENT SPONSORSHIP 🔍

SIGN IN

Try Now >

< Press releases

O'Reilly's Microservices Adoption in 2020 Report Finds that 92% of Organizations are Experiencing Success with Microservices

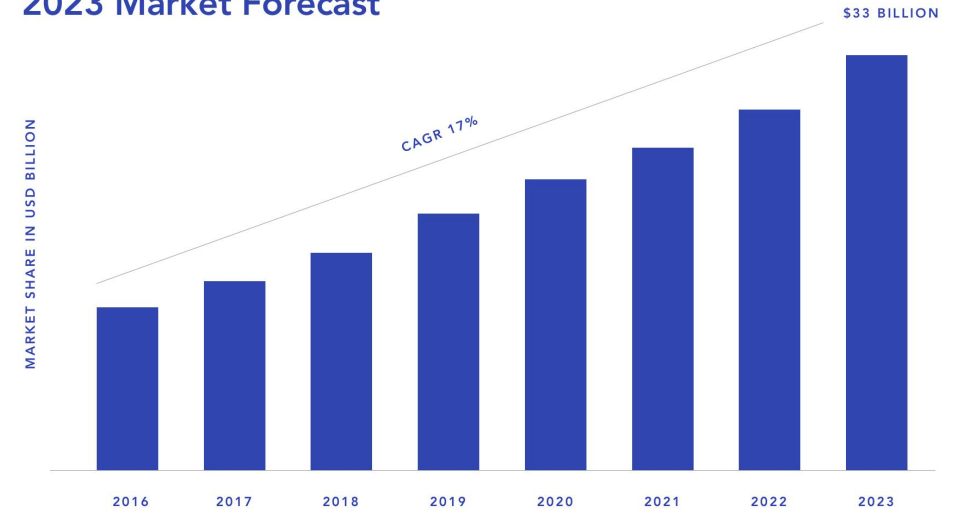
Press release: July 16, 2020

77%



1502

Microservices Architecture 2023 Market Forecast



Microservices Architecture Market Research Report:
Global Forecast 2023, Market Research Future, 2020

prime^{ts}r



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 定义

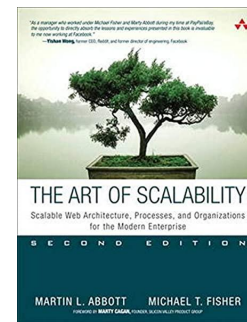
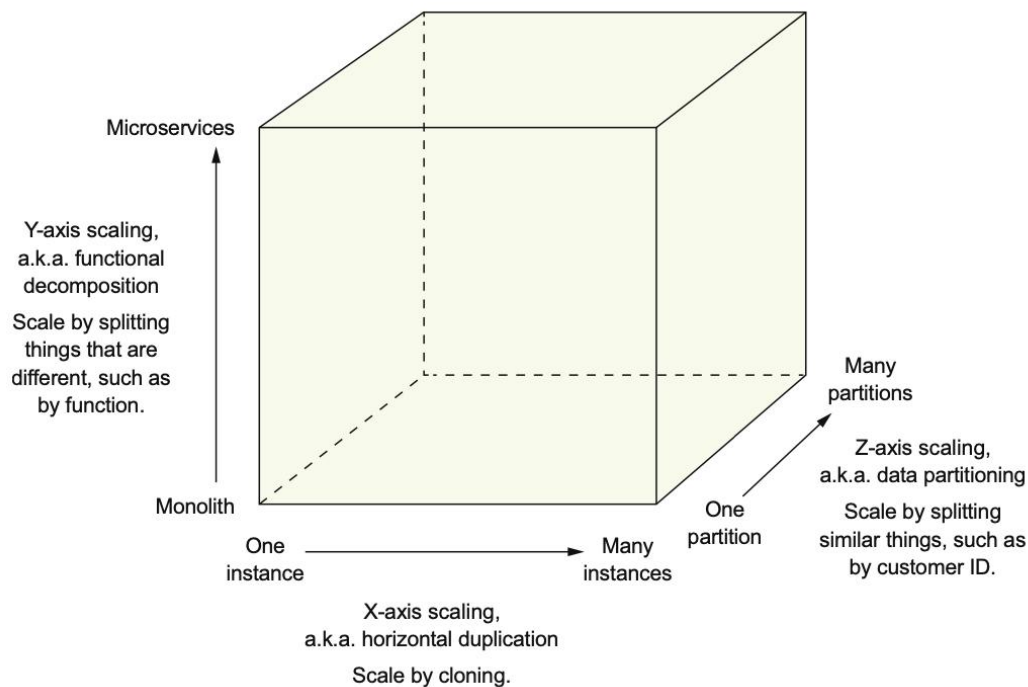
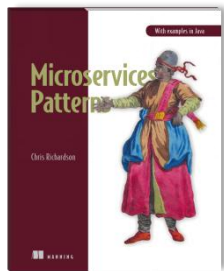
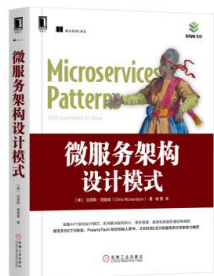
概括性描述：

微服务架构是把应用程序**功能性分解**为一组服务的架构风格，每一个服务都是由一组**专注、内聚**的功能职责组成。



Chris Richardson

<https://microservices.io>





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 定义

“

微服务架构是一种将单体应用拆分为**细粒度**的服务，并使其运行在**独立进程**中，服务之间采用**轻量级通信机制**（如HTTP RESTful API）进行交互的架构风格。这些服务**围绕系统的业务能力构建**，且可以通过全自动的部署机制进行**独立部署**。服务可以进行**分布式管理**，从而支持**不同的编程语言**进行开发和**不同的数据存储技术**进行存储。

FOWLER M, LEWIS J. *Microservices - A definition of this new architectural term*[EB/OL]. 2014.
<https://martinfowler.com/articles/microservices.html>.

”



Microservices Guide

In short, the microservice architectural style is an approach to developing a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API. These services are **built around business capabilities and independently deployable** by fully automated deployment machinery. There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies.

-- James Lewis and Martin Fowler (2014)

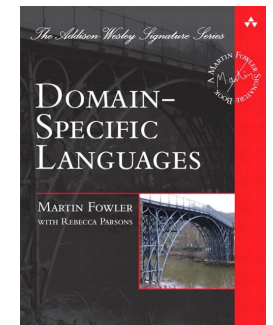
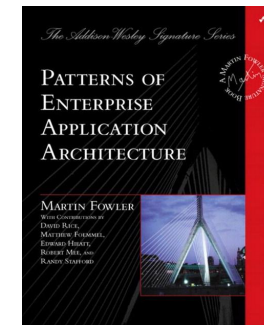
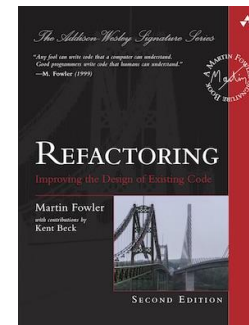
A guide to material
on martinfowler.com
about microservices.

Martin Fowler

21 Aug 2019



Martin Fowler



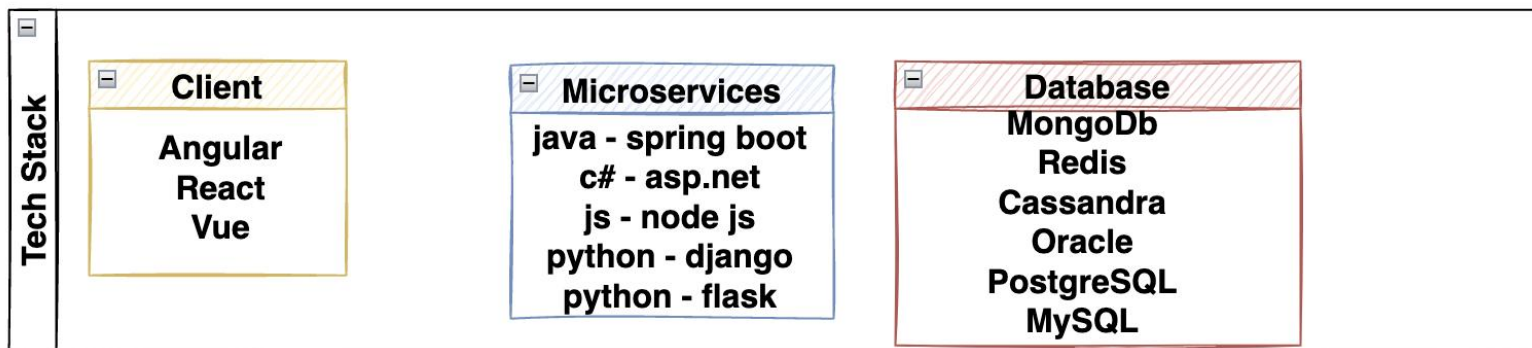
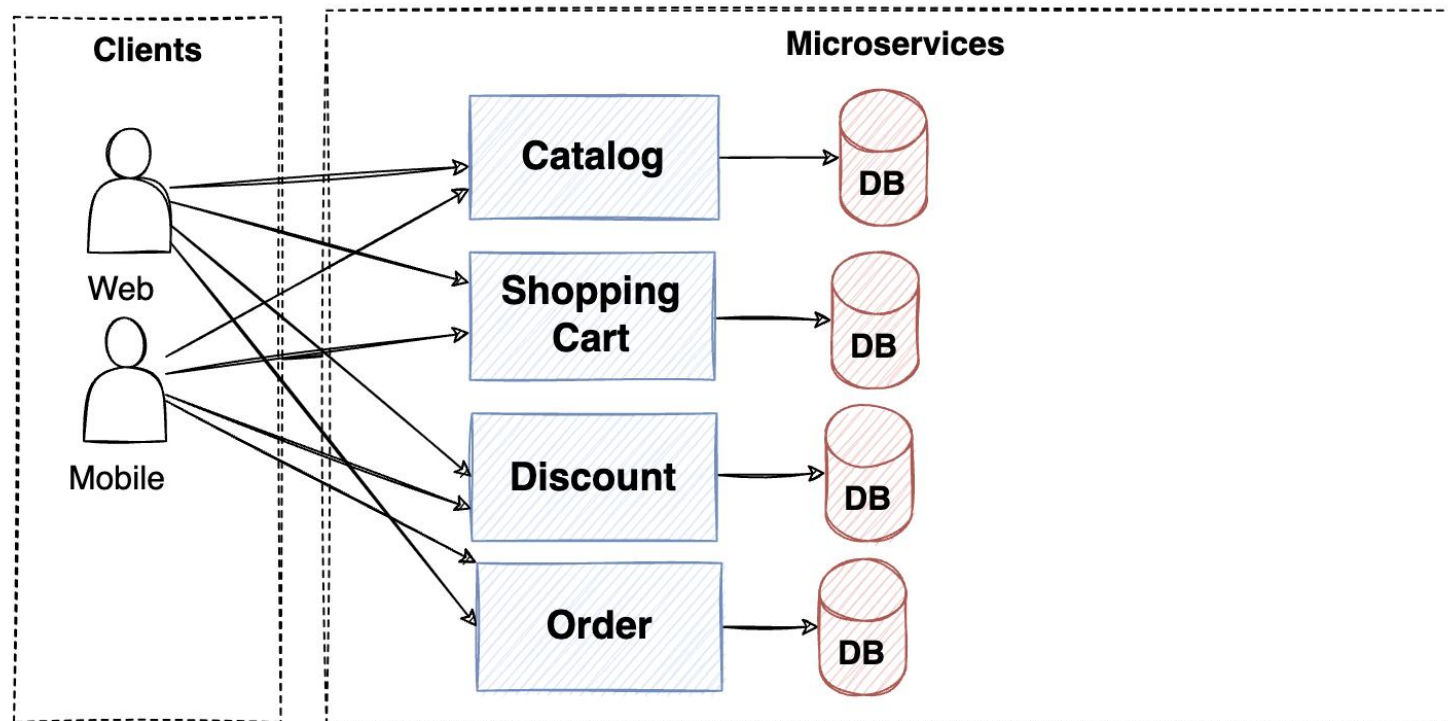


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 定义

微服务架构示例1 (某电商应用)



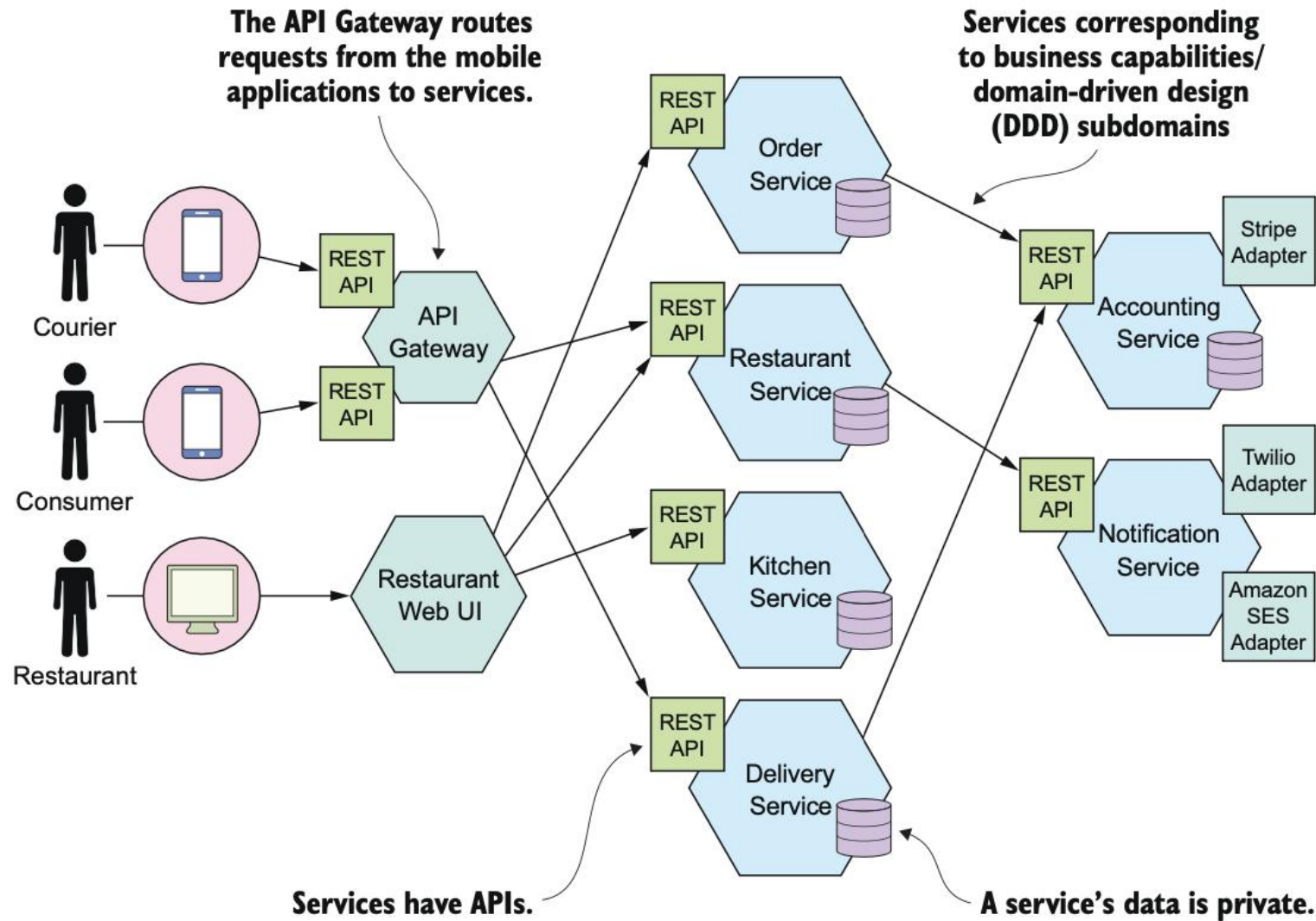


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 定义

微服务架构示例1 (FTGO应用)

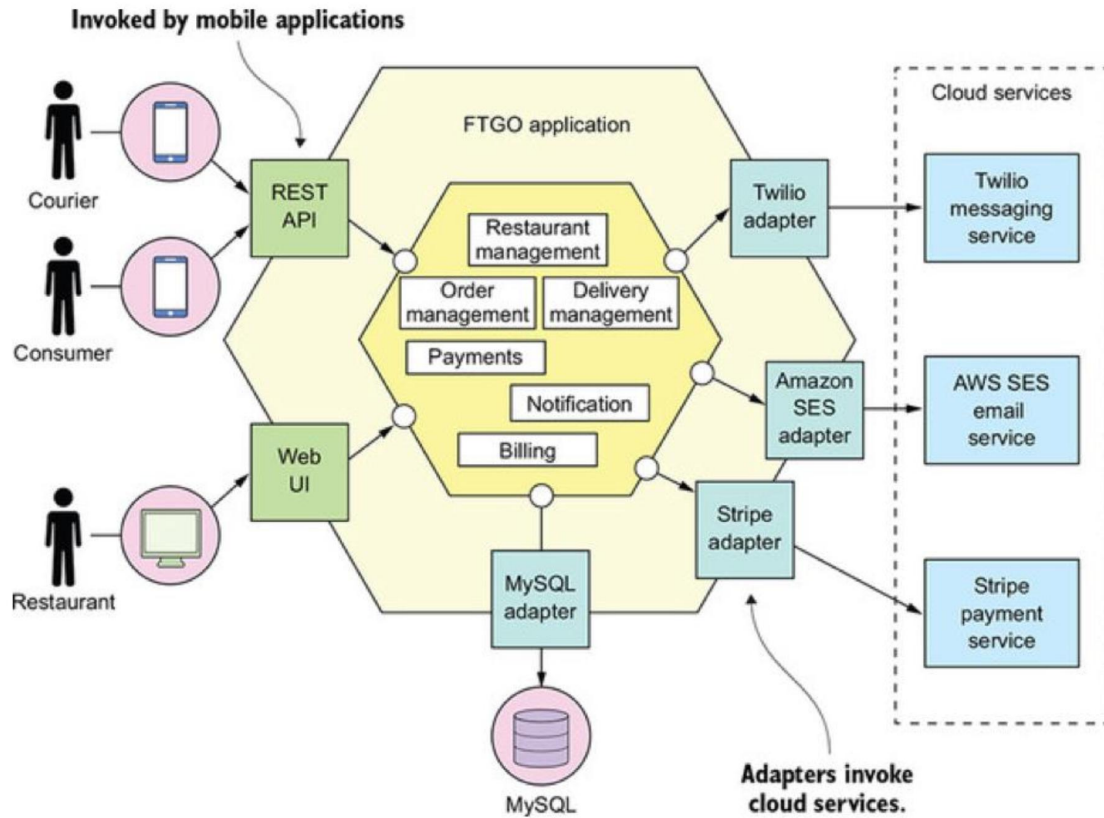




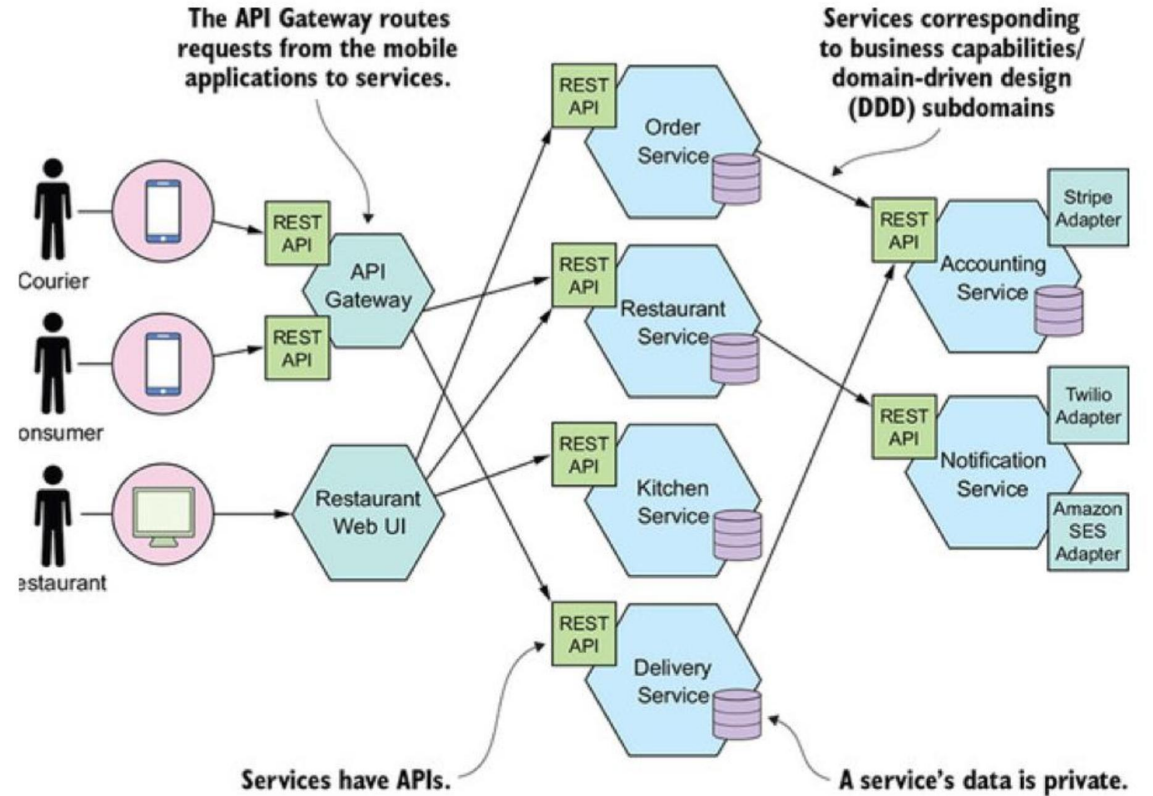
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 定义



单体架构
(FTGO应用)



微服务架构
(FTGO应用)



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性

1. 通过服务组件化

- 组件：可以独立替换和升级的软件单元
- 进程内组件
 - 类、对象或库的形式
 - 一般直接调用、以内存方式进行功能调用（共享内存）
- 进程外组件
 - 微服务架构中的独立服务
 - 实现组件化的方式是分解成服务
 - 通过Web服务请求或 RPC机制通信
 - 轻量级消息传递机制（如RabbitMQ）
 - 产生明确的组件发布接口，封装（区别于文档）
 - 耦合度低，隔离性好、独立开发、部署
 - 远程调用性能损耗、合适的API粒度





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性

2. 围绕业务能力组织

- 传统软件系统开发管理通常聚焦在技术层面
 - UI团队、服务逻辑团队、数据库团队...
 - 跨团队的沟通、交接和预算审批等
- 采用围绕业务能力的划分方法
 - 服务业务领域内的宽栈实现
 - 团队跨职能、全方位开发技能
 - 如用户体验、数据库、项目管理.....
- 采用产品开发模式
 - 传统：项目模式,开发-维护，开发完解散
 - 开发团队负责软件的整个产品周期
 - 持续关注软件如何帮助用户提升业务能力，实现价值交付




**YOU
BUILD IT
YOU
RUN IT**



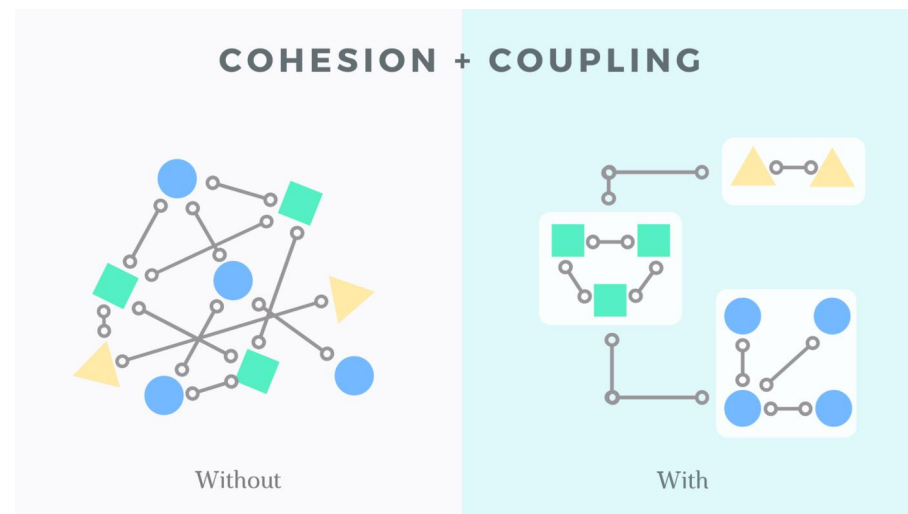
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性

3. 内聚和解耦

- 内聚：单一职责，有各自的领域逻辑
- 解耦：微服务间尽量减少直接依赖，独立自主
 - 服务边界的确定（划分）有助于澄清和强化分离
 - 业务功能分解：每个微服务负责独立的业务能力
 - 领域驱动设计：通用领域划分为多个子域，识别限界上下文（Bounded Context）-服务边界





南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性

4. 去中心化

- 去中心化治理

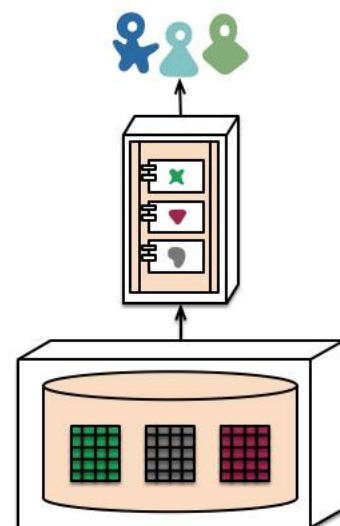
- 构建微服务时可以有服务自己的技术栈选择
- 服务之间只需约定接口，无需关注内部实现
- 运维只需了解服务部署规范

- 去中心化数据存储

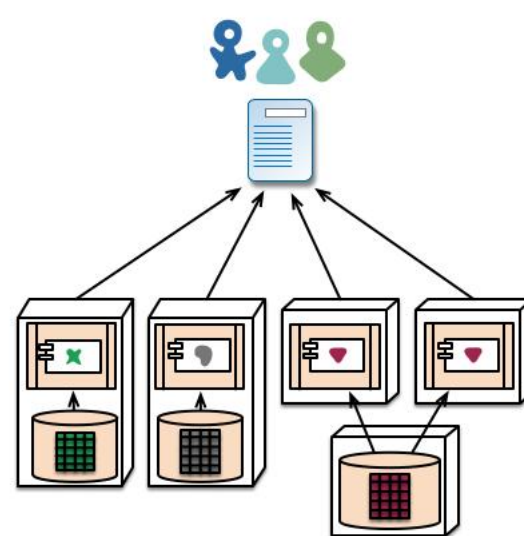
- 让每个微服务管理自己的数据库
- 或同一数据库技术的不同实例
- 或完全不同的数据库系统

- 去中心化数据管理

- 传统架构采用事务保证数据一致性，分布式微服务架构中数据管理困难
- 强调服务间的无事务协作，最终一致性和补偿策略
- 需权衡更大一致性的业务损失与修复错误的代价



monolith - single database



microservices - application databases



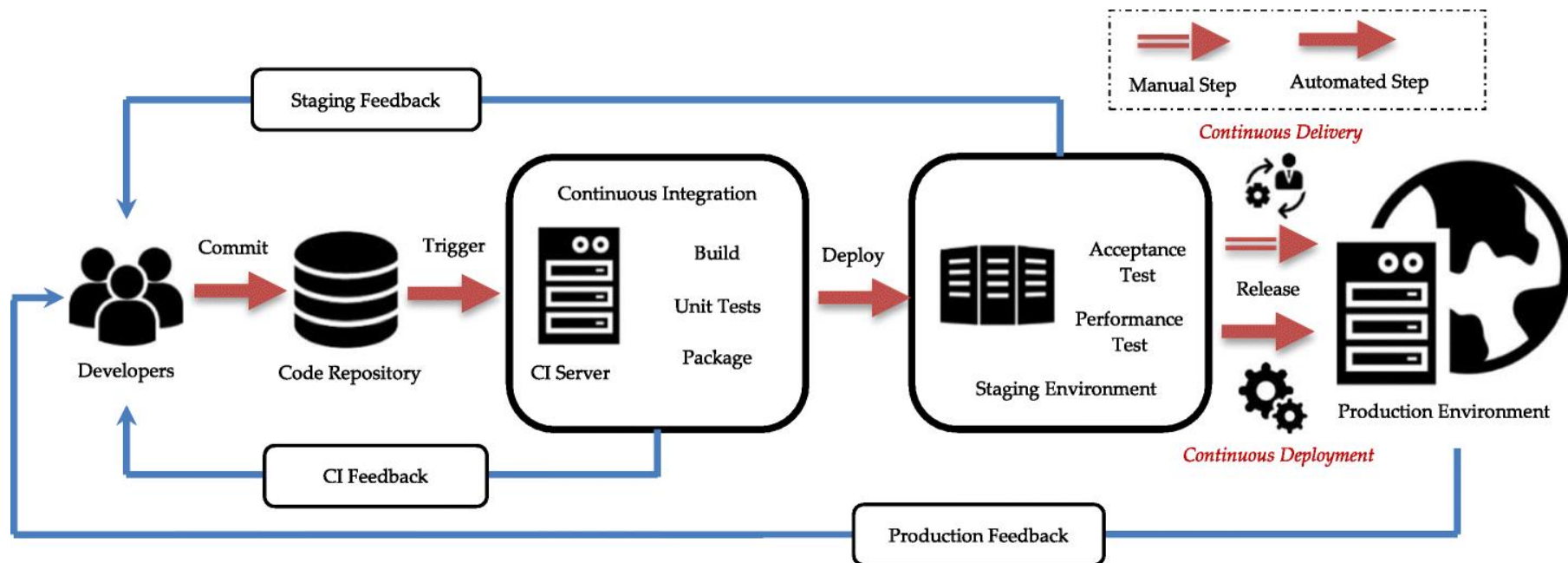
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性

5. 基础设施自动化

- 依赖自动化的基础设施，降低开发和运维微服务的操作复杂度
- 持续部署和交付：编码、管理代码库、集成（构建、测试、打包）、部署、监控和运维
- 测试：单元测试、集成测试、组件化测试、契约测试和端到端测试等

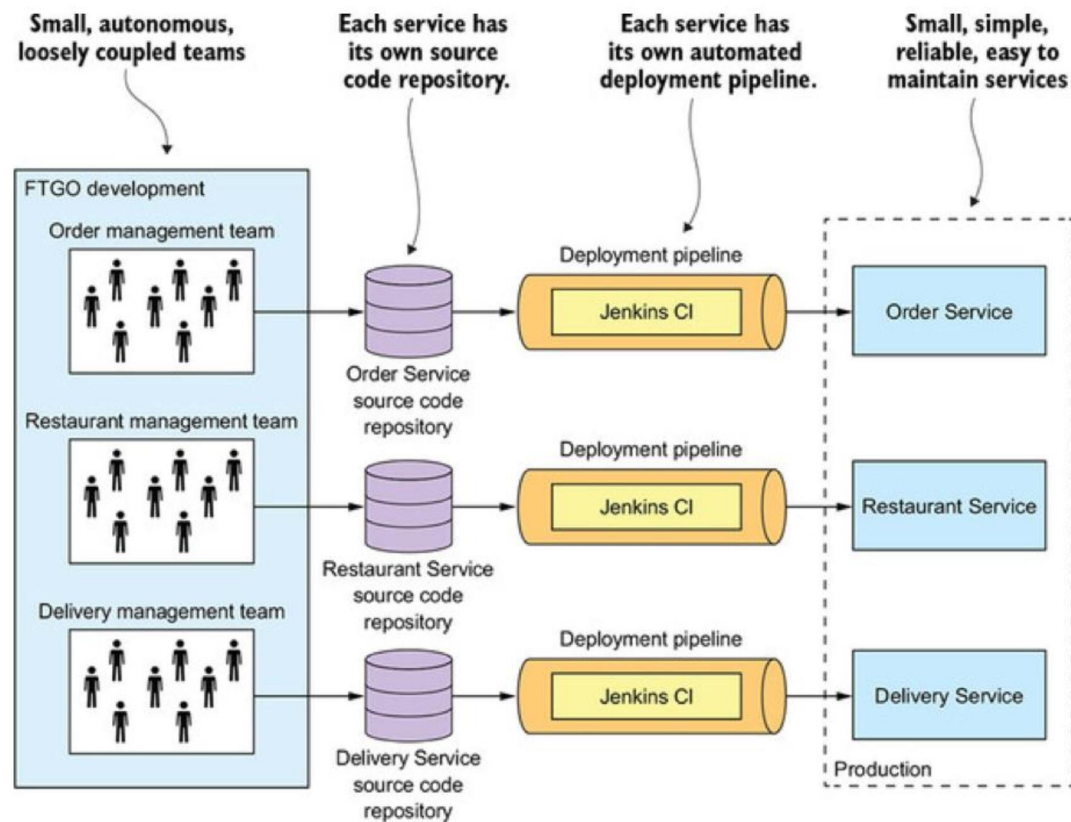
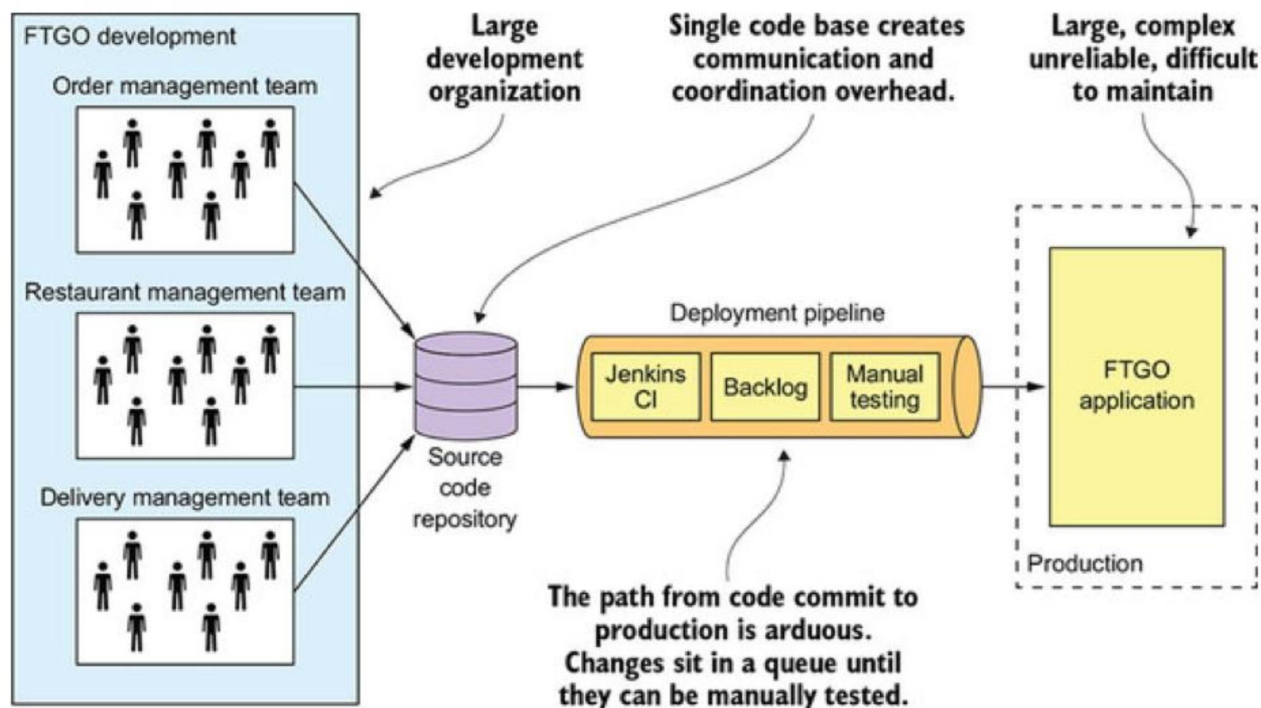




南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性





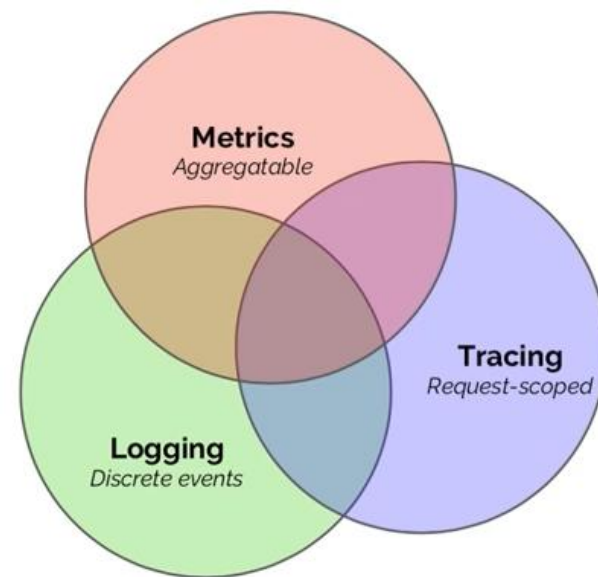
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



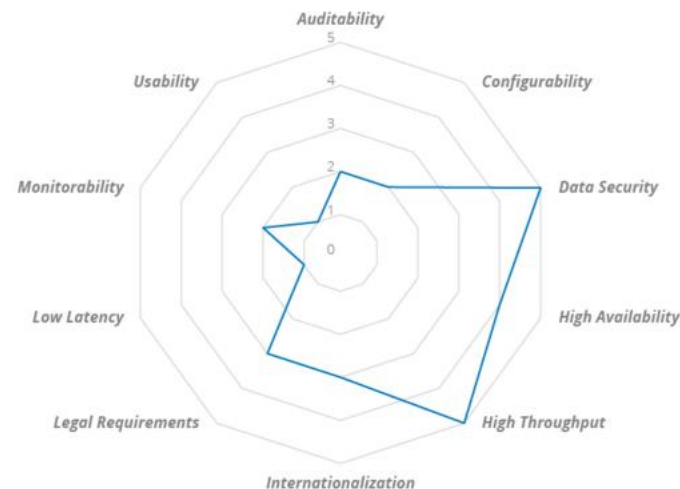
微服务架构 || 主要特性

6. 服务设计与演进

- 高可用设计
 - 容忍服务失败，客户端须尽可能有效地做出响应
 - 完善的监控和日志记录，架构元素或业务指标、链路追踪
 - 快速发现不良突发行为并尽早修复
- 演进式设计
 - 传统架构软件变更难以预测且改造成本高昂
 - 合理设计实现频繁、快速且控制良好的增量变更和演化
 - 合适的服务解耦，只需重新部署修改的服务
 - 变更频率不同，拆分（相同，合并）
 - 架构适应度函数（Architectural fitness function）



FITNESS FUNCTION FIT





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构 || 主要特性



高可伸缩

系统中的某个微服务遇到了瓶颈，根据业务请求，增加或减少资源如内存、CPU或节点，提高资源利用率



每个服务相对较小并容易维护

应用被分解为多个可管理的分支或服务，每个服务都有一个用RPC或者消息驱动API定义清楚的边界



技术栈不受限

开发者可以自由选择开发技术，提供API服务，针对项目开始时采用的过时技术，可以选择现在的技术



服务可以独立扩展

每个服务之间松耦合，功能扩展性高，扩展后的服务可快速上线



大型复杂程序可以持续部署和交付

可测试性、可部署性、团队自主且松散耦合，持续化部署和交付成为可能



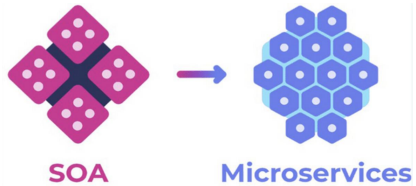
- 设计复杂性
- 网络延迟
- 联调
- 故障定位和修复
-



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



架构风格对比



SOA (Service-Oriented Architecture) = MSA (Microservice Architecture)?

	SOA	MSA
服务间通信	1. 智能管道，如 (ESB) 2. 重量级协议，如SOAP或WS * 标准	1. 哑管道，如消息代理，或服务之间点对点通信 2. 轻量级协议，如 REST或gRPC
数据管理	全局数据模型并共享数据库	每个服务有自己的数据模型和数据库
典型服务规模	较大的单体应用	较小的服务



南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

02

微服务设计核心模式

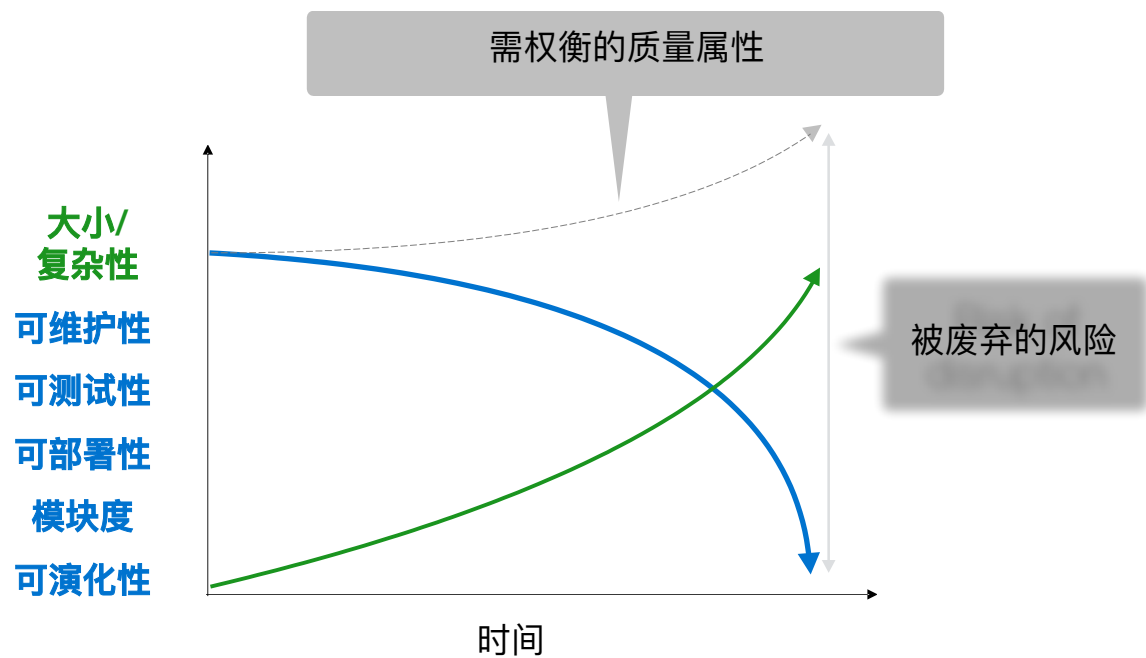
- 迁移动机
- 挑战问题
- 模式和模式语言
- 微服务拆分设计
- 其他核心模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



迁移动机



单体地狱:

单体架构并不是反模式，但：

- 过度的复杂性会吓退开发者
- 从代码到实际部署周期长，且容易出问题
- 难以扩展
- 难以实现可靠交付
- 需要长时间依赖某个可能已经过时的技术栈



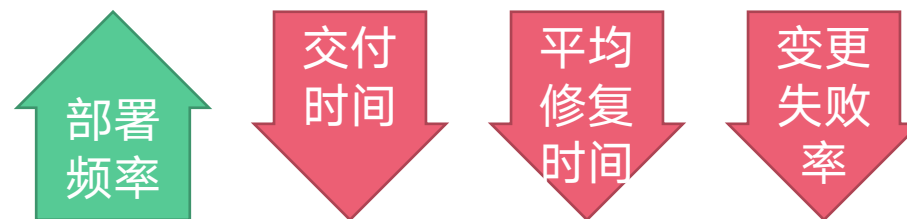
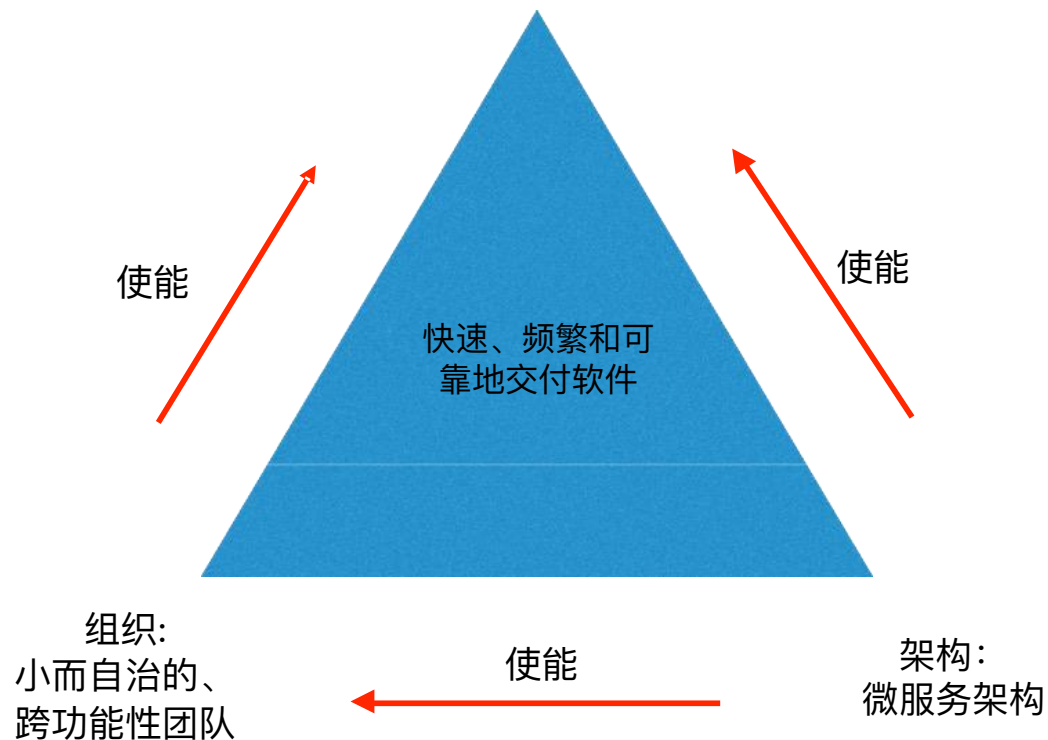


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



迁移动机

流程：DevOps/持续交付/持续部署



微服务架构直接支持DevOps/持续交付/持续部署

- 拥有DevOps所需要的可测试性（服务较小）
- 拥有DevOps所需要的可部署性（独立部署）
- 使得开发团队能够自主松散耦合（两个披萨团队）



南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



挑战问题

❑ 服务的拆分和定义（粒度问题）

- 如何拆？
- 结果优劣？

微服务架构不是一颗银弹！

❑ 分布式系统带来的复杂性

- 进程间通信机制复杂性高于方法调用、局部故障
- 跨服务的事务和查询（Saga维护数据一致性、API组合或CQRS视图从多个服务中检索数）
- 编写包含多项服务在内的自动化测试
- 运维复杂性（自动化部署工具、产品化PaaS平台、Docker容器编排平台）
- 部署跨服务的功能需要协调更多开发团队



超过 60% 的受访者提到，其在实践中缺乏系统化的服务拆分指导，相关决策主要基于经验给出。

谨慎决策、权衡取舍 => 微服务架构模式语言



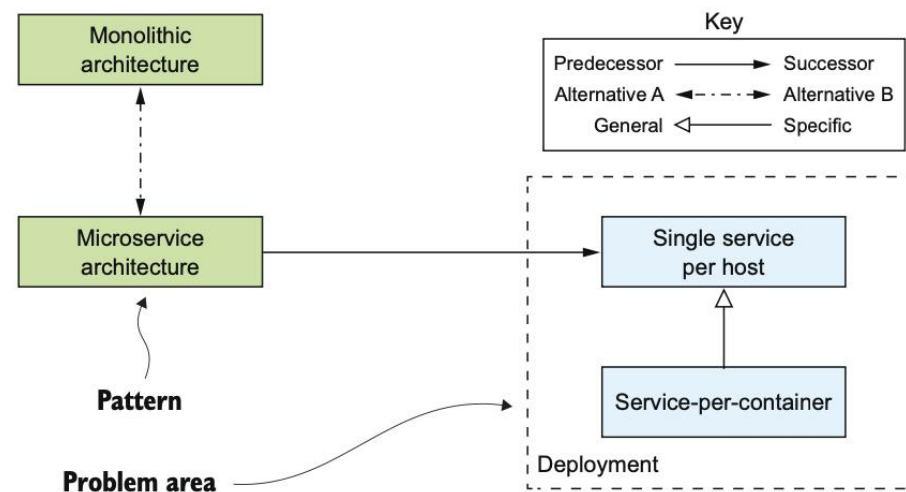
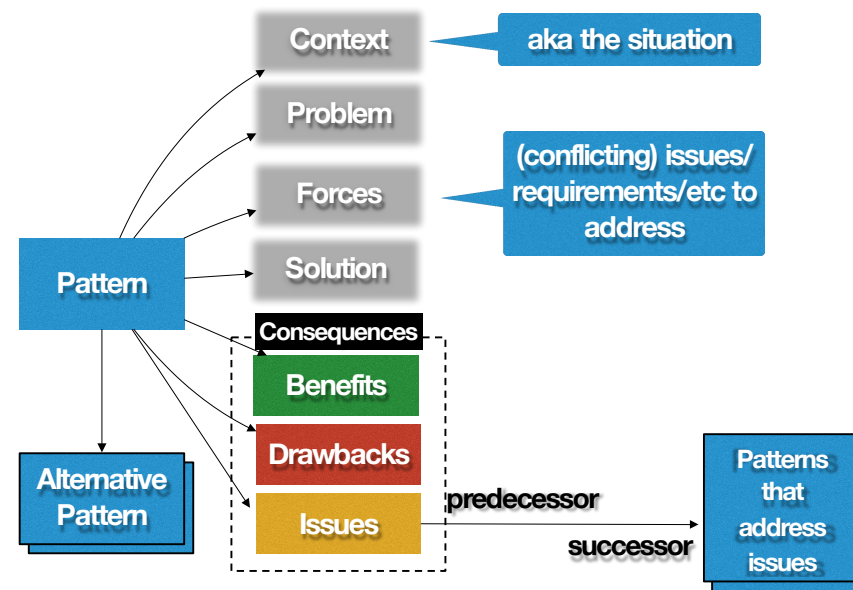
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



模式和模式语言

模式：针对**特定上下文**中发生的**问题**的**可重用解决方案**。

- 上下文（Context）：问题特定的场景
- 问题（Problems）：需要解决的问题
- 需求（Forces）：需求、优先级和其他约束
- 方案（Solution）：可重用的解决方案
- 结果（Consequences）：采用模式后的后果
 - 好处
 - 弊端
 - 问题
- 相关模式（Related patterns）：与其他模式之间的关系
 - 前导（Predecessor）
 - 后续（Successor）
 - 替代（Alternative）
 - 泛化（Generalization）
 - 特化（Specialization）





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

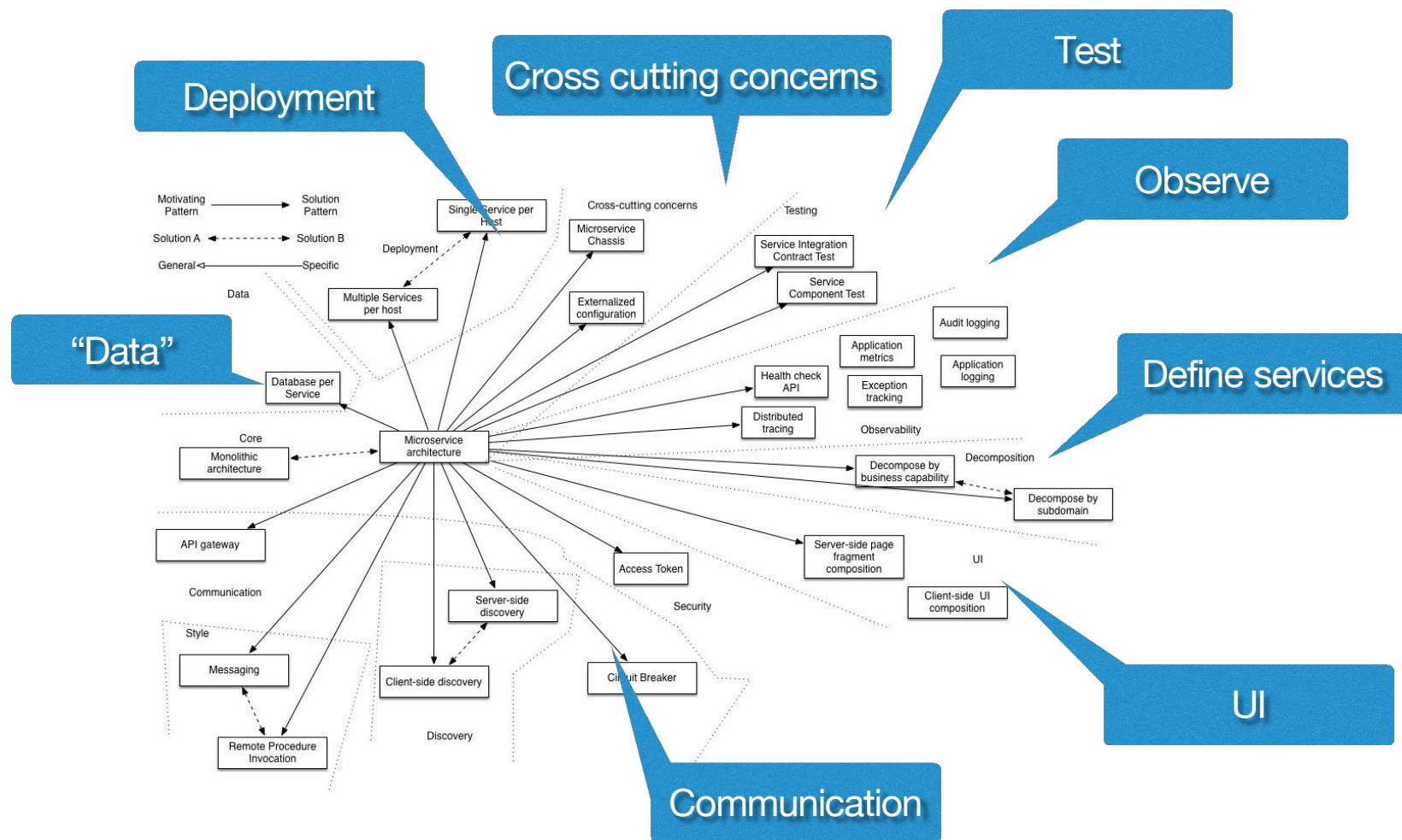


模式和模式语言

微服务架构模式语言（一组模式）

- 帮助解决微服务架构设计问题

- 拆分问题
- 数据问题
- 通信问题
- 部署问题
- 运维问题
-





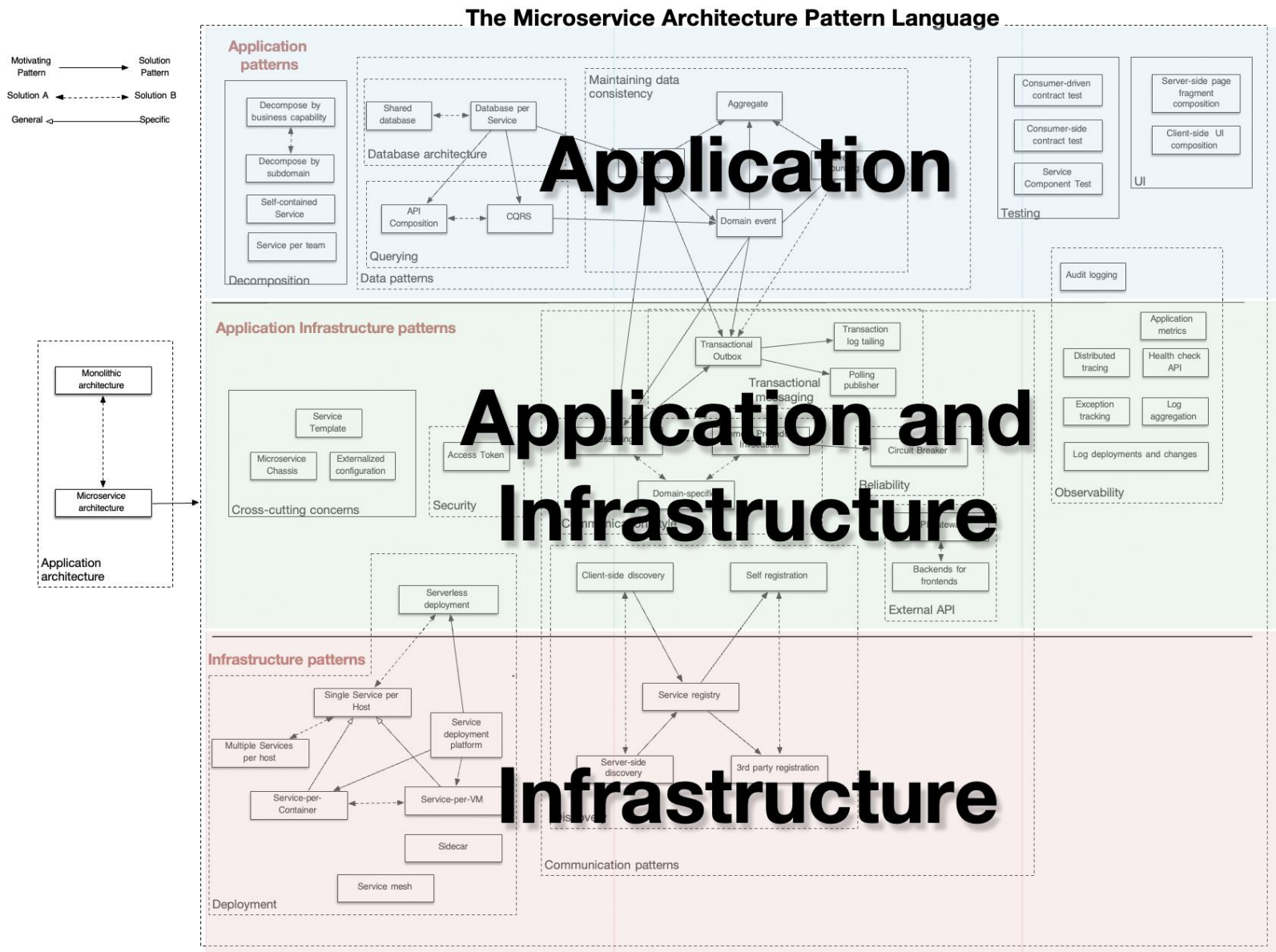
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



模式和模式语言

微服务架构模式语言（一组模式）

- 应用相关模式组
- 应用基础设施相关模式组
- 基础设施相关模式组





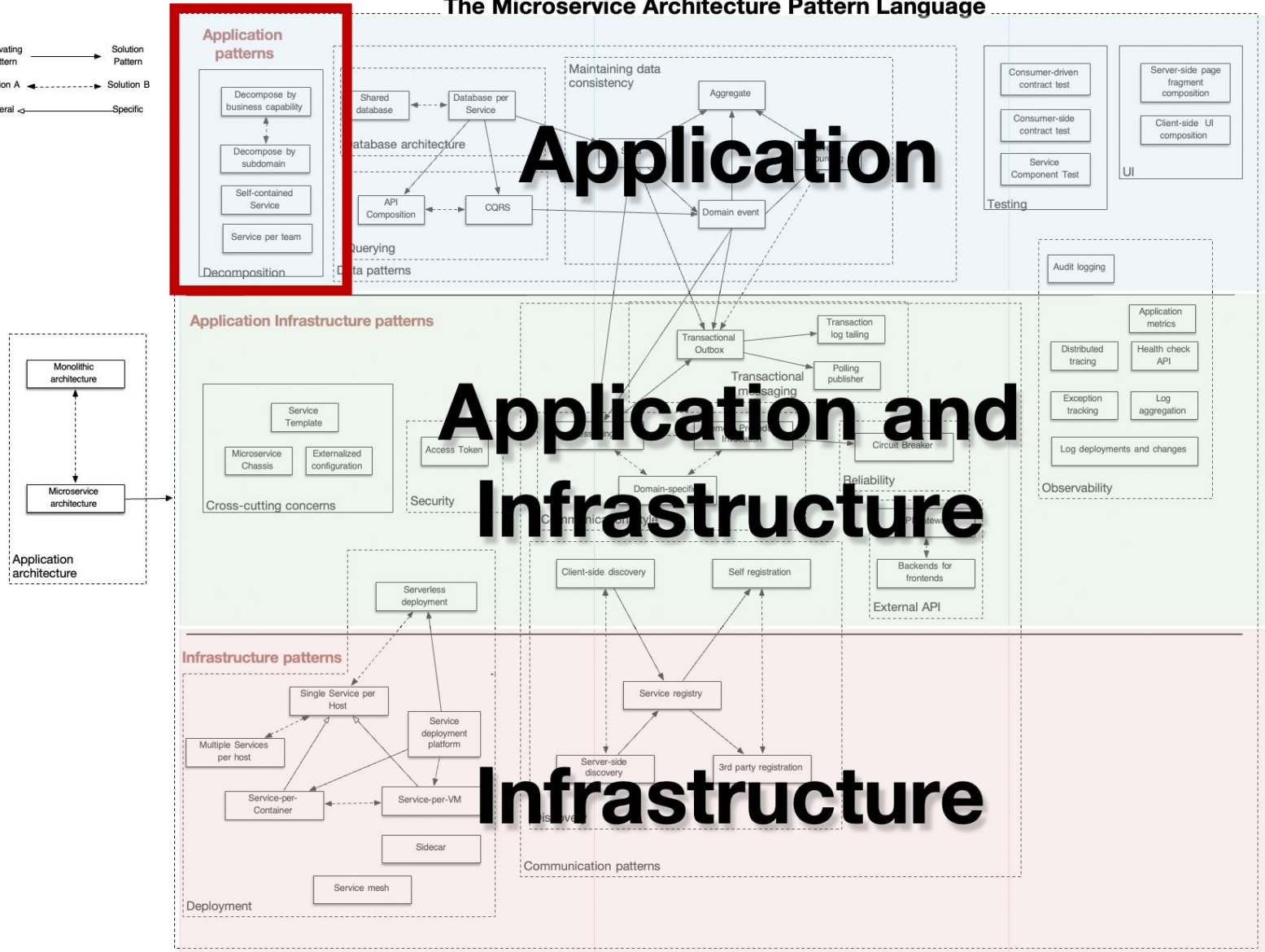
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



模式和模式语言

The Microservice Architecture Pattern Language

Motivating Pattern → Solution Pattern
Solution A ← Solution B
General ← Specific

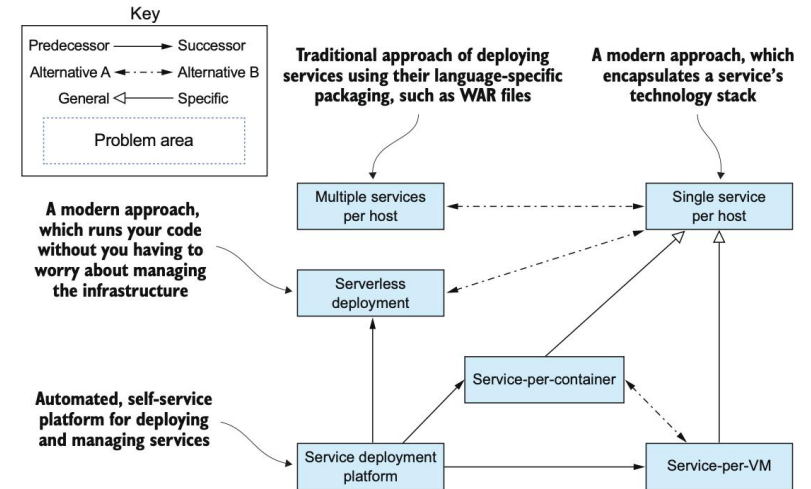
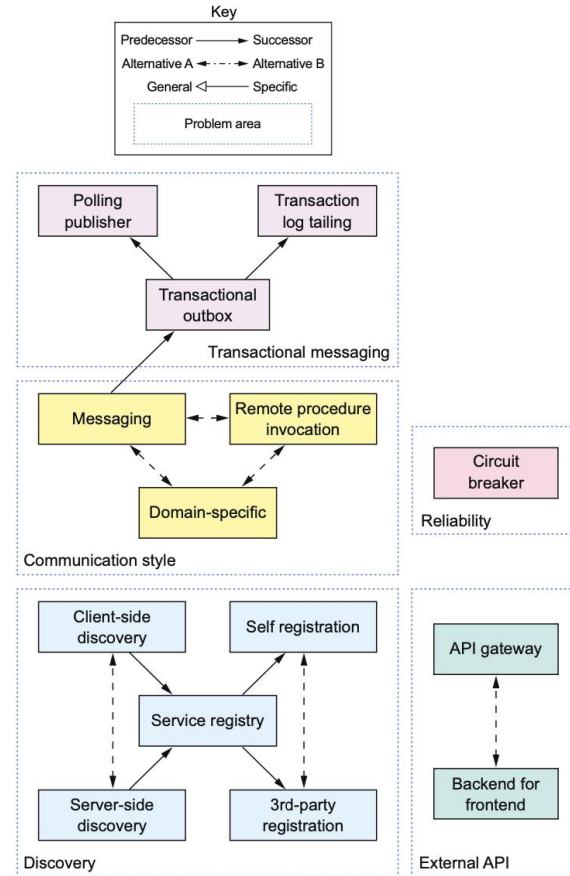
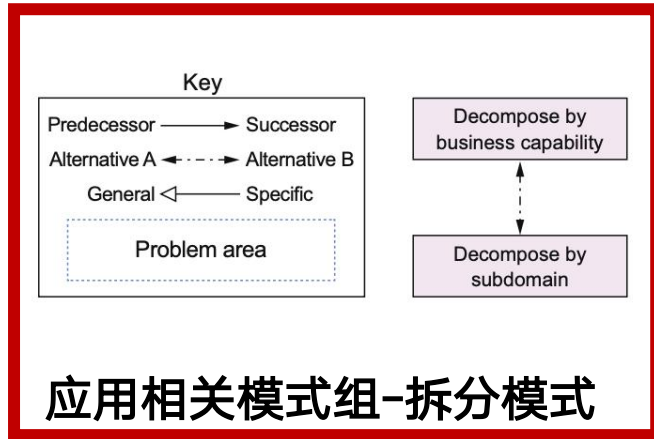




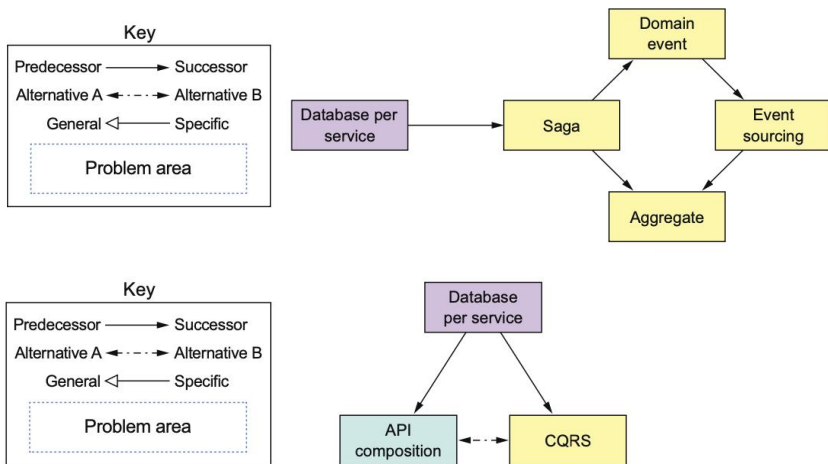
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



模式和模式语言



应用相关模式组-数据模式



应用和基础设施相关模式组-通信模式

基础设施相关模式组-部署模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

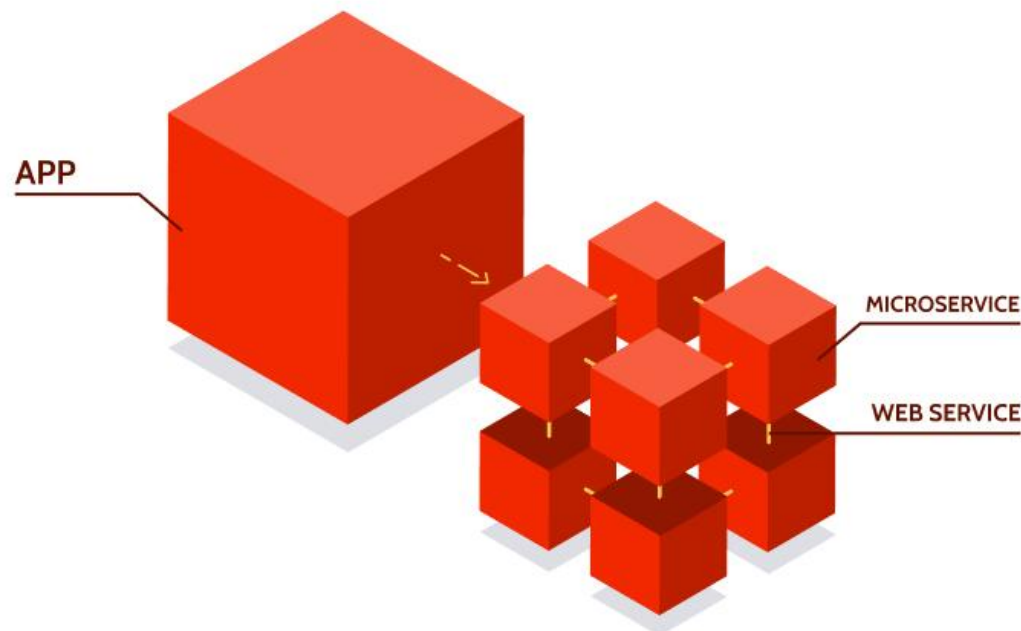


微服务拆分设计

- **问题：**如何将应用拆分为合适粒度的微服务？或如何划分合适的微服务边界？

- **需求：**

- 高内聚：服务实现一组密切相关的功能
- 低耦合：对外封装内部细节，API交互
- 单一职责原则（SRP）
- 共同封闭原则（CCP）
- 双披萨团队开发
- 团队自治
-





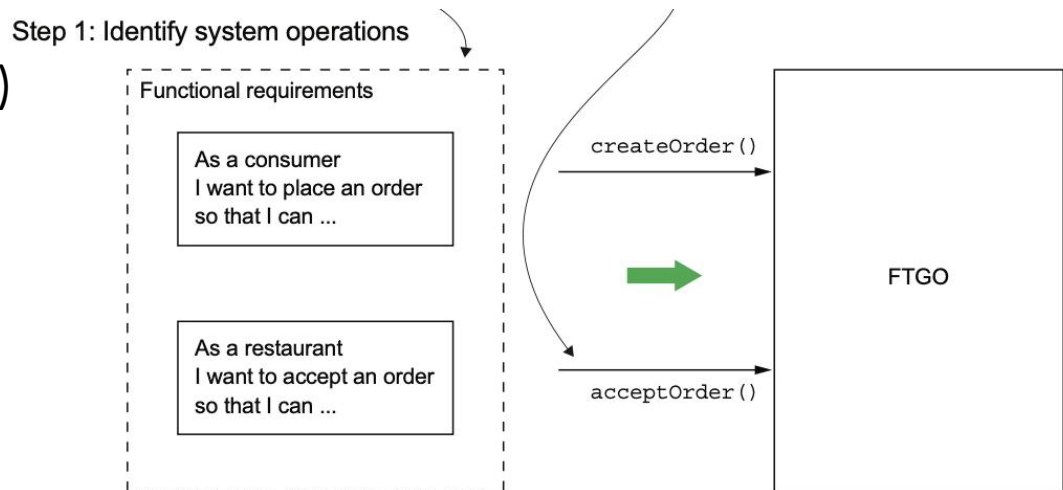
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 关键步骤

1. 定义系统操作

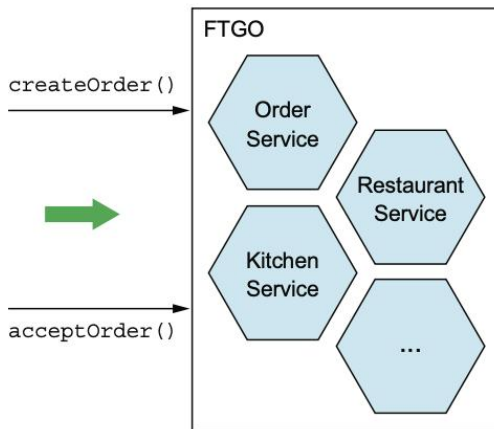
- 将需求提炼为系统必须处理的关键请求（系统操作）
 - 定义架构的第一步
 - 描述服务之间协作方式的架构场景
 - 如更新/检索数据等
- 由抽象的领域模型定义
 - 领域模型从需求派生



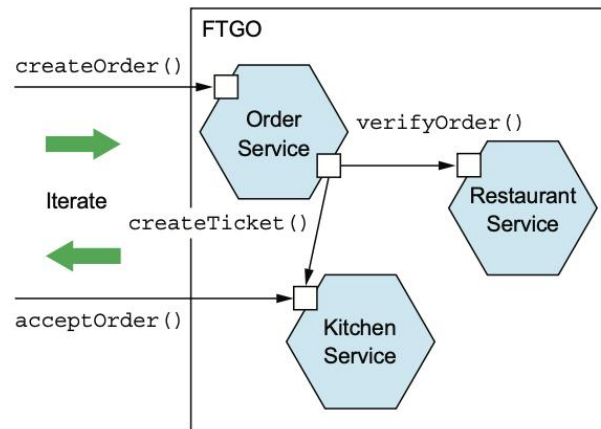
2. 定义微服务（围绕业务概念而非技术）

- 根据业务能力进行服务拆分
- 根据子域进行服务拆分
- 根据动静态调用关系进行服务拆分

Step 2: Identify services



Step 3: Define service APIs and collaborations



3. 定义服务API和协作方式

- 将标识的系统操作分配给服务
- 独立或与其他服务协作（涉及通信方式）实现操作



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 定义系统操作

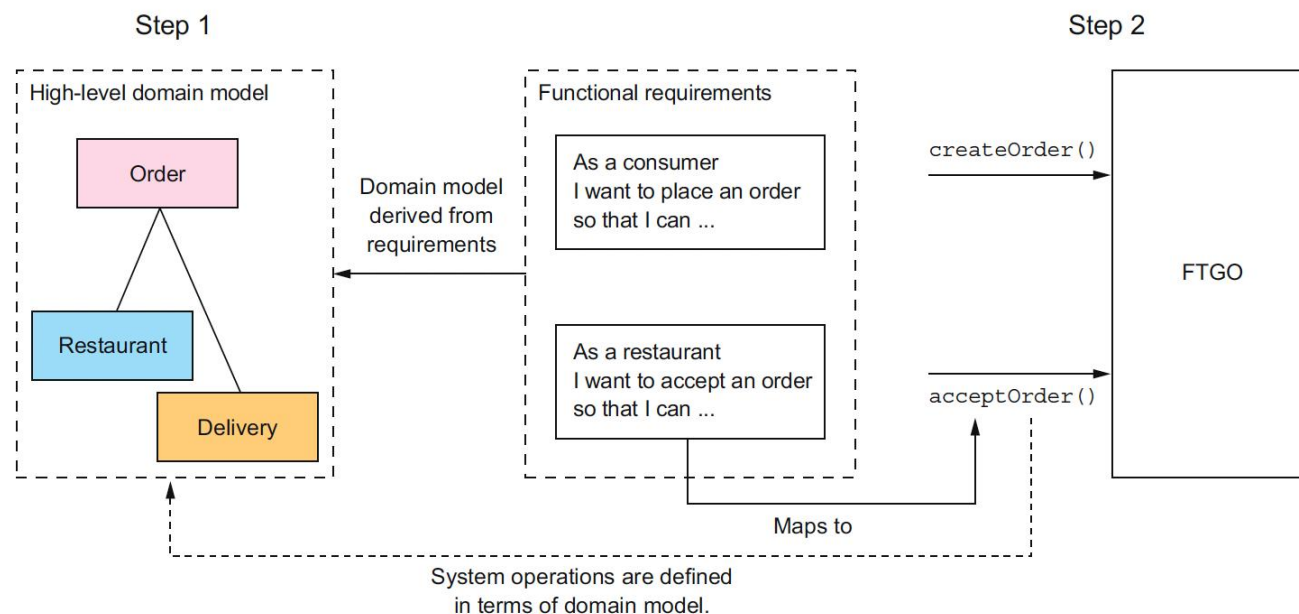
- 输入：需求，用户故事/相关用户场景/源代码恢复等
- 流程：

- 步骤1：创建领域模型

- ⇒ 关键概念/类组成
- ⇒ 关键类是用户故事中的名词
- ⇒ 从输入中提取
- ⇒ 领域专家沟通或事件风暴

- 步骤2：确定系统操作

- ⇒ 用户故事中的动词
- ⇒ 操作行为指对领域对象的影响及之间关系
- ⇒ 创建/删除/更新领域对象，创建/破坏关系





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 定义系统操作

步骤1：分析用户故事中频繁出现的名词，创建（抽象的）领域模型

Place Order用户故事

Given a **consumer**
And a **restaurant**
And a **delivery address/time** that can be served by that **restaurant**
And an **order** total that meets the **restaurant's order minimum**
When the **consumer** places an order for the **restaurant**
Then consumer's **credit card** is authorized
And an **order** is created in the PENDING_ACCEPTANCE **state**
And the **order** is associated with the **consumer**
And the **order** is associated with the **restaurant**

Accept Order用户故事

Given an **order** that is in the PENDING_ACCEPTANCE **state**
and a **courier** that is available to deliver the **order**
When a **restaurant** accepts an **order** with a promise to prepare by a particular time
Then the **state** of the **order** is changed to ACCEPTED
And the **order's promiseByTime** is updated to the promised time
And the **courier** is assigned to deliver the **order**



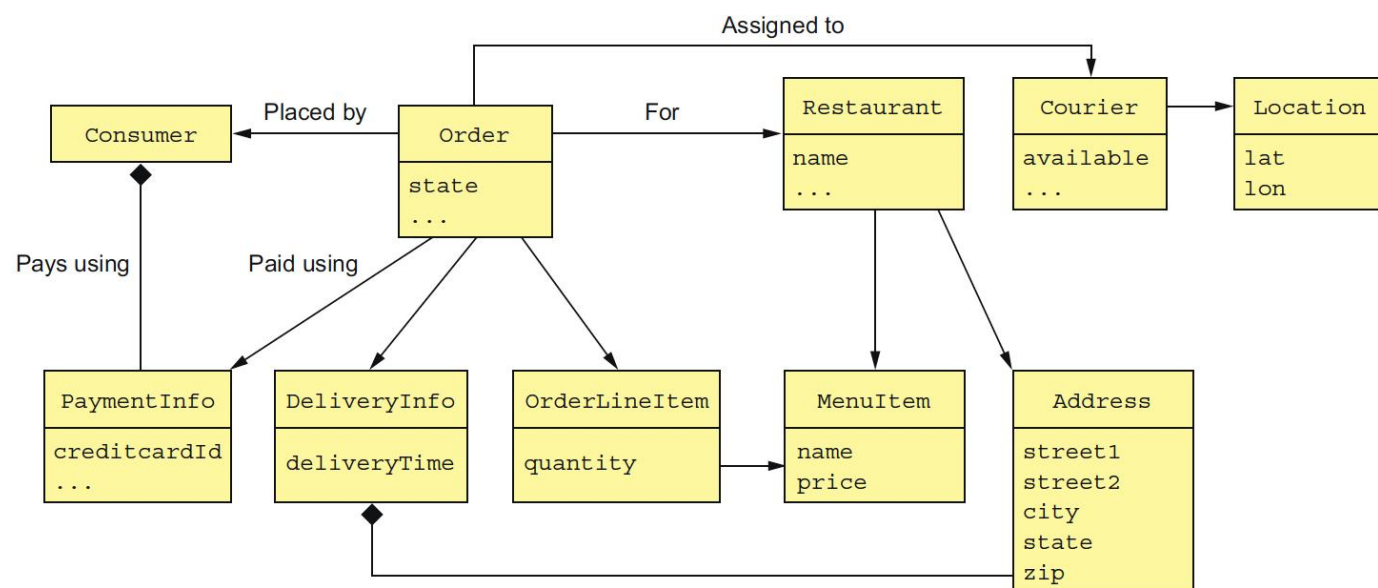
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 定义系统操作

步骤1：分析用户故事中频繁出现的名词，创建（抽象的）领域模型

- Consumer: 下订单的用户
- Order: 订单，描述并跟踪状态
- OrderLineItem: 订单中的一个条目
- DeliveryInfo: 送餐信息（时间、地点…）
- Restaurant: 餐馆，接收制作订单，发起送货
- MenuItem: 餐馆菜单中的一个条目
- Courier: 送餐员，配送，可用性和位置
- Address: Consumer或Restaurant地址
- Location: Courier当前位置，经纬度表示



FTGO领域模型中的关键类



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 定义系统操作

步骤2：分析用户故事和场景中的动词，确定系统操作

- 命令型或查询型（增删改查）
- 用户场景中前端用户的界面向后端业务逻辑发出请求
- 后端业务逻辑进行数据获取和处理
- 系统操作规范
 - 命令对应的参数、返回值
 - 领域模型类的行为：前置和后置条件

Operation	<code>createOrder (consumer id, payment method, delivery address, delivery time, restaurant id, order line items)</code>
Returns	<code>orderId, ...</code>
Preconditions	- The consumer exists and can place orders. - The line items correspond to the restaurant's menu items. - The delivery address and time can be serviced by the restaurant.
Post-conditions	- The consumer's credit card was authorized for the order total. - An order was created in the <code>PENDING_ACCEPTANCE</code> state.

CreateOrder() 系统操作规范

Actor	Story	Command	Description
Consumer	Create Order	<code>createOrder()</code>	Creates an order
Restaurant	Accept Order	<code>acceptOrder()</code>	Indicates that the restaurant has accepted the order and is committed to preparing it by the indicated time
Restaurant	Order Ready for Pickup	<code>noteOrderReadyForPickup()</code>	Indicates that the order is ready for pickup
Courier	Update Location	<code>noteUpdatedLocation()</code>	Updates the current location of the courier
Courier	Delivery picked up	<code>noteDeliveryPickedUp()</code>	Indicates that the courier has picked up the order
Courier	Delivery delivered	<code>noteDeliveryDelivered()</code>	Indicates that the courier has delivered the order

FTGO的重要系统操作命令

Operation	<code>acceptOrder(restaurantId, orderId, readyByTime)</code>
Returns	—
Preconditions	- The <code>order.status</code> is <code>PENDING_ACCEPTANCE</code>. - A courier is available to deliver the order.
Post-conditions	- The <code>order.status</code> was changed to <code>ACCEPTED</code>. - The <code>order.readyByTime</code> was changed to the <code>readyByTime</code>. - The courier was assigned to deliver the order.

AcceptOrder() 系统操作规范



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 根据业务能力拆分

业务能力：业务架构建模的术语

- 为企业产生价值的商业活动，较为**稳定**
 - 保险公司：承保、理赔服务、账务和合规等
 - 在线商店：订单管理、库存管理和发货等
 -
- 业务能力的**实现方式相对不稳定**
 - 银行“兑现支票”：纸质/ATM机/手机等
- 通过分析组织**目标、结构、商业流程**得到
- 一个业务能力往往对应特定的业务对象
- 能力**可分解**为子能力
 - 订单获取和履行的5个子能力

- 供应商管理
 - 送餐员信息管理
 - 餐馆和其他信息管理
- 消费者管理
- 订单获取和履行
 - 创建和管理订单
 - 餐馆管理订单生产过程
 - 送餐
 - 送餐员实时状态管理
 - 配送管理
- 会计记账
 - 管理消费者相关的会计记账
 - 管理餐馆相关的会计记账
 - 管理送餐员相关的会计记账
- 其他

FTGO的业务能力



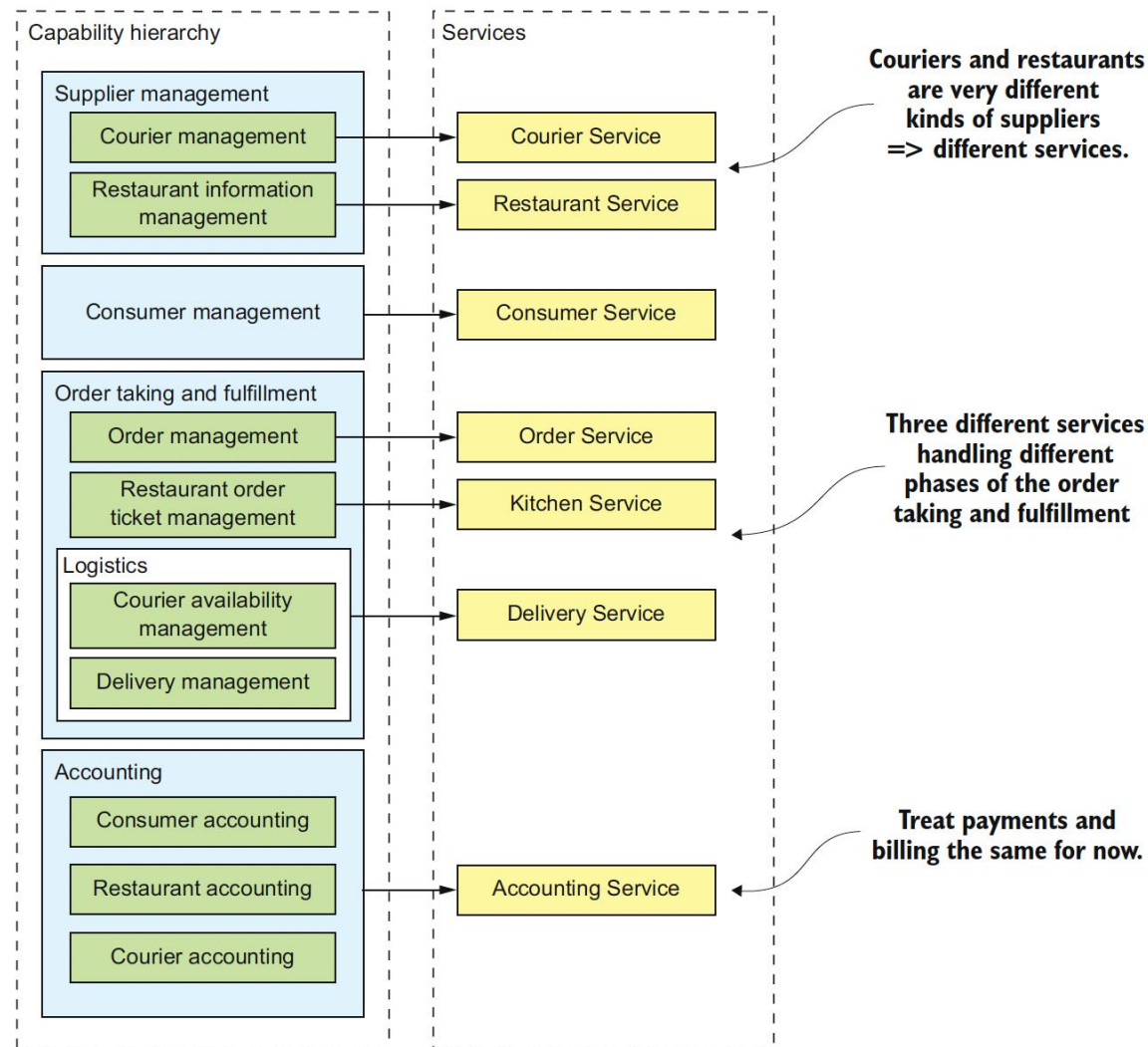
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 根据业务能力拆分

业务能力到服务

- 顶级能力或子能力映射到服务
 - 顶级能力直接映射：
 - 用户服务
 - 会计记账服务（统一处理支付和计费）
 - 子能力分解映射：
 - 供应商管理2个服务（两个不同的供应商）
 - 订单获取和履行3个服务（3个阶段）
- 映射具有一定的主观性
- 分解方式可能会变化（组合或进一步拆分）
 - 通信效率
 - 变更频率
 -



FTGO的业务能力映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 根据业务能力拆分

• 结果-优点:

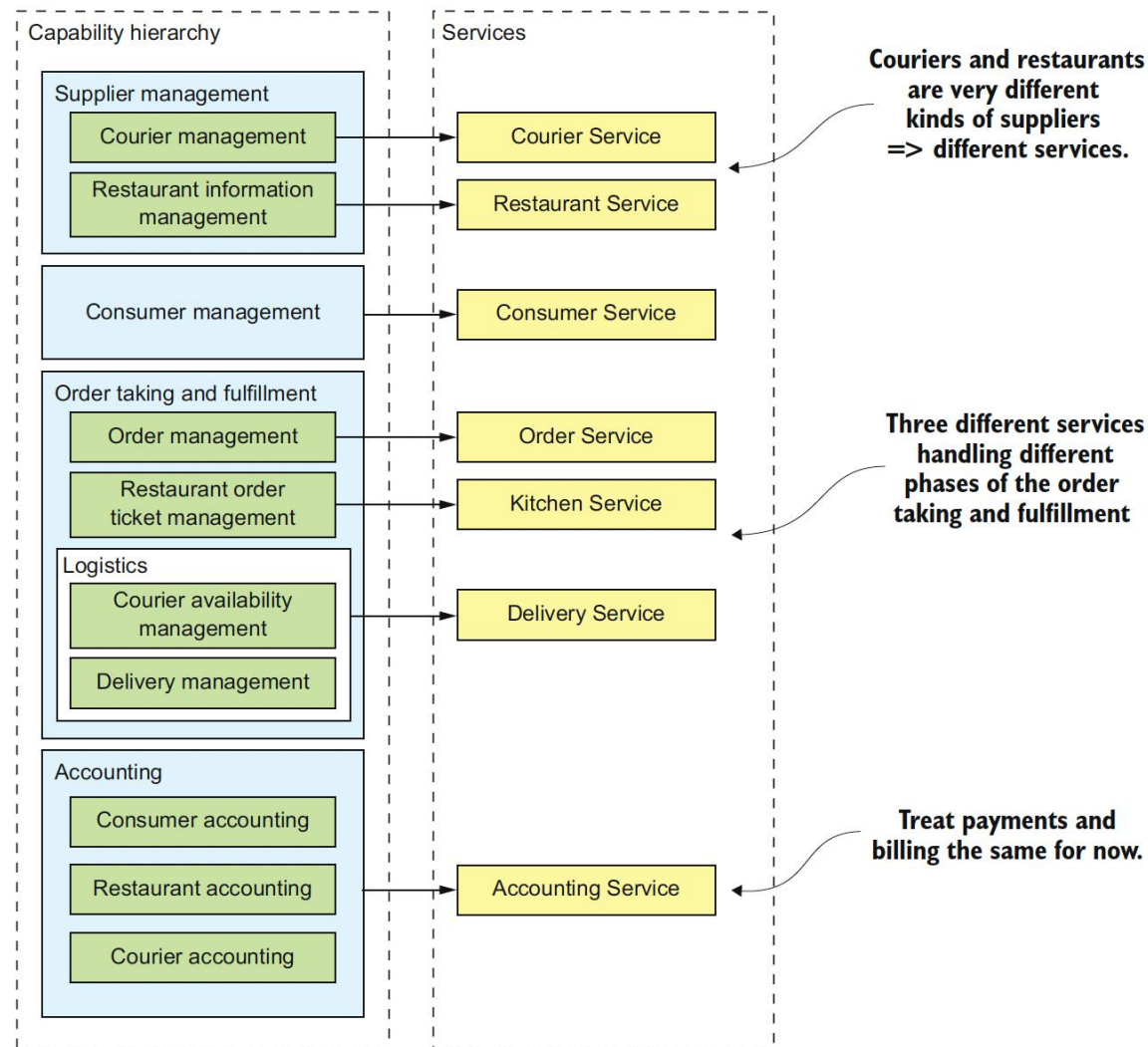
- 架构稳定 (业务能力相对稳定)
- 开发团队跨职能、自治, 围绕交付业务价值而非技术特性进行组织
- 服务高内聚和松耦合

• 结果-问题:

- 如何分析业务能力: 分析组织的目的、结构、业务流程

• 相关模式:

- 根据子域拆分、动静态调用关系进行服务拆分



FTGO的业务能力映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分 || 根据子域拆分

子域：领域驱动设计方法论的核心

• 领域驱动设计（DDD）

- 解决复杂软件业务领域范围/业务边界划分的问题
- 业务出发，以面向对象和领域模型为核心

• 领域

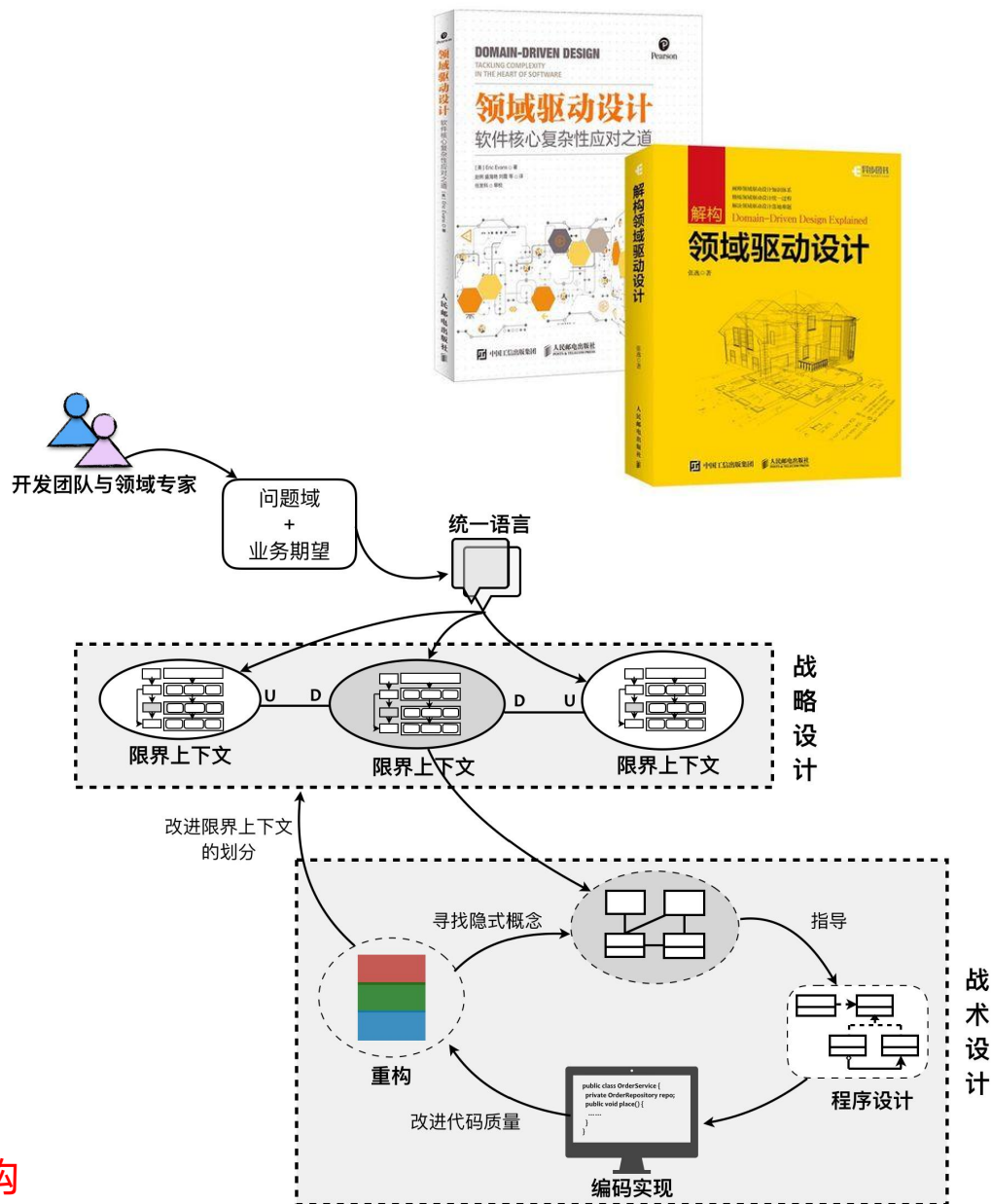
- 描述问题域，一种特定的范围，电商、外卖、保险…
- 子域是领域的细分，电商（订单、商品、物流）
- 核心域、通用域、支撑域

• 核心思想和流程

- 将问题域逐级细分，降低理解和实现的复杂度
- 从业务需求中提炼**统一语言**
- **战略设计**：
 - 构建**领域模型**，识别**限界上下文**，确定**领域边界**
 - **上下文映射**建立领域间关系
 - **划分（微）服务**的逻辑和物理边界

• 战术设计

- 限界上下文内领域建模，指导程序**设计、编码和重构**





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分 || 根据子域拆分

- 通用语言 (Ubiquitous Language)

- 定义领域内相关团队的**词汇表**
- 统一、简单、清晰、准确描述业务规则和业务
- 理解不一致问题，如账户
 - 身份和访问管理上下文，身份验证和授权的凭证
 - 客户管理上下文，人口统计和联系人属性
 - 财务会计上下文，付款和历史交易的特定信息

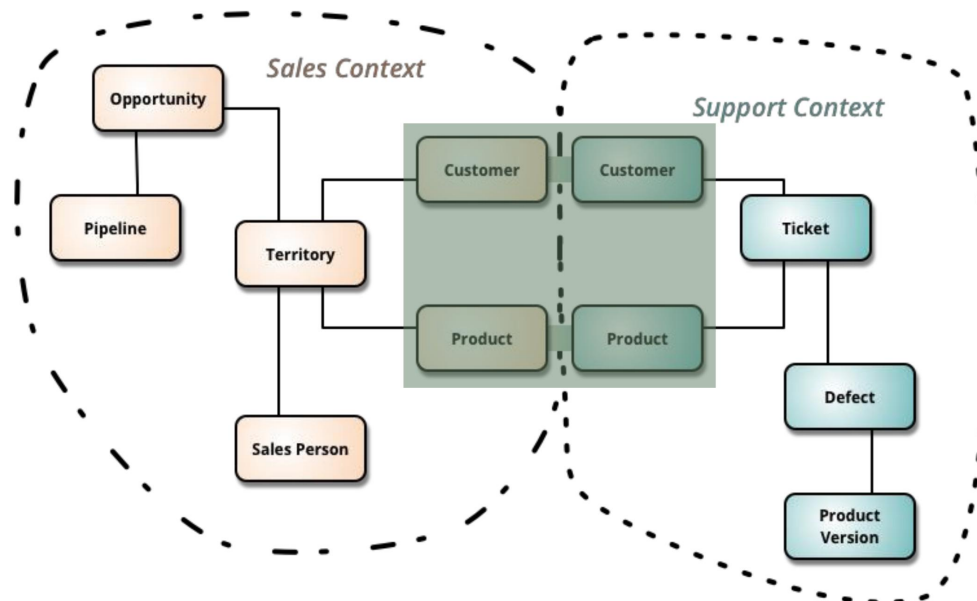


- 领域模型 (Domain Model)

- 以解决具体问题的方式包含一个**领域的知识**
- 为每一个子域定义单独的领域模型

- 限界上下文 (Bounded Context)

- 领域模型的**边界**
- 包括实现模型的**代码集合**
- 对应微服务架构中**一个或一组服务**





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分 || 根据子域拆分

• 结果-优点:

- 高内聚和松耦合
- 子域和限界上下文与微服务边界匹配
- 架构稳定（子域相对稳定）
- 领域模型由团队独立开发、支持团队自治
- 子域用于自己的领域模型，消除上帝类和优化拆分

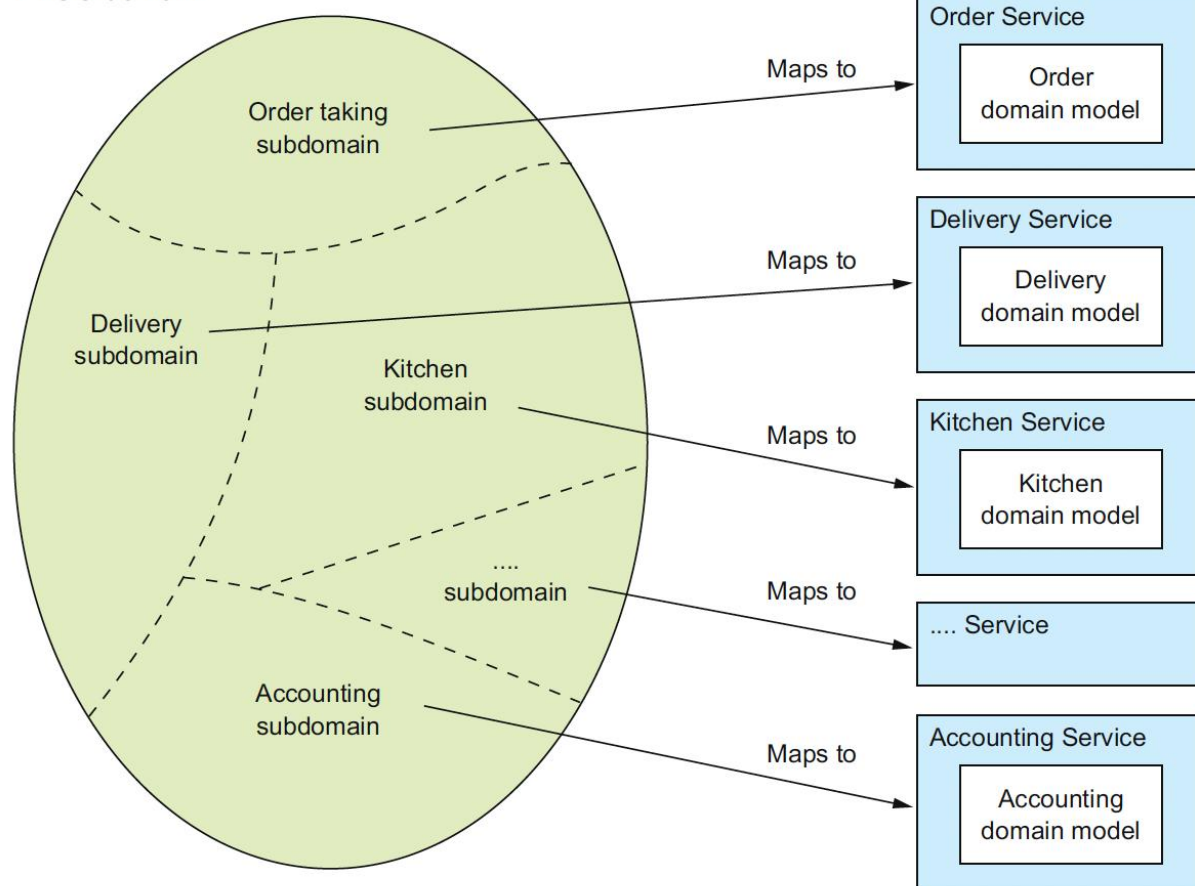
• 结果-问题:

- 如何识别子域：分析业务及其组织结构并识别不同的专业领域

• 相关模式:

- 根据业务能力、动静态调用关系进行服务拆分

FTGO domain



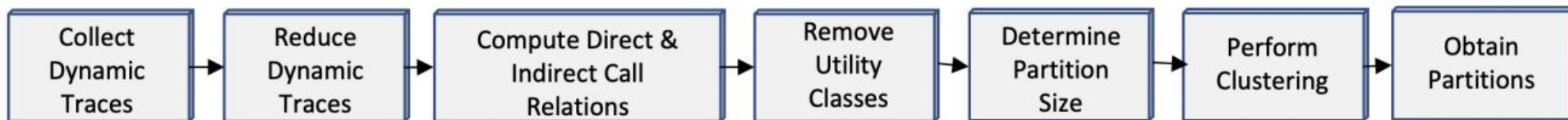
FTGO的子域映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 根据动静态调用关系拆分



- 收集单体应用动静态调用信息

- 静态：用例分析、字节码解析、API接口
- 动态：调用链路、数据流图、控制流图

- 构建有向带权图（调用频率、变更频率等）

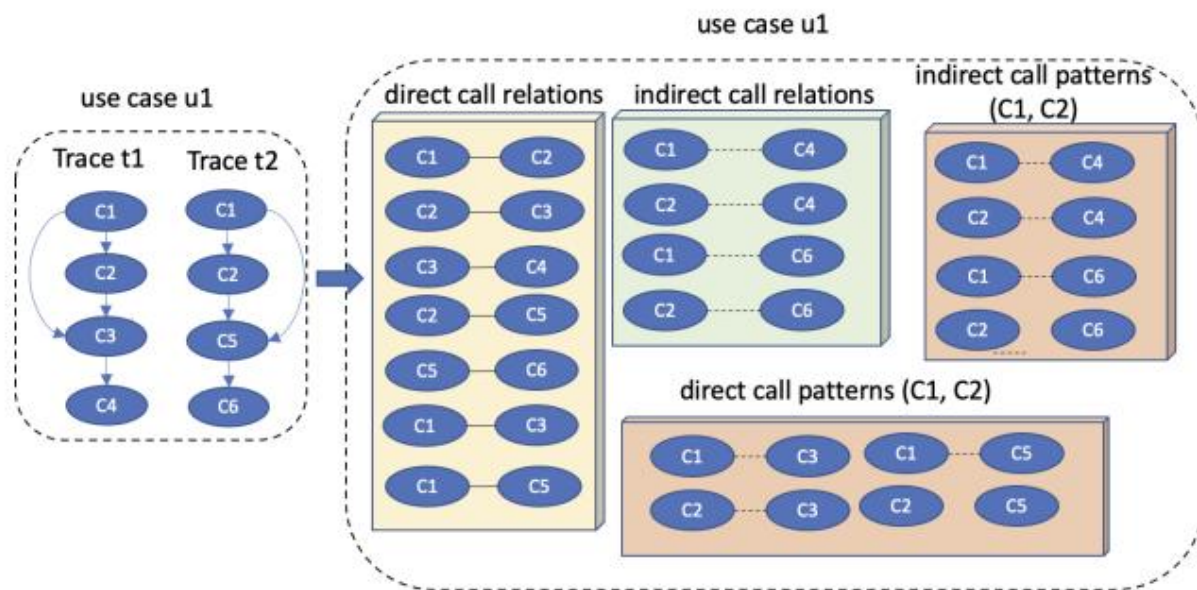
- 基于聚类算法拆分

- 结果：

- 棕地开发场景的拆分（遗留系统）
- 自动化、效率高
- 受限于遗留系统的分析和数据收集难度

- 相关模式：

- 根据业务能力、子域进行服务拆分



调用关系图映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || API定义、实现

- 服务API操作的存在原因

- 某些操作对应于系统操作（外部客户端/其他服务调用）
- 存在其他操作支持服务间协作（其他服务调用）
- 通过事件发布支持协作

- API定义流程

- 将系统操作分配给服务
- 确定支持协作的API

Service	Operations
Consumer Service	<code>createConsumer()</code>
Order Service	<code>createOrder()</code>
Restaurant Service	<code>findAvailableRestaurants()</code>
Kitchen Service	■ <code>acceptOrder()</code> ■ <code>noteOrderReadyForPickup()</code>
Delivery Service	■ <code>noteUpdatedLocation()</code> ■ <code>noteDeliveryPickedUp()</code> ■ <code>noteDeliveryDelivered()</code>

FTGO的系统操作映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || API定义、实现

- 步骤1：将系统操作分配给服务
- 确定请求的初始入口服务
- 映射不清晰的服务
 - 将操作分配给需要操作提供信息的服务
 - noteUpdatedLocation() 操作
 - 送餐员服务或配送服务都有关系
 - 配送服务需要该操作提供位置信息
- 其他情况将分配给具有处理它所需信息的服务
 - noteOrderReadyForPickup() 操作

Service	Operations
Consumer Service	createConsumer()
Order Service	createOrder()
Restaurant Service	findAvailableRestaurants()
Kitchen Service	■ acceptOrder() ■ noteOrderReadyForPickup()
Delivery Service	■ noteUpdatedLocation() ■ noteDeliveryPickedUp() ■ noteDeliveryDelivered()

FTGO的系统操作映射到服务



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || API定义、实现

- 步骤2：确定支持协作的API
- 某些操作可以由单个服务处理
 - verifyConsumerDetails()
- 某些操作跨越多个服务（数据分散）
 - createOrder()调用多个服务
 - 验证前置条件并使后置条件成立
 - acceptOrder()
 - 调用配送服务安排送餐员并交付订单
- 进程间通信技术
 - 基于同步请求和响应
 - 异步消息

Service	Operations	Collaborators
Consumer Service	verifyConsumerDetails()	—
Order Service	createOrder()	■ Consumer Service verifyConsumerDetails() ■ Restaurant Service verifyOrderDetails() ■ Kitchen Service createTicket() ■ Accounting Service authorizeCard()
Restaurant Service	■ findAvailableRestaurants() ■ verifyOrderDetails()	—
Kitchen Service	■ createTicket() ■ acceptOrder() ■ noteOrderReadyForPickup()	■ Delivery Service scheduleDelivery()
Delivery Service	■ scheduleDelivery() ■ noteUpdatedLocation() ■ noteDeliveryPickedUp() ■ noteDeliveryDelivered()	—
Accounting Service	■ authorizeCard()	—

FTGO的服务修订后的API及协作者



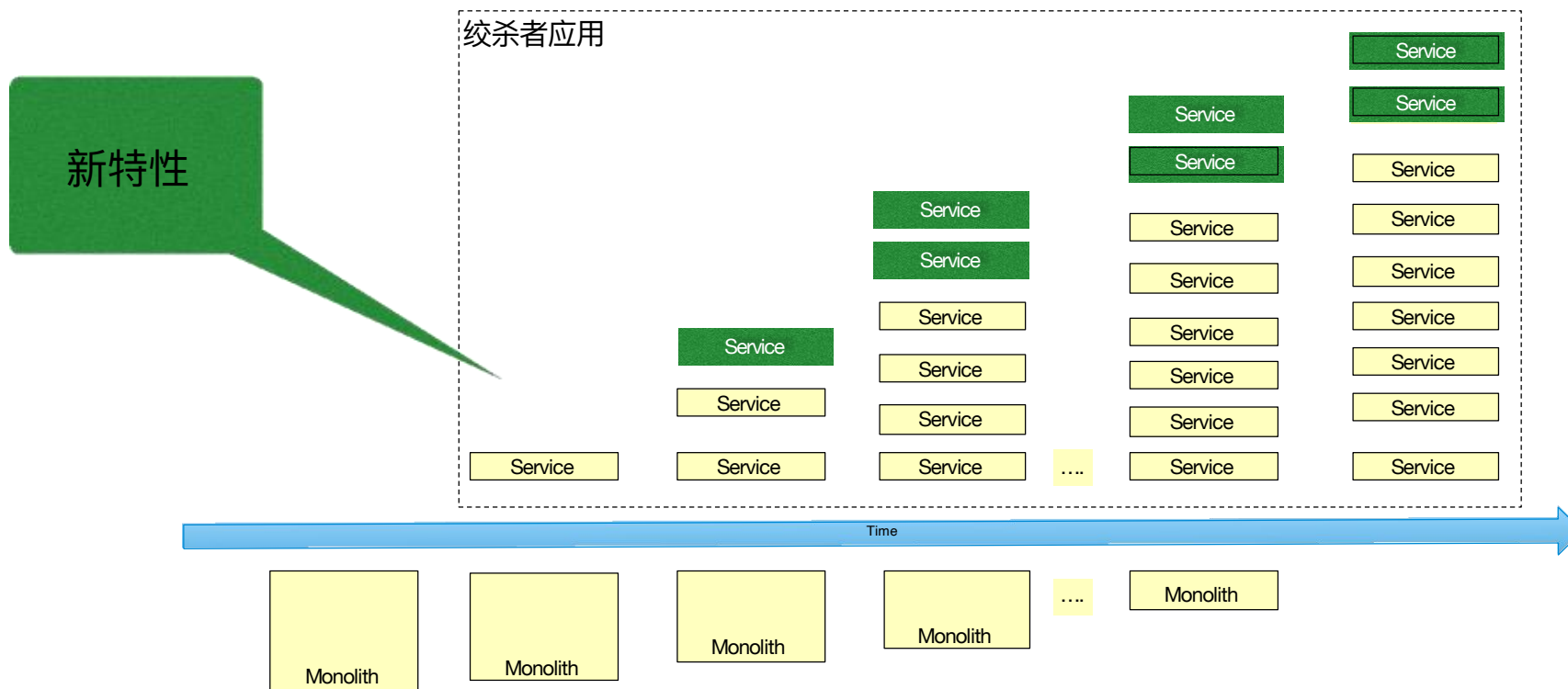
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 重构策略

Best done incrementally!

绞杀者应用的规模逐渐扩大



单体应用逐渐萎缩



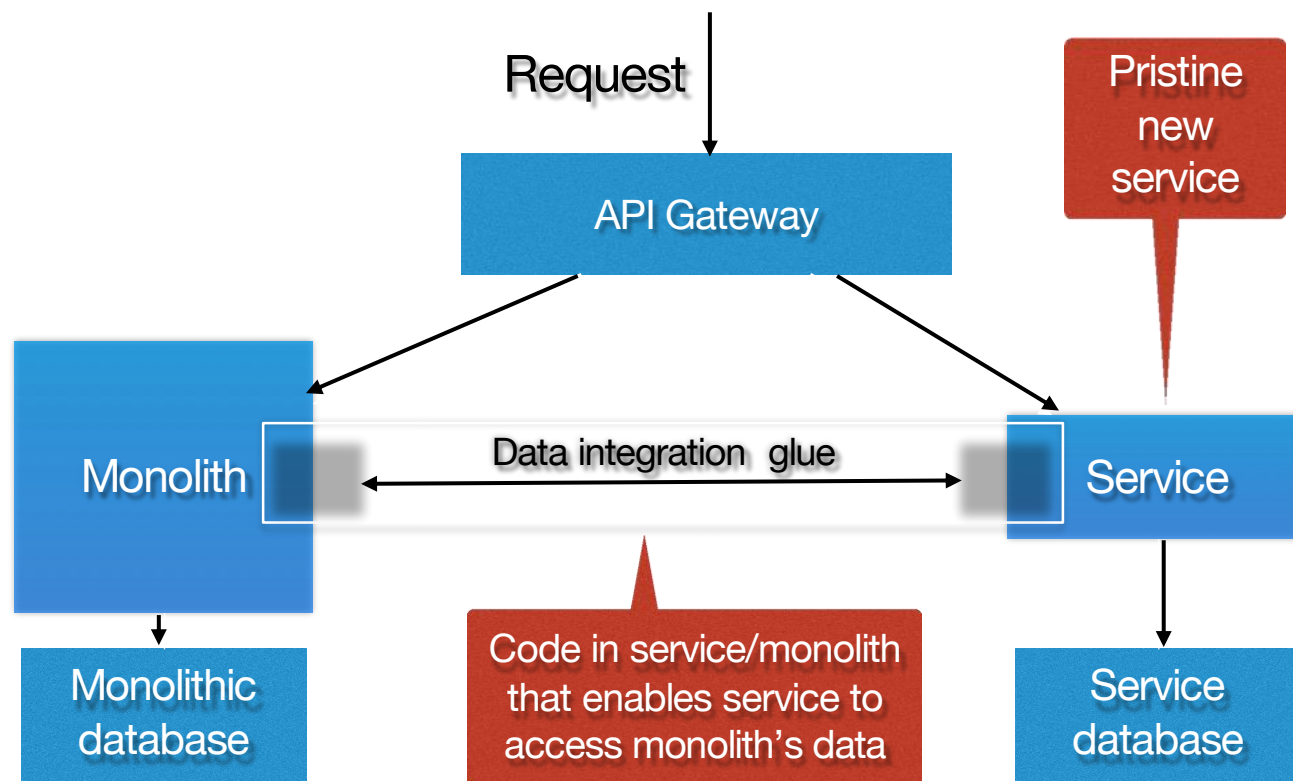
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 重构策略

策略1. 将新功能实现为微服务:

- **API Gateway**: 新功能的请求路由到新服务, 遗留系统请求路由到单体
- **集成胶水 (Integration Glue) 代码**: 将服务与单体结合, 访问单体所有数据, 并能调用单体实现的功能
 - API请求
 - 复制单体应用数据到服务本地用于读





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

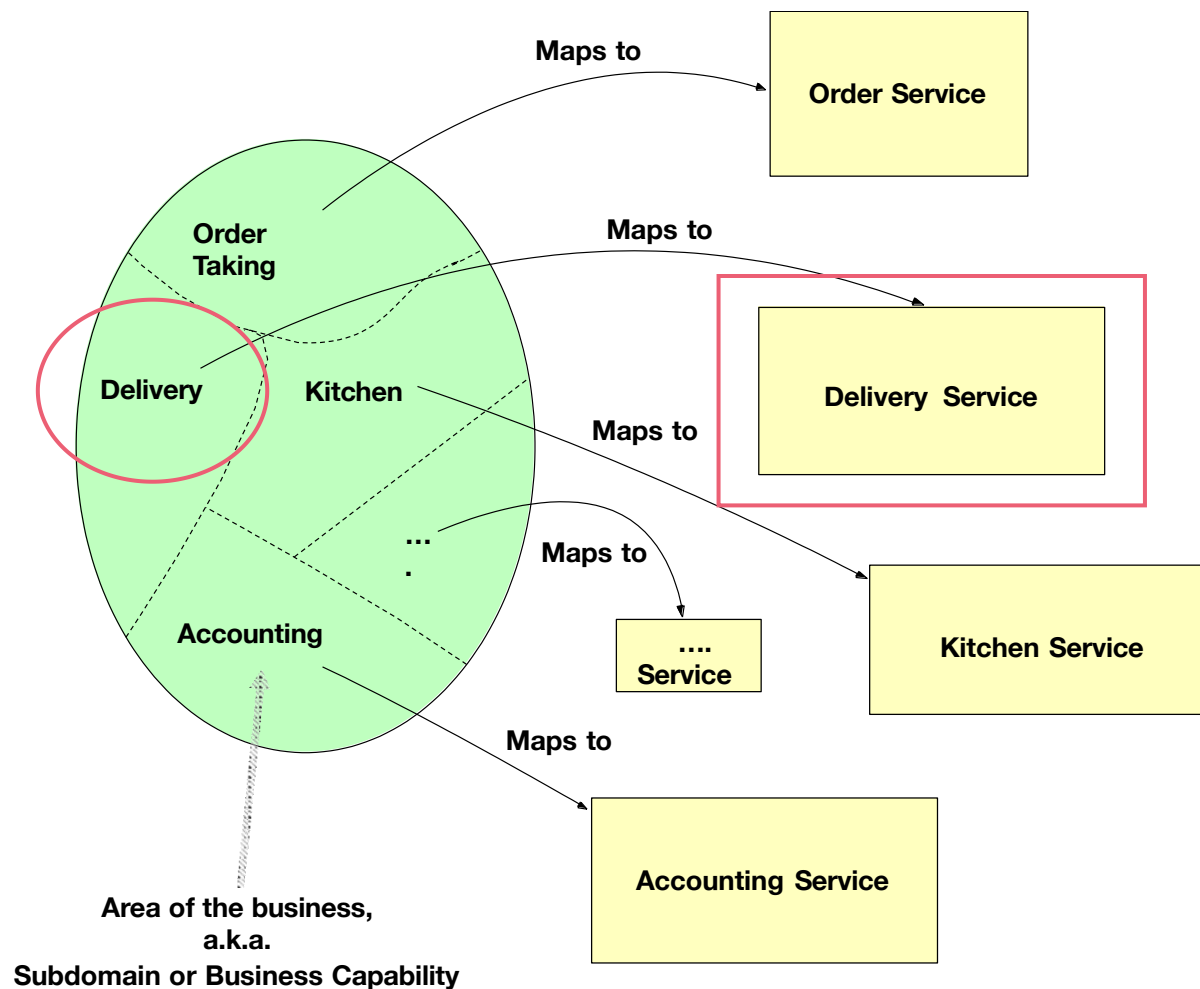


微服务拆分设计 || 重构策略

策略2. 提取业务能力到服务中:

以Delivery Service为例:

- 步骤1: 拆分源码, 转换为模块
- 步骤2: 拆分数据库
- 步骤3: 定义和部署Delivery服务
- 步骤4: 调用Delivery服务
- 步骤5: 删除单体应用中的遗留代码



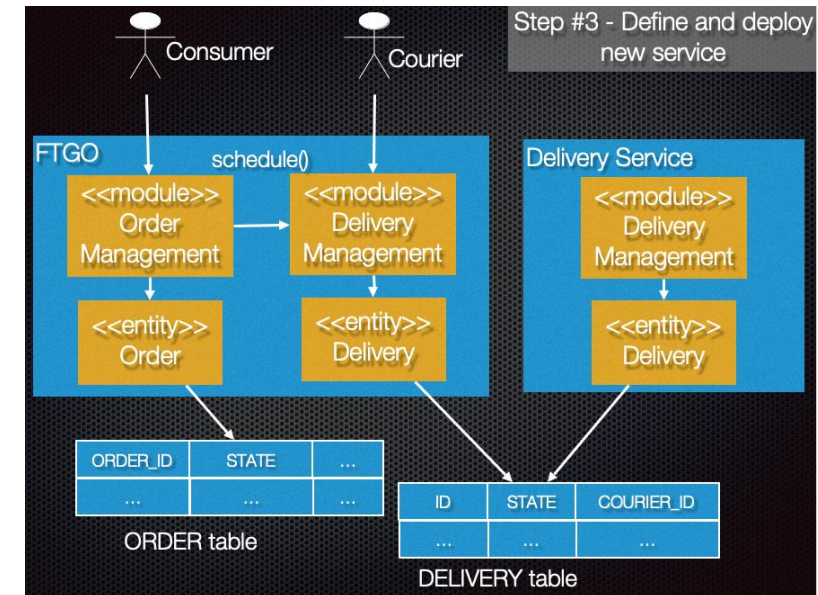
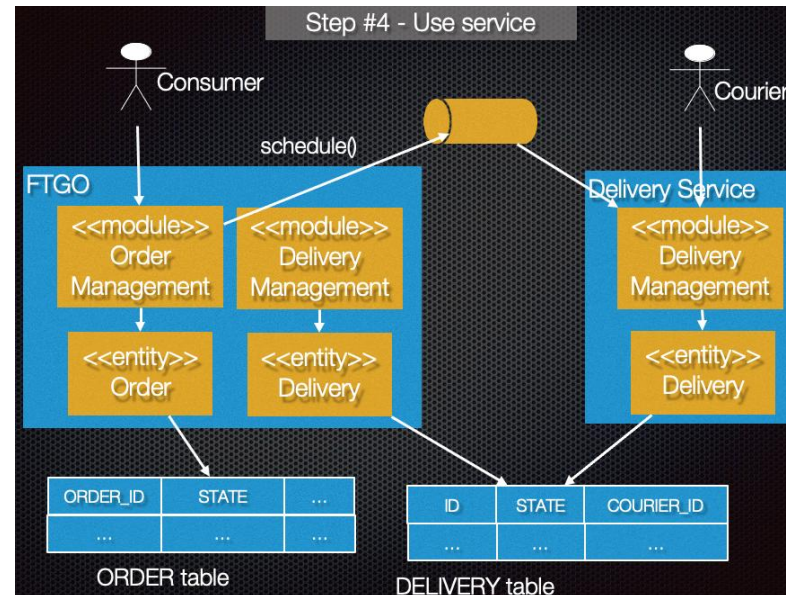
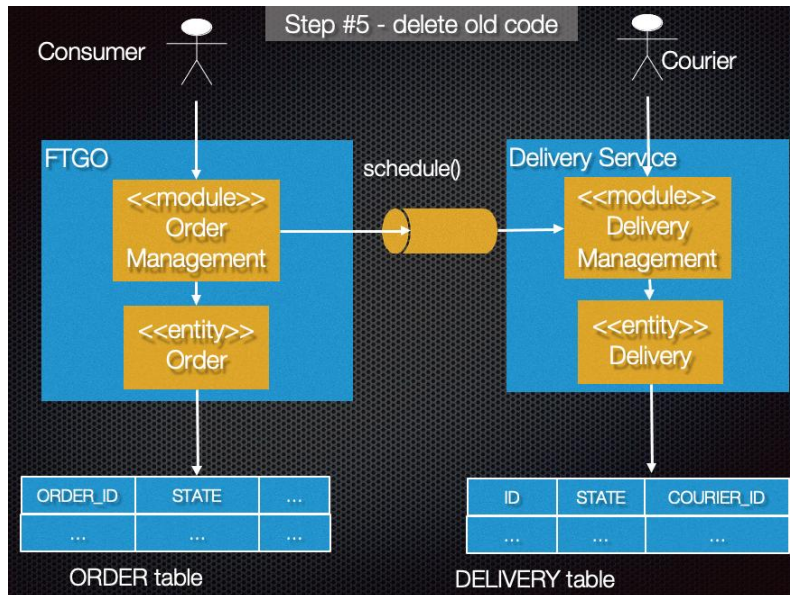
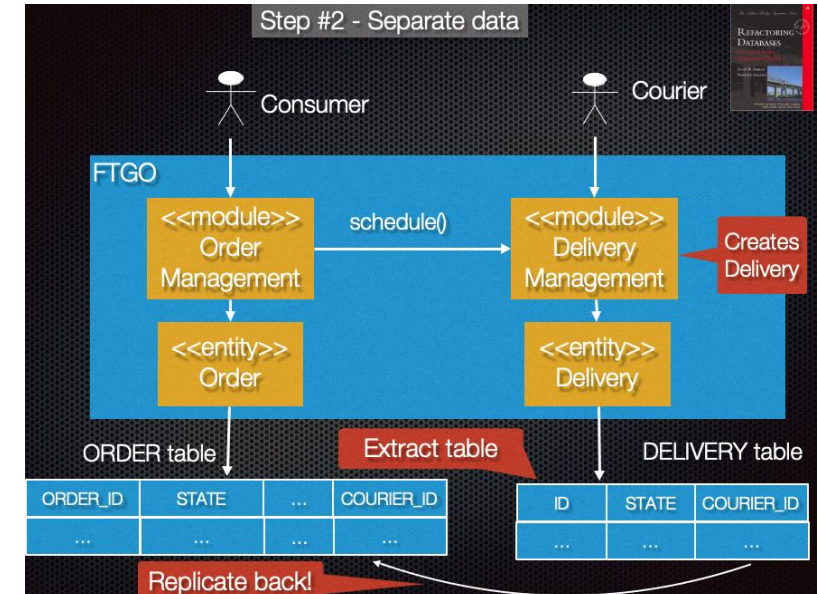
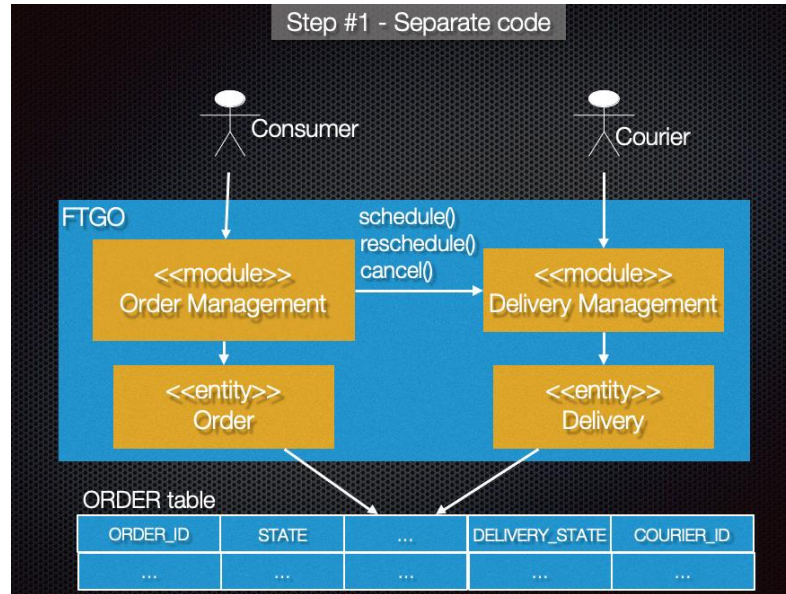


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务拆分设计 || 重构策略

策略2. 提取业务能力到服务中:





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



其他核心模式 || 服务注册与发现

- **问题**：基于 RPI 的客户端如何在网络上发现服务实例的位置？

- **模式**：应用层服务发现模式

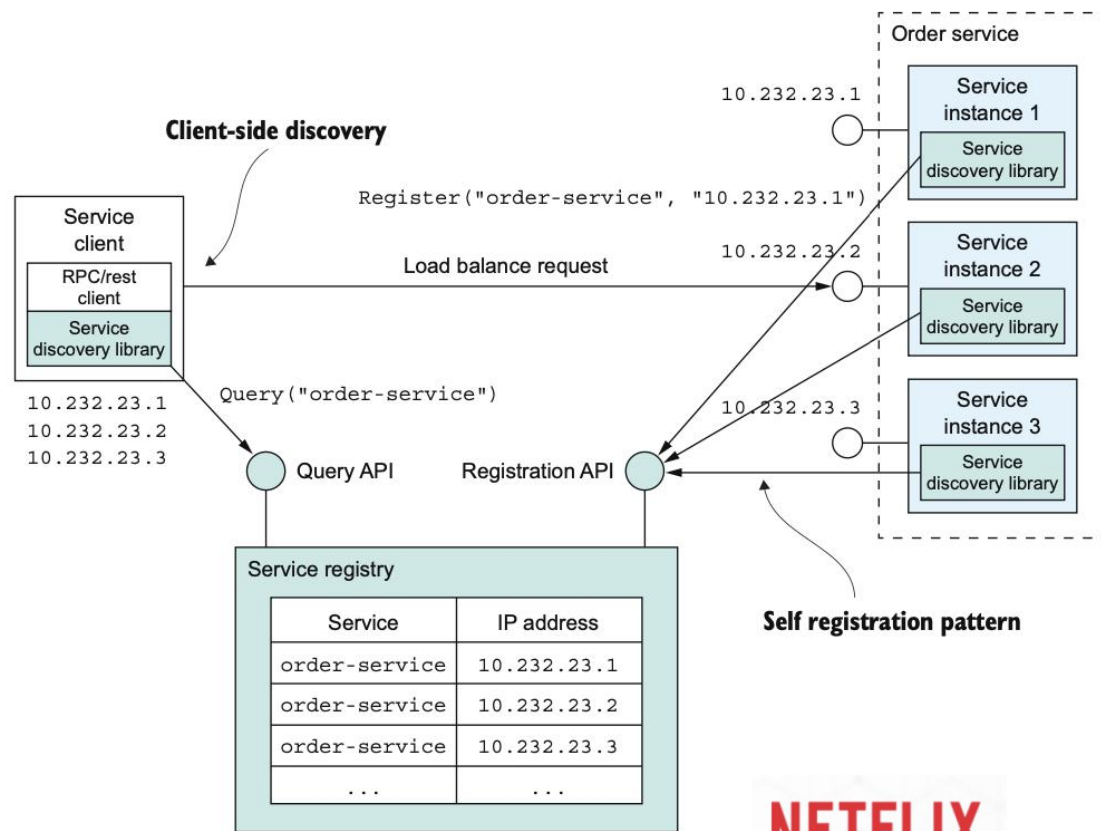
- 自注册：服务实例调用服务注册表的注册API来注册其网络位置的（服务注册表定期调用心跳API）
- 客户端发现：客户端查询服务注册表以获取服务实例的列表（缓存+负载均衡，提高性能）

- **结果**：

- 处理多平台部署的问题（服务发现机制与具体的部署平台无关，Eureka vs. K8s）
- 需要为使用的每种编程语言（可能还有框架）提供服务发现库（Spring Cloud仅支持）
- 开发者负责设置和管理服务注册表，分散精力

- **相关模式**：

- 前序模式：同步远程过程调用
- 替代模式：平台层服务发现模式



应用层服务发现

NETFLIX
EUREKA

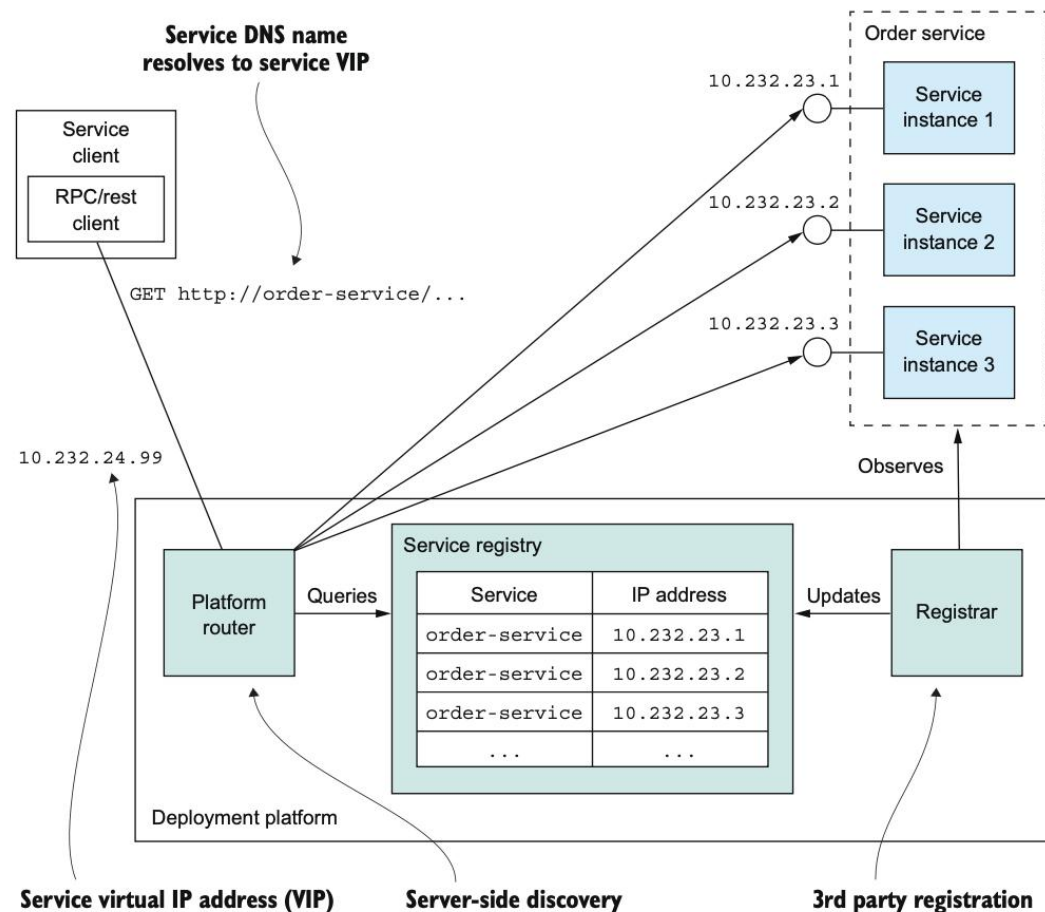


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



其他核心模式 || 服务注册与发现

- **问题：**基于 RPI 的客户端如何在网络上发现服务实例的位置？
- **模式：**平台层服务发现模式
 - 第三方注册：由第三方负责（注册服务器，通常是部署平台的一部分）处理注册，而不是服务本身向服务注册表注册自己。
 - 服务端发现：客户端无需查询服务注册表，而是向DNS名称发出请求，对该DNS名称的请求被解析到路由器，路由器查询服务注册表并对请求进行负载均衡。
- **结果：**
 - 完全交给部署平台，服务端、客户端代码减负
 - 多语言支持度较高
 - 存在平台约束
- **相关模式：**
 - 前序模式：同步远程过程调用
 - 替代模式：应用层服务发现模式



平台层服务发现

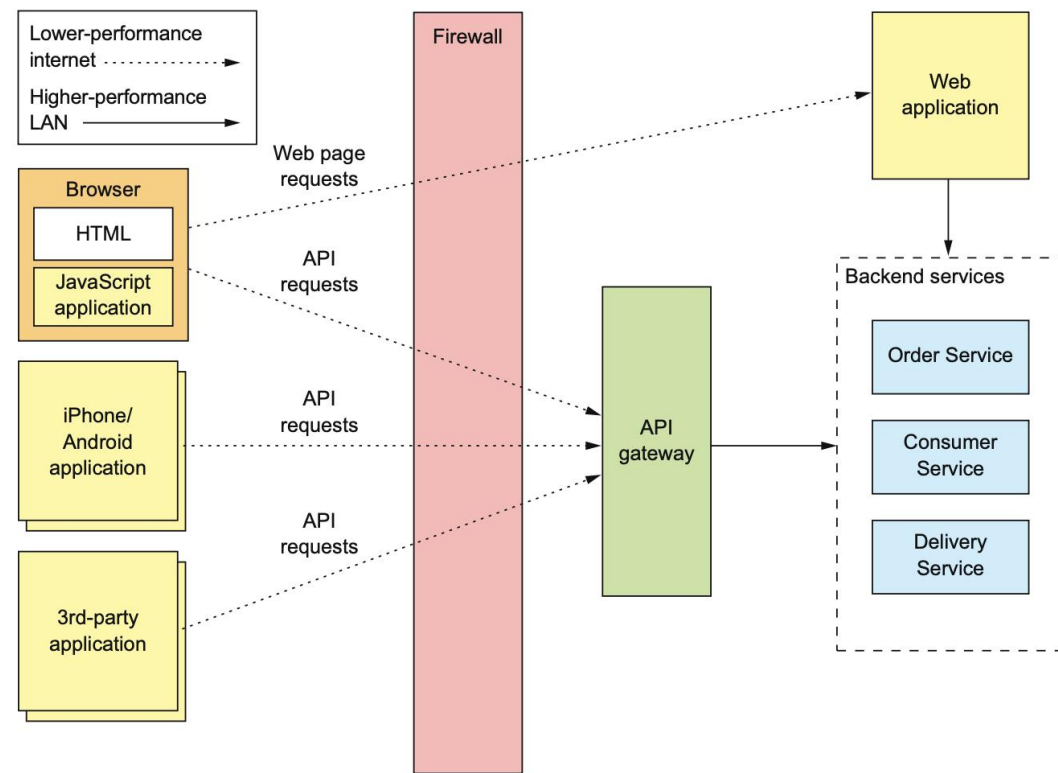


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



其他核心模式 || API网关

- **问题：** 如何处理外部客户端与服务之间的通讯？
- **需求：** 支持多种客户端的API
 - 细粒度API，客户端与多项服务交互请求效率低
 - 不同客户端需要不同的数据，性能要求亦有区别
 - 缺少封装，影响可修改性（服务划分方式变化）
 - 服务的实例数量与位置（地址和端口）动态变化
 - 对客户端不友好的进程间通信（防火墙内外协议不同）
- **模式：** API Gateway模式
 - 实现一个服务，外部API 客户端进入基于微服务应用的入口点
 - 针对不同客户端提供不同的API
- **结果：**
 - 封装应用程序内部结构，减少交互（请求往返）次数
 - 确保客户端不受服务实例位置影响
 - API组合、协议转换和边缘功能，身份验证等
 - 问题：性能和可扩展性、局部故障等
- **相关模式：**
 - 替代模式：后端前置模式
 - 后续模式：断路器模式、服务发现模式



API网关（API Gateway）

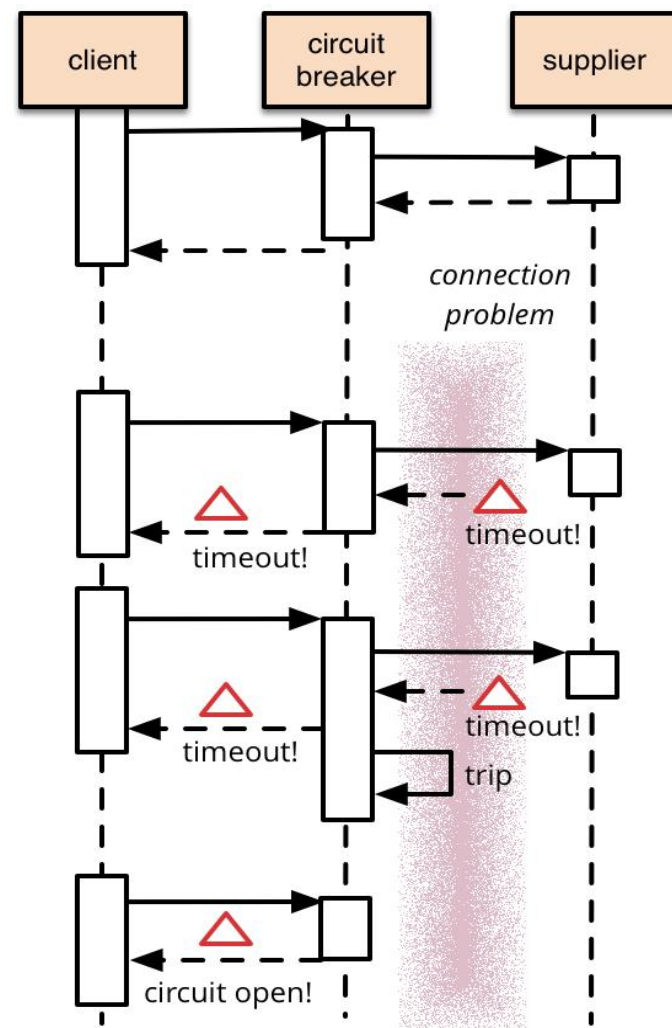


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



其他核心模式 || 断路器

- **问题**：同步通信中如何避免由于服务故障或网络中断所引起的故障蔓延到其他服务？
- **模式**：断路器（Circuit Breaker）
 - **闭合状态**：对程序的请求能够直接引起方法的调用。
 - **断开状态**：对程序的请求会立即返回错误响应。
 - **半断开状态**：允许对程序的一定数量的请求可以调用服务，如调用成功，可认为之前导致调用失败的错误已经修正，熔断器切换到闭合状态；如调用失败，则认为问题仍存在，熔断器切回到断开状态。
- **结果**：
 - 防止不断地尝试执行可能会失败的操作；
 - 使程序能够诊断错误是否已经修正，进而再次尝试调用操作
- **相关模式**：
 - 前序模式：同步远程过程调用模式



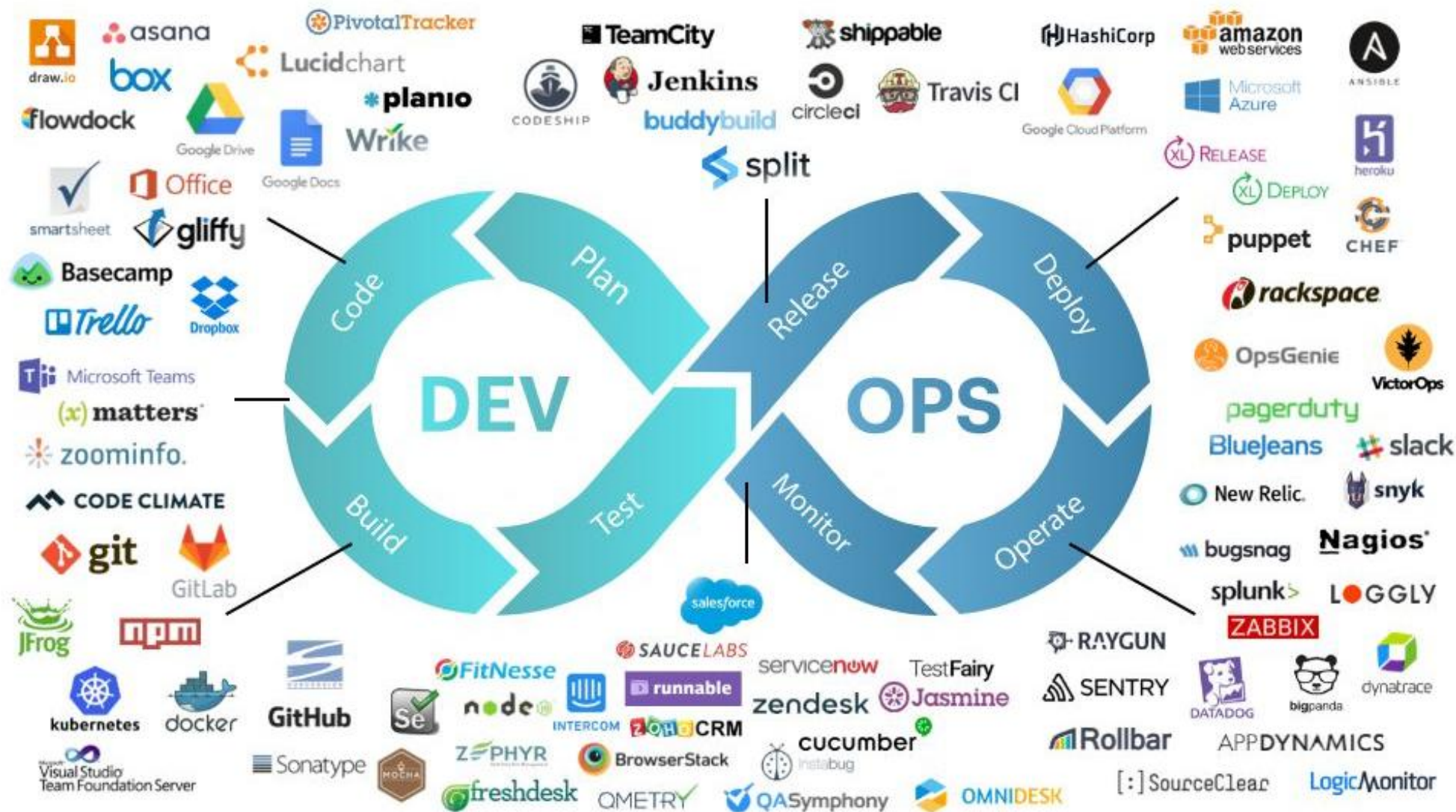
HYSTRIX
DEFEND YOUR APP



南京大學120周年校慶
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构实现





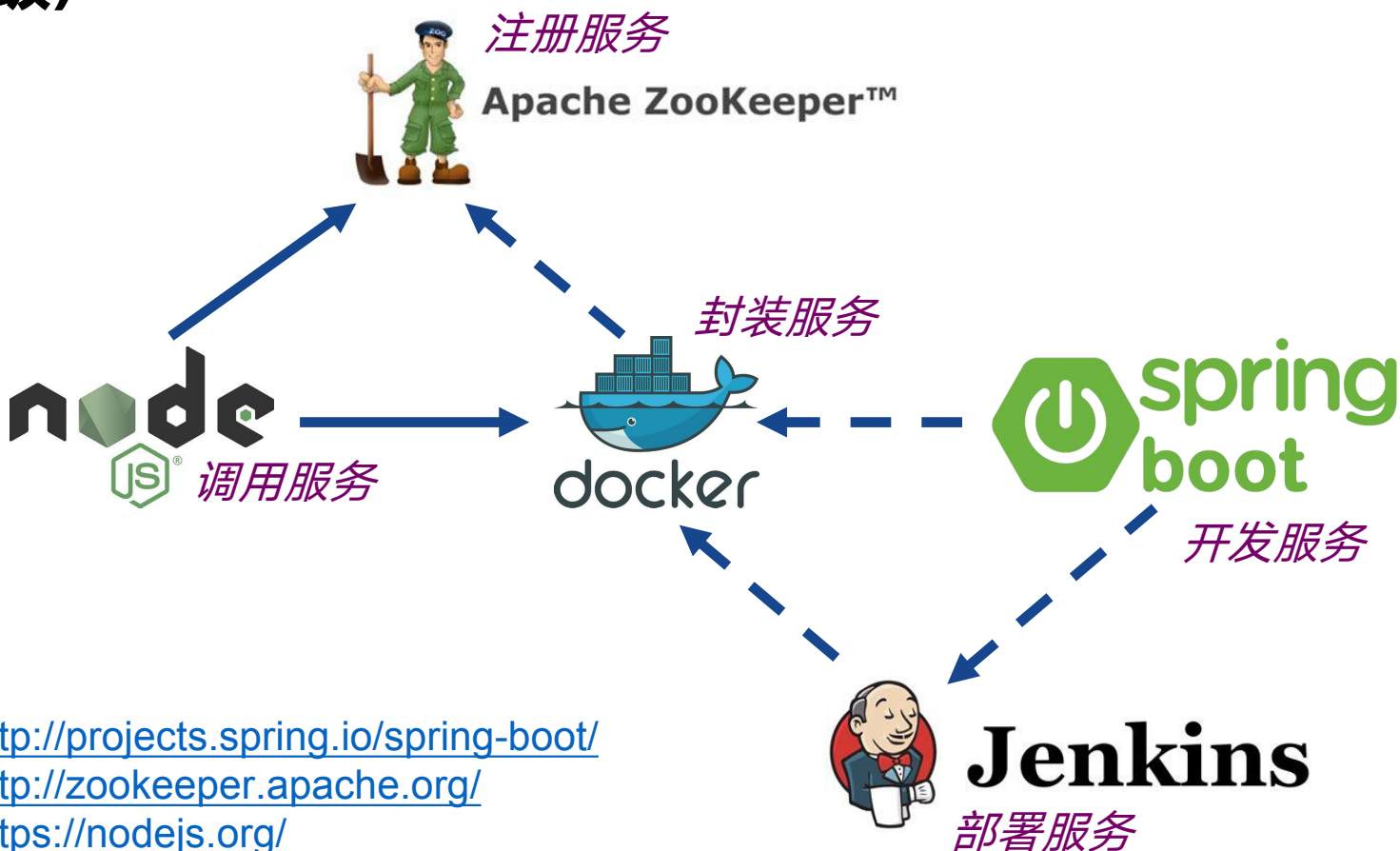
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构实现

• 微服务技术选型（轻量级）

- **开发**服务: *Spring Boot*
- **封装**服务: *Docker*
- **部署**服务: *Jenkins*
- **注册**服务: *ZooKeeper*
- **调用**服务: *Node.js*



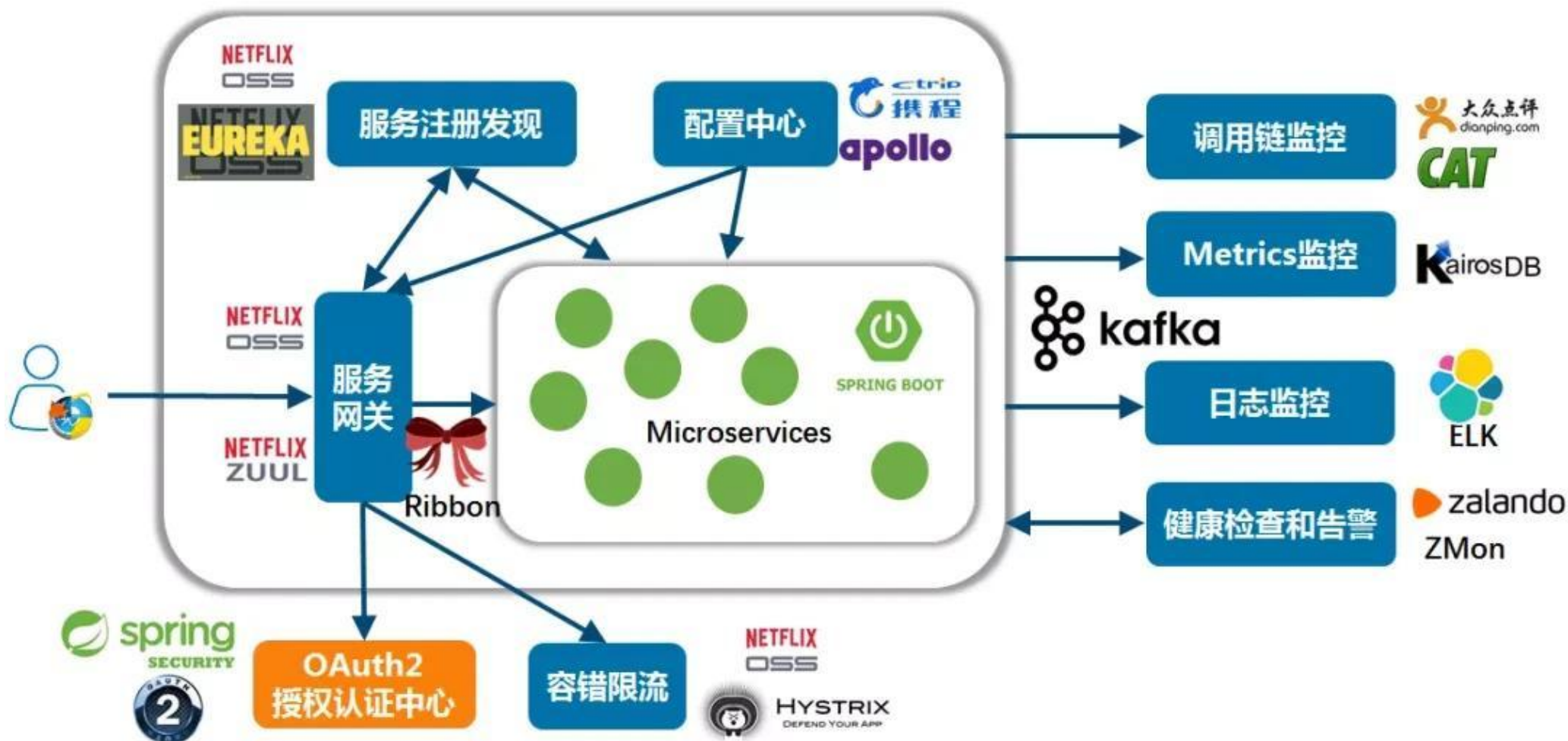
<http://projects.spring.io/spring-boot/>
<http://zookeeper.apache.org/>
<https://nodejs.org/>
<https://jenkins.io/>
<https://docker.com/>



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



作业

1. 选取一个开源的单体系统，分析需求（用户故事）、识别系统操作
 - GitHub上检索
 - Star数量超过500
 - 有完整需求分析和文档
 - 无对应的微服务版本
2. 根据服务业务能力和子域进行微服务的划分
3. 定义候选微服务的API并体现和业务操作的对应关系
4. 撰写设计报告
 - 需求分析
 - 两种方案的划分过程
 - 包含划分结果的微服务架构设计图
 - 候选微服务的API并体现和业务操作的对应关系



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

- 20世纪90代(企业级Java应用)

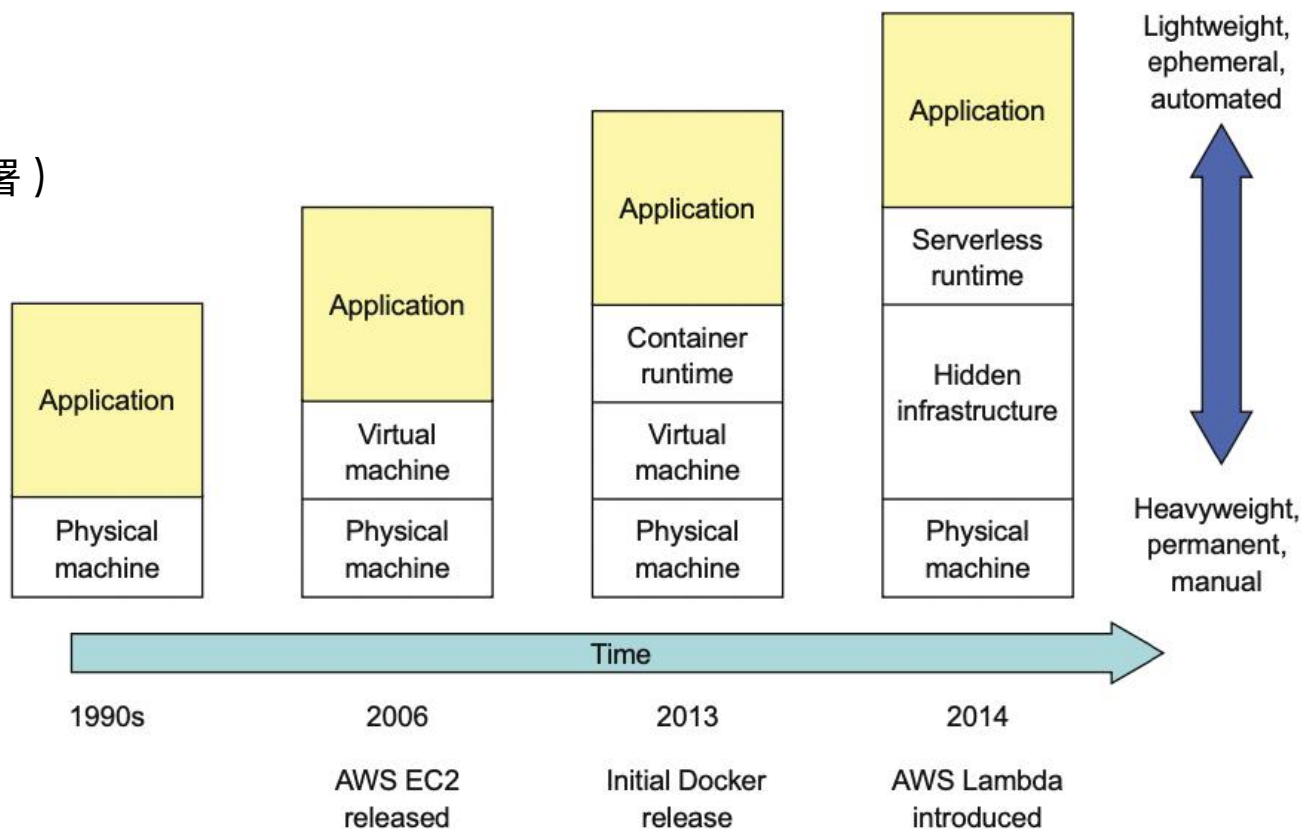
- 微型机时代
- 开发团队向运维团队提交部署工单（手动部署）
- 昂贵且重量级的应用服务器，如WebLogic
- 定期安装补丁并更新软件

- 21世纪最初10年

- 开源的轻量级Web容器，如 Tomcat
- 虚拟机开始取代物理机，仍手动部署

- 2010年后-至今

- 高度自动化云上虚拟机
- 更轻量级、一次性资源、用过即丢弃/重建
- 容器、serverless等
- DevOps实践和自动化基础设施对微服务的支持



部署的流程和架构发展变化



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

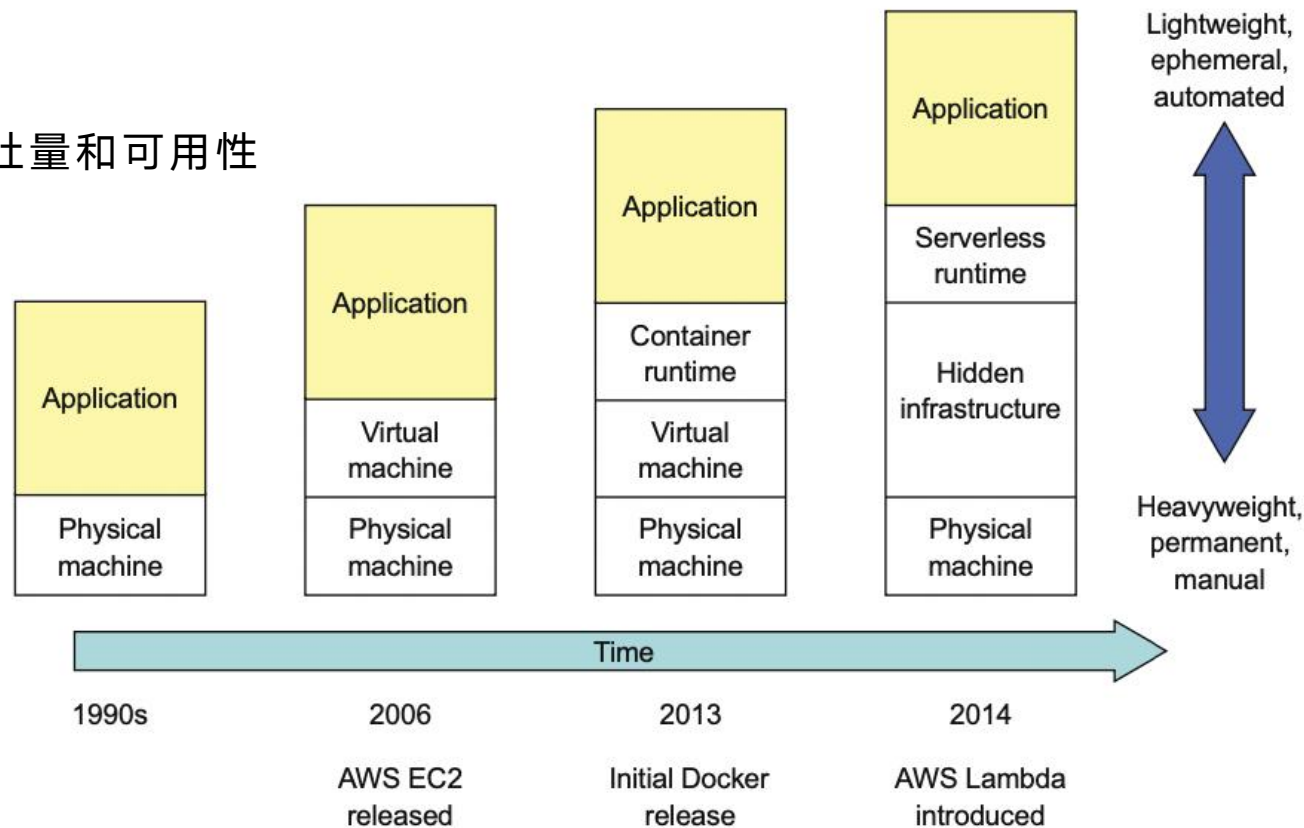
• 上下文

- 微服务架构包含一组服务
- 每个服务都部署为一组服务实例，以实现吞吐量和可用性

• 问题：如何部署？

• 需求：

- 服务使用各种语言、框架和框架版本编写
- 需要快速构建、独立部署和扩展服务
- 服务实例需要相互隔离
- 需要监控每个服务实例的行为、部署可靠
- 需要限制服务消耗的资源（CPU 和内存）
- 尽可能经济高效地部署应用程序





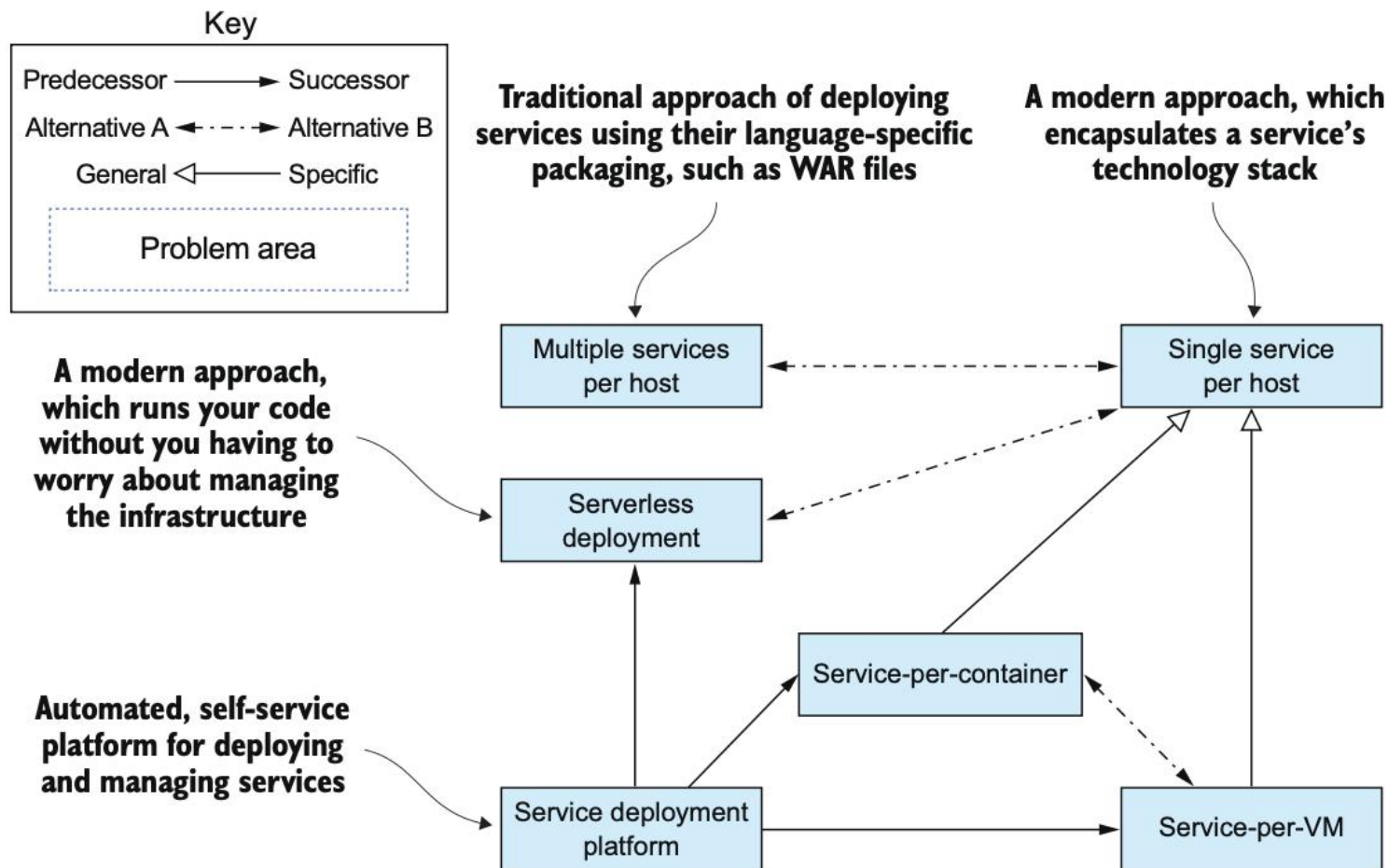
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 部署模式列表

- 单主机部署多个服务实例
- 单主机部署单个服务实例
- 将服务部署到虚拟机
- 将服务部署到容器
- 服务部署平台
- 无服务器部署



基础设施相关模式组-部署模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：单主机部署多个服务实例

- 资源需求冲突的风险
- 在主机（物理机或虚拟机）上运行不同服务的多个实例。
- 有多种方法可以在共享主机上部署服务实例，包括：
 - 将每个服务实例部署为一个 JVM 进程。例如，每个服务实例一个 Tomcat 或 Jetty 实例。
 - 在同一个 JVM 中部署多个服务实例。例如，作为 Web 应用程序或 OSGI 包。

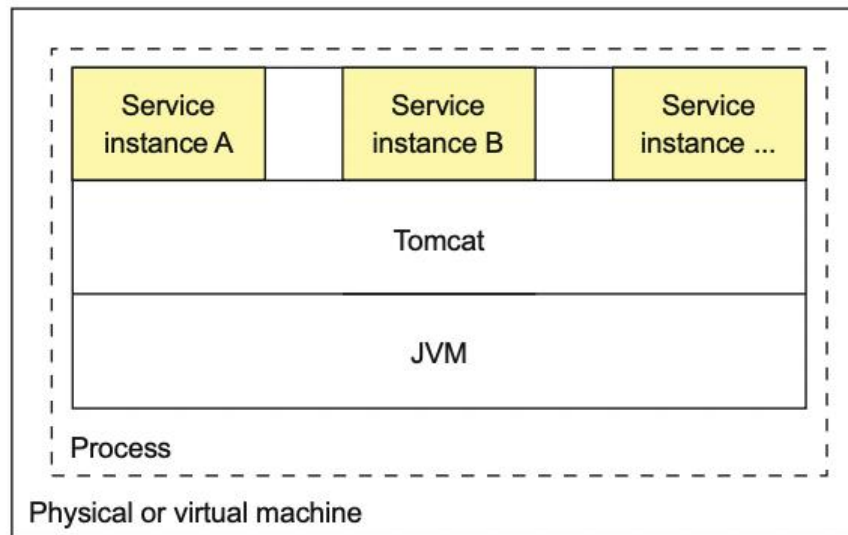
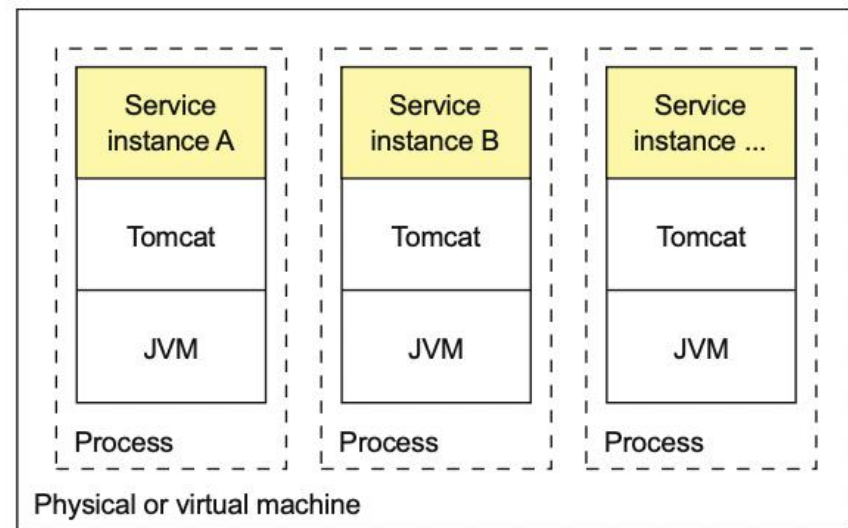
• 优点：资源利用率相对较高

• 缺点：

- 资源需求冲突的风险
- 依赖版本冲突的风险
- 难以限制服务实例消耗的资源
- 在同一个进程中部署多个服务实例，很难监控每个服务实例的资源消耗，也不可能隔离每个实例

• 相关模式：

- 替代模式：单主机部署单个服务实例



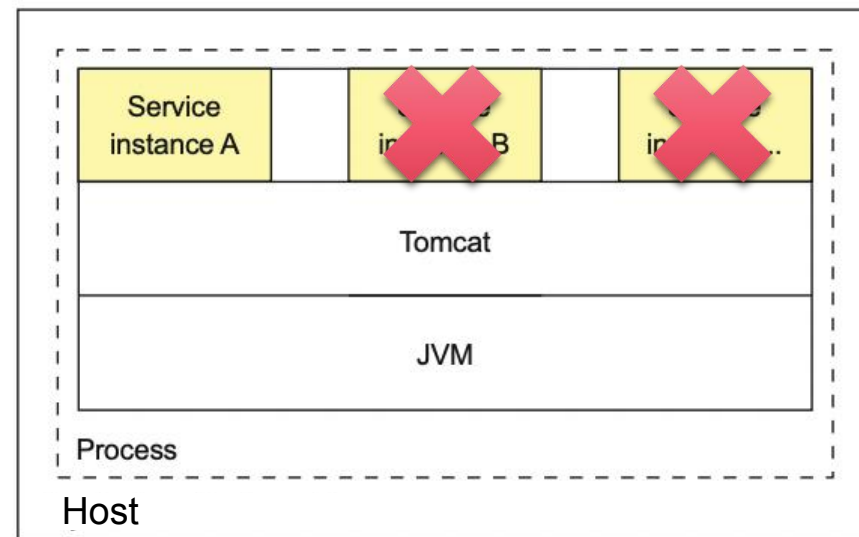


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

- **模式：**单主机部署单个服务实例
 - 在自己的主机上部署单个服务实例
- **优点：**
 - 服务实例彼此隔离
 - 不存在资源需求或依赖版本冲突的可能性
 - 一个服务实例最多只能消耗单个主机的资源
 - 监控、管理和重新部署每个服务实例非常简单
- **缺点：**
 - 与单主机部署多个服务实例模式相比，资源利用效率可能较低，因为主机更多
- **相关模式：**
 - 替代模式：单主机部署多个服务实例、无服务器部署
 - 特化模式：将服务部署到虚拟机、将服务部署到容器





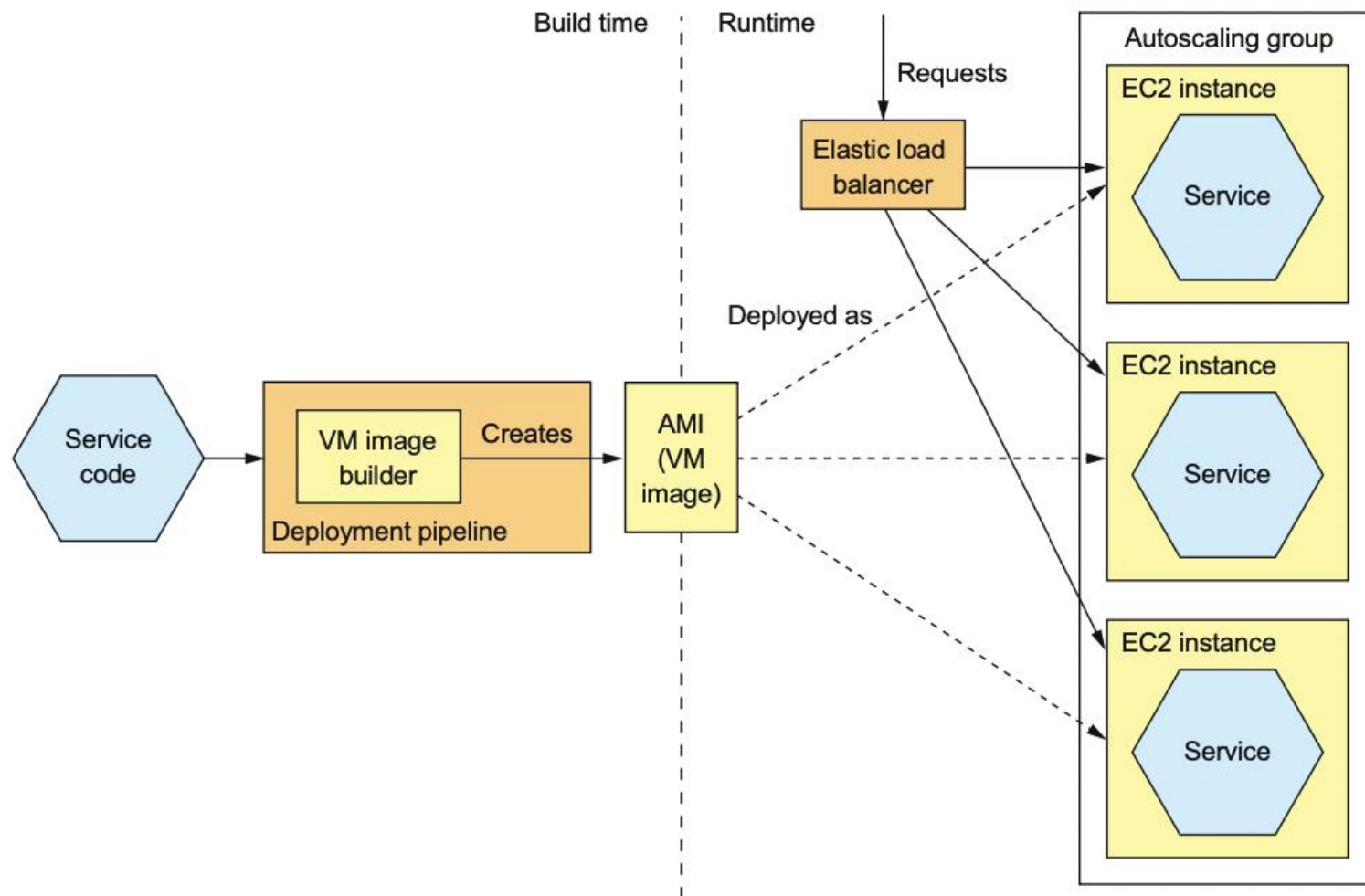
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：将服务部署到虚拟机

- 将服务打包为虚拟机镜像，并将每个服务实例部署为单独的VM
- 比如Netflix部署流水线将每个服务打包为一个 EC2 AMI（包含服务运行所需要的所有内容）
- 运行时每个服务实例是该镜像实例化的虚拟机，如EC2 实例
- EC2弹性负载均衡器(Elastic Load Balancer)将请求路由到对应的实例





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 优点：

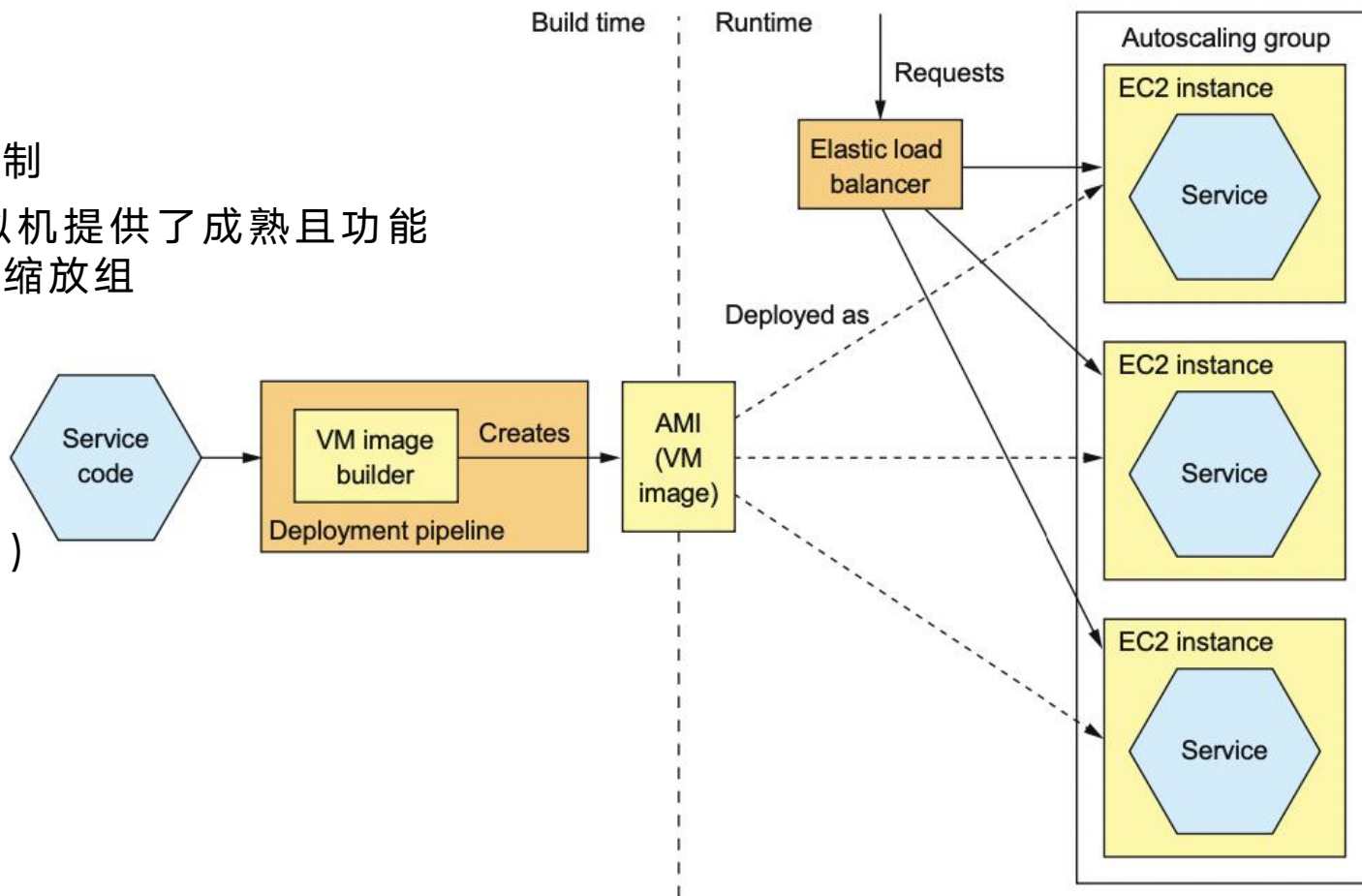
- 通过增加实例数量来扩展服务很简单。Amazon Autoscaling Groups 可以根据负载自动执行此操作
- VM 封装了用于构建服务的技术细节，例如所有服务都以完全相同的方式启动和停止
- 每个服务实例都是隔离的
- VM 对服务实例消耗的 CPU 和内存施加限制
- AWS 等 IaaS 解决方案为部署和管理虚拟机提供了成熟且功能丰富的基础设施，如弹性负载均衡器、自动缩放组

• 缺点：

- 资源利用效率较低（整台虚拟机）
- 部署速度相对较慢（分钟级）
- 系统管理的额外开销（操作系统、运行补丁）

• 相关模式：

- 替代模式：将服务部署到容器
- 泛化模式：单主机部署单个服务实例





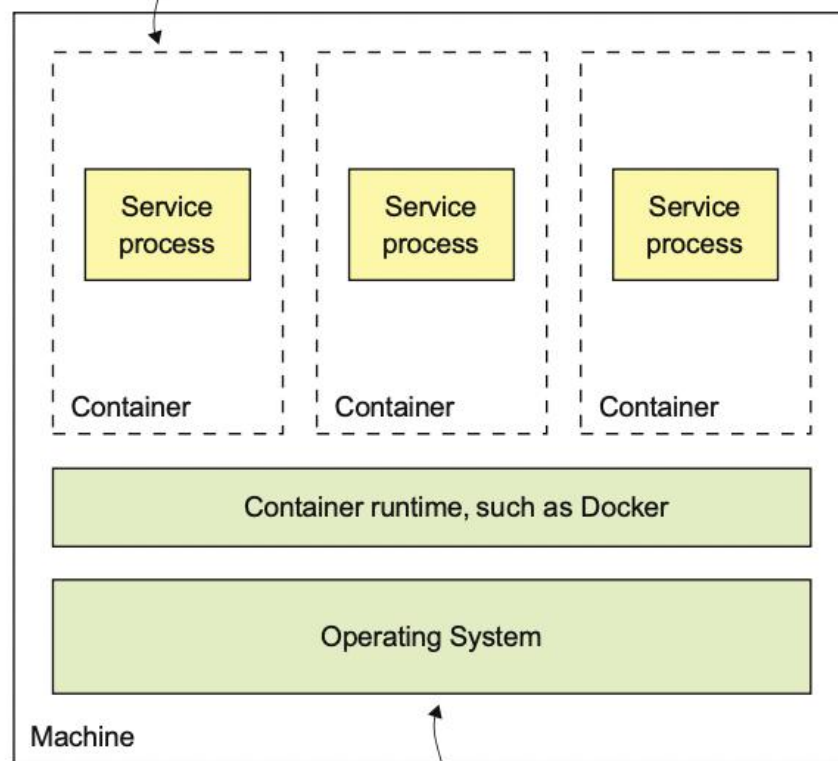
南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

- **模式：** 将服务部署到容器
- 将服务打包为 (Docker) 容器镜像并将每个服务实例部署到容器
 - 容器是一种更现代、更轻量级的部署机制，操作系统级的虚拟化机制
 - 容器由在隔离的沙箱中运行的一个或多个进程组成，多个容器通常在一台机器上运行，容器共享操作系统
 - 从在容器中运行的进程的角度来看，它就好像在自己的机器上运行一样，有独立IP、可消除端口冲突
 - 使用 Docker 编排框架指定并协调容器资源，如 Kubernetes、Marathon/Mesos、Amazon EC2 Container Service
- 部署过程：
 - 构建 Docker 镜像：在构建时，部署流水线使用容器镜像构建工具，该工具读取服务代码和镜像描述，以创建容器镜像并将其存储在镜像仓库中。
 - 运行 Docker 镜像：在运行时，从镜像仓库中拉取容器镜像，并用于创建容器。

Each container is a sandbox that isolates the processes.



Shared by all of the containers



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：将服务部署到容器

• 构建Docker镜像

- 创建Dockerfile：指定基础容器镜像、一系列用于安装软件和配置容器的指令，以及在创建容器时运行的脚本命令（Listing 12.1）

- 为Restaurant Service构建镜像的Dockerfile
- 构建一个包含服务的可执行JAR文件的容器镜像
- 基础镜像openjdk:8u171-jre-alpine
- 安装curl用于健康检查
- 容器配置：在启动时运行java-jar命令
- 容器配置：定期调用健康检查端点（30s，5s）
- 将服务的JAR从Gradle构建目录复制到镜像中

• 构建容器镜像： docker build命名

- -t指定镜像的名称
- .指定Docker调用上下文的内容（当前目录）
- 将上下文上传到Docker守护进程，构建镜像

• 推送到镜像仓库： docker push命名

- Docker Hub：公共Docker镜像仓库的示例
- 私有镜像仓库，如Docker Cloud镜像仓库或AWS EC2 Container Registry

• 运行Docker镜像： docker run命令

Listing 12.1 The Dockerfile used to build Restaurant Service

```
FROM openjdk:8u171-jre-alpine
RUN apk --no-cache add curl
CMD java ${JAVA_OPTS} -jar ftgo-restaurant-service.jar
HEALTHCHECK --start-period=30s --interval=5s CMD curl http://localhost:8080/actuator/health || exit 1
COPY build/libs/ftgo-restaurant-service.jar .
```

The base image

Install curl for use by the health check.

Configure Docker to run java -jar .. when the container is started.

Copies the JAR in Gradle's build directory into the image

Configure Docker to invoke the health check endpoint.

Listing 12.2 The shell commands used to build the container image for Restaurant Service

```
cd ftgo-restaurant-service
./gradlew assemble
docker build -t ftgo-restaurant-service .
docker push registry.acme.com/ftgo-restaurant-service:1.0.0.RELEASE
```

Change to the service's directory.

Build the service's JAR.

Build the image.

Listing 12.3 Using docker run to run a containerized service

```
docker run \
  -d \
  --name ftgo-restaurant-service \
  -p 8082:8080 \
  -e SPRING_DATASOURCE_URL=... -e SPRING_DATASOURCE_USERNAME=... \
  -e SPRING_DATASOURCE_PASSWORD=... \
  registry.acme.com/ftgo-restaurant-service:1.0.0.RELEASE
```

Runs it as a background daemon

The name of the container

Binds port 8080 of the container to port 8082 of the host machine

Environment variables

Image to run



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 优点：

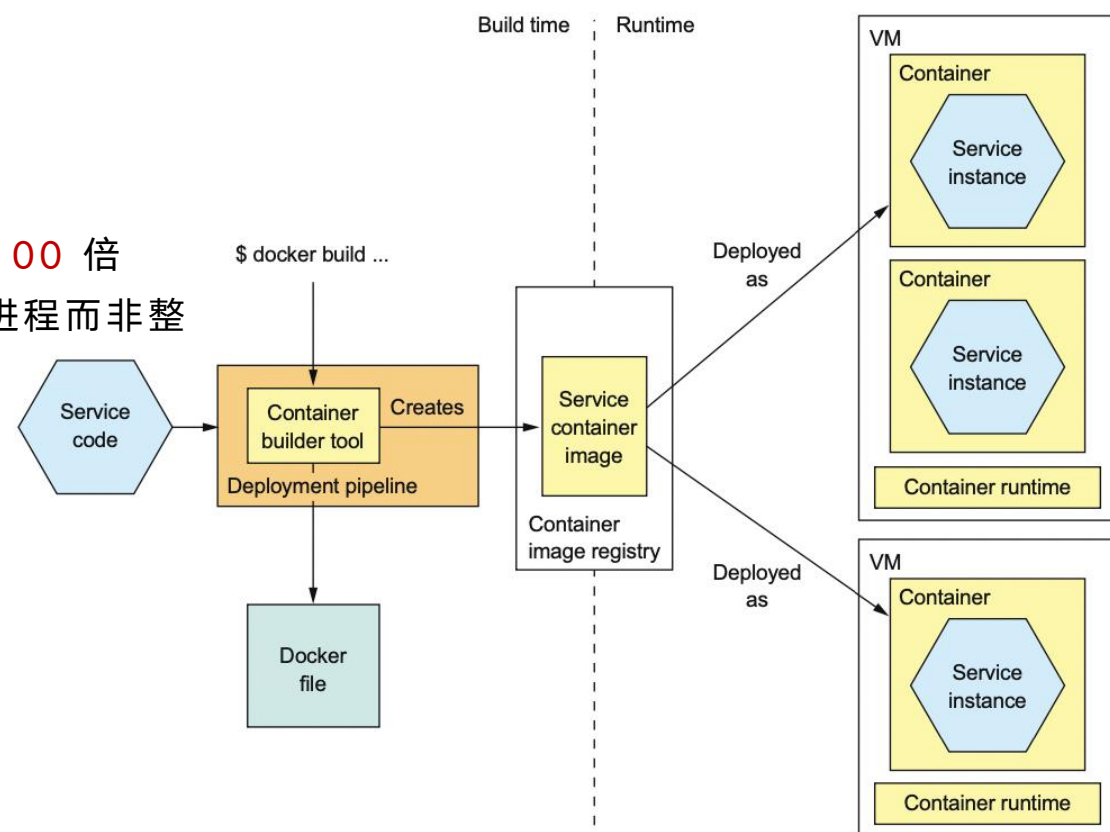
- 通过更改容器实例的数量可以直接扩展和缩减服务
- 容器封装了用于构建服务的技术细节，所有服务都以完全相同的方式启动和停止
- 每个服务实例都是隔离的
- 容器对服务实例消耗的 CPU 和内存施加限制
- 容器的构建和启动速度非常快
 - 将应用程序打包为 Docker 容器比将其打包为 AMI 快 100 倍
 - Docker 容器启动速度明显快于 VM（仅启动应用程序进程而非整个操作系统）

• 缺点：

- 大量的容器镜像管理工作（操作系统补丁、基础设施）
- 部署容器的基础设施不如部署虚拟机的基础设施丰富

• 相关模式：

- 替代模式：将服务部署到虚拟机
- 泛化模式：单主机部署单个服务实例





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：无服务器部署

- 使用公有云提供的serverless部署机制部署服务
- 部署细节对用户隐藏，用户和其组织不负责管理低级基础设施(无服务器概念)
- 基础设施获取服务代码并运行，根据消耗的资源为每个请求付费
- 需打包代码（例如 ZIP），将其上传到部署基础设施
- 公有云serverless平台：AWS Lambda、Google Cloud Functions、Azure Functions
- 开源serverless框架：Apache Openwhisk、Fission on Kubernetes

Listing 12.8 A Java lambda function is a class that implements the RequestHandler interface.

```
public interface RequestHandler<I, O> {  
    public O handleRequest(I input, Context context);  
}
```

- ✓ AWS Lambda支持Java、Node.js、C#、GoLang和Python
- ✓ AWS Lambda函数是无状态服务



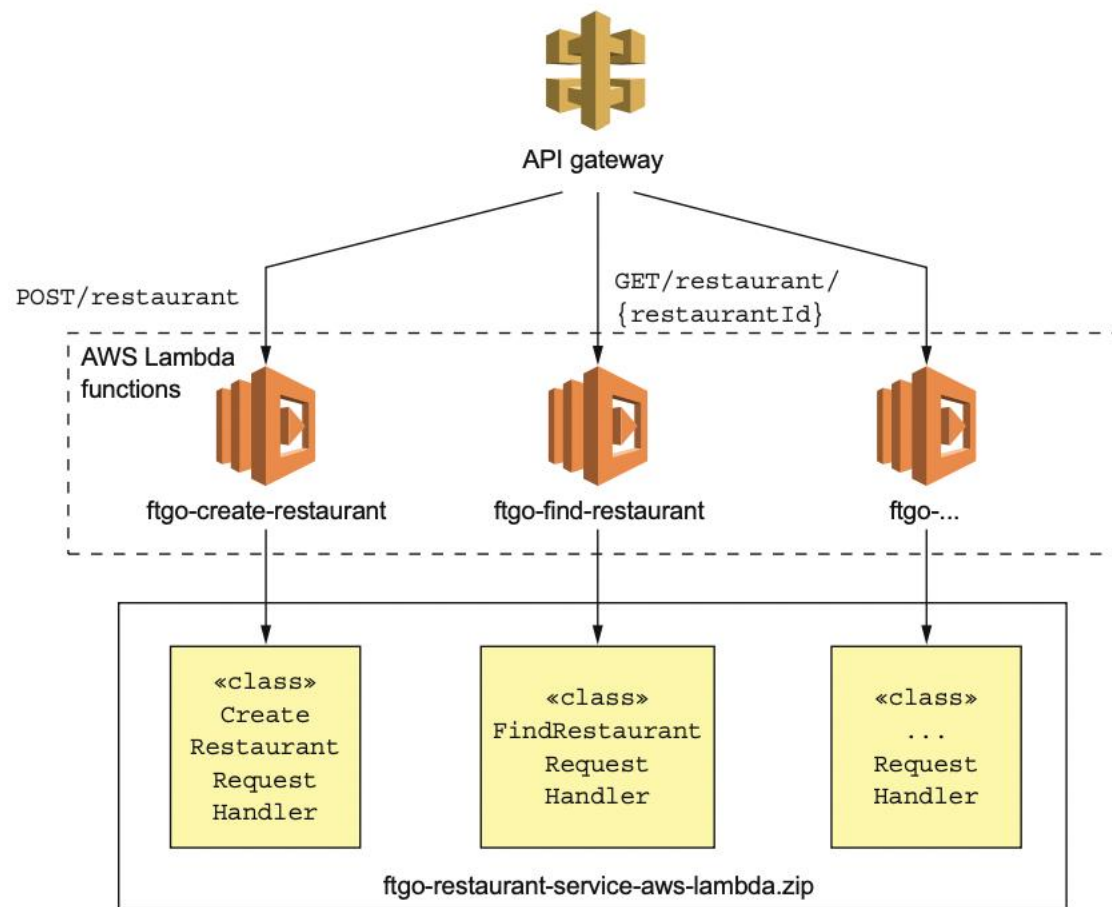
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：无服务器部署 - AWS Lambda

- 将服务的代码打包到一个 ZIP 文件中，将其上传到 AWS Lambda
 - restaurant - service-aws-lambda.zip
 - 实现Restaurant Service服务功能
- 每一个Lambda函数都有一个请求处理类
 - ftgo-create-restaurant Lambda函数
 - CreateRestaurantRequestHandler类
- 当事件发生时，AWS Lambda 寻找函数的空闲实例，没有可用实例则新启动并调用处理程序函数
- AWS Lambda 运行足够的函数实例来处理负载，使用容器来隔离 lambda 函数的每个实例（EC2 上运行）



使用AWS Lambda部署Restaurant Service



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 优点：

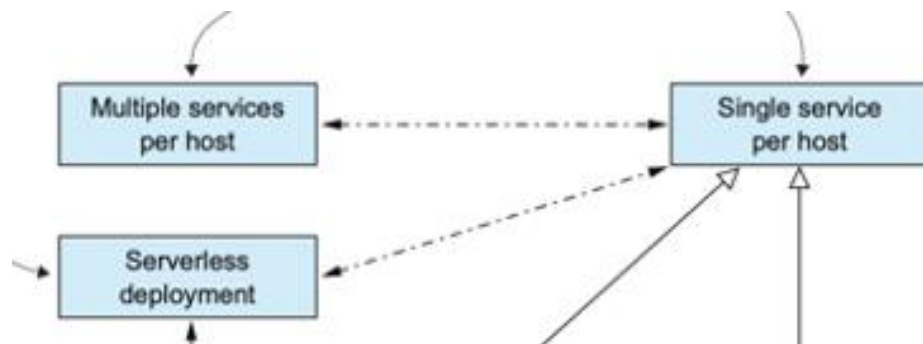
- AWS服务集成简单：AWS服务生成事件、AWS API Gateway处理HTTP请求的Lambda函数
- 消除系统管理任务：底层系统管理、操作系统或运行时打补丁，专注于开发应用程序
- 弹性伸缩：AWS Lambda运行应用程序所需的多个实例以动态处理负载
- 基于使用情况的定价：与典型的IaaS云不同，AWS Lambda按请求所消耗的资源收费

• 缺点：

- 长尾延迟：AWS Lambda动态运行代码，需花费时间配置和启动应用，某些请求具有高延迟（Java服务通常需要至少几秒钟，不适合对延迟敏感的服务）
- 基于有限事件与请求的编程模型：不用于长时间运行的服务（使用第三方消息代理的消息服务）

• 相关模式：

- 替代模式：
 - 单主机部署单个服务实例





南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



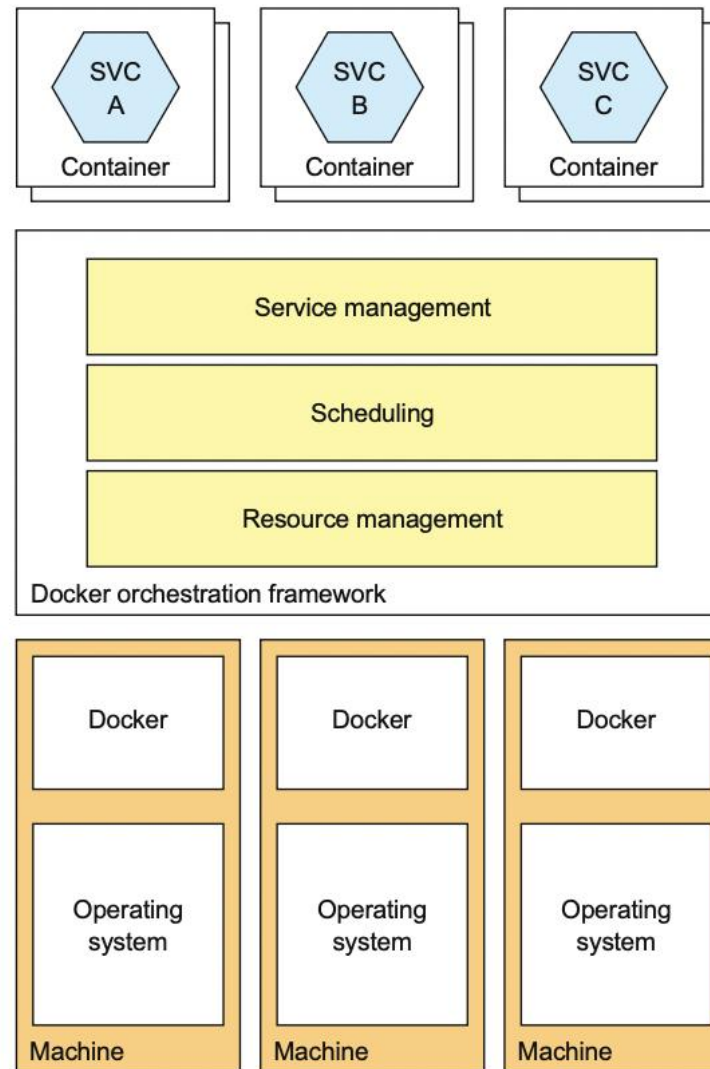
补充：微服务部署模式

• 模式：服务部署平台

- 使用部署平台作为应用程序部署的自动化基础设施
- 提供服务抽象（一组命名的、高度可用的服务实例）
 - Docker 编排框架，包括Docker swarm 模式和 Kubernetes
 - 无服务器平台，例如 AWS Lambda
 - PaaS，包括Cloud Foundry和AWS Elastic Beanstalk
- Docker编排框架将运行Docker的一组计算机转变为资源集群，将容器分配给机器，提供**资源管理、调度、服务管理**功能

• 相关模式：

- 后续模式：
 - 将服务部署到虚拟机
 - 将服务部署容器
 - 无服务器部署





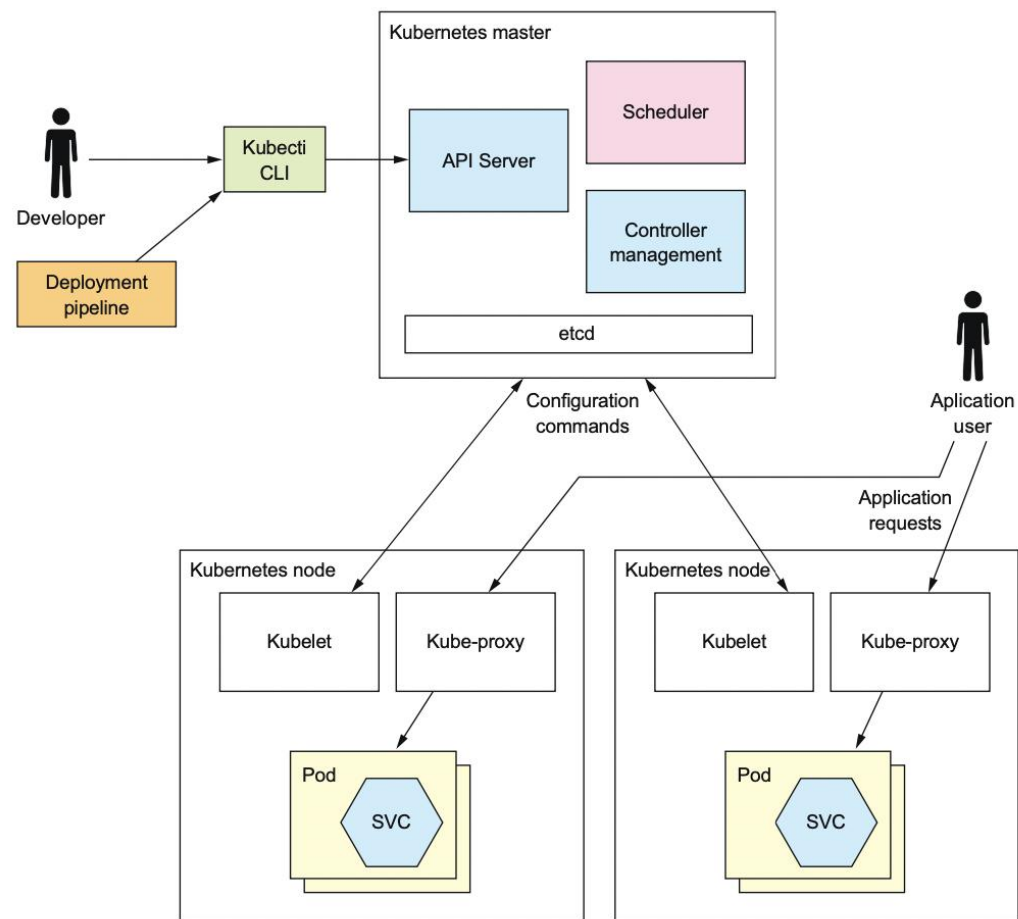
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

• 模式：服务部署平台 - Kubernetes集群

- Google于2014年开源，简称K8s
- 由管理集群的主节点和运行服务的普通节点组成
- 通过API服务器与Kubernetes交互，API服务器与主节点上运行的其他集群管理软件一起运行
- 应用程序容器在节点上运行，每个节点运行多个组件
 - Pods：应用程序服务容器
 - Kubelet：创建和管理节点上的应用程序容器（Pod）
 - Kube-proxy：管理网络，将应用程序请求路由到Pod，包括跨Pod的负载均衡。可以直接使用代理，也可以通过配置Linux内核中内置的iptables路由规则间接地完成路由工作





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务部署模式

- **小结：** 应选择支持服务要求的最轻量级部署模式：Serverless、容器、虚拟机……

模式	优点	缺点
无服务器	• 消除管理操作系统和运行时状态的必要性 • 自动弹性配置 • 基于请求的定价	• 长尾延迟 • 使用基于事件 / 请求的编程模型
容器	• 必须管理操作系统和运行时 • 需要管理 Docker编排框架及其底层运行的虚拟机	• 轻量级，比Serverless部署更灵活 • 延迟可预测
虚拟机	• Amazon EC2等现代云 • 部署小型简单应用程序可能比设置Docker编排框架更容易	• 重量级且部署速度较慢 • 使用更多的资源
服务部署平台	• 多种后续模式支持 • 功能强大	• 额外的技术学习和资源成本
主机（单/多）	——	——



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

• 上下文

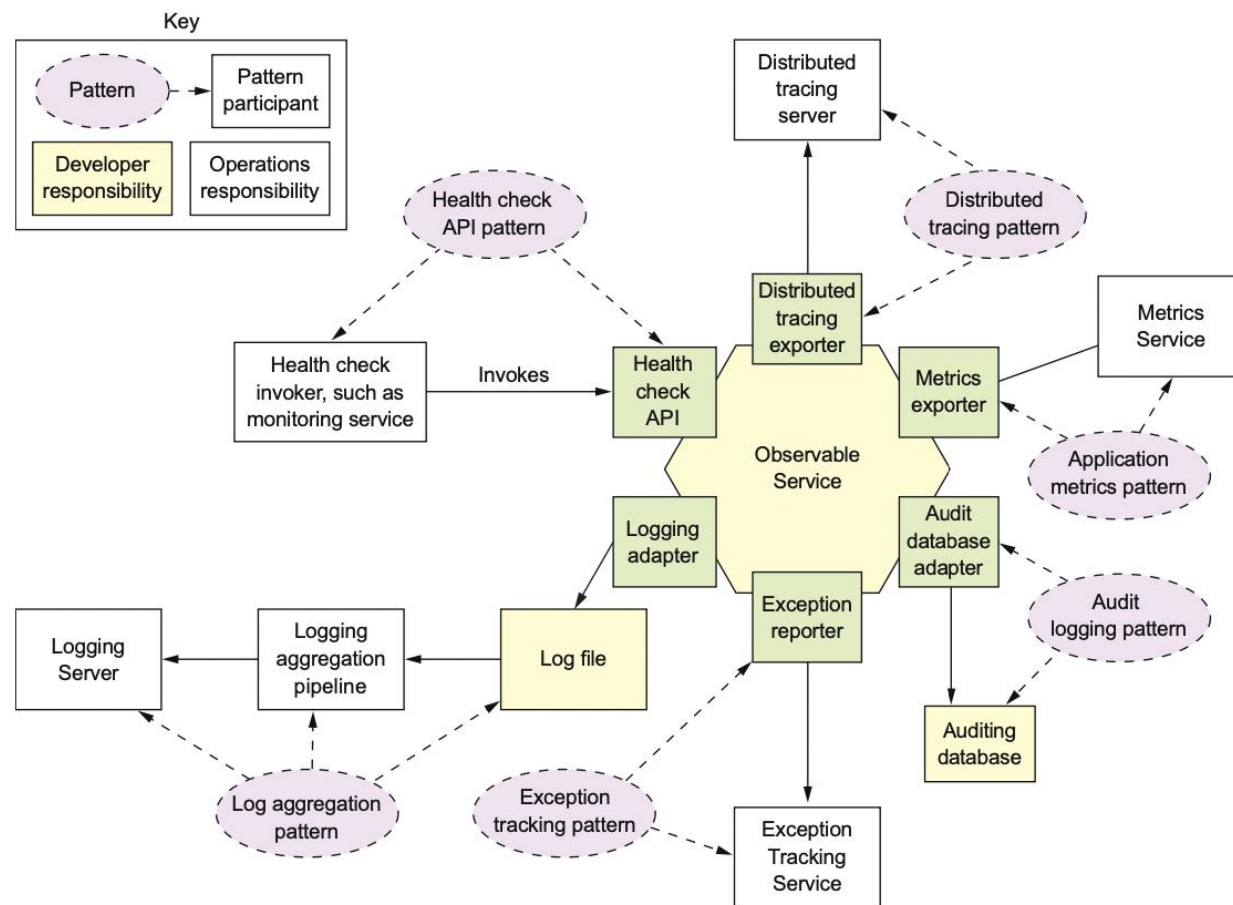
- 多台机器上、多个服务和服务实例
- 请求跨越多服务实例，每个服务通过执行一个或多个操作来处理请求
- 以标准化格式将操作信息写入日志文件，跟踪用户行为和代码异常
- 服务实例可能无法处理请求但仍在运行

• 问题：

- 如何理解用户和应用程序的行为并解决问题？
- 如何检测正在运行的服务实例无法处理请求？

• 模式列表：

- 日志聚合
- 审计日志
- 应用程序指标
- 分布式跟踪
- 异常跟踪
- 健康检查API



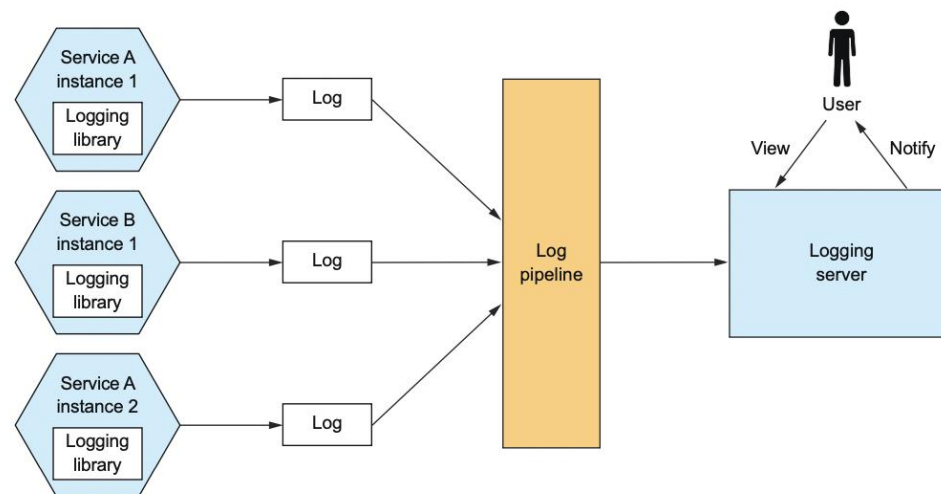


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

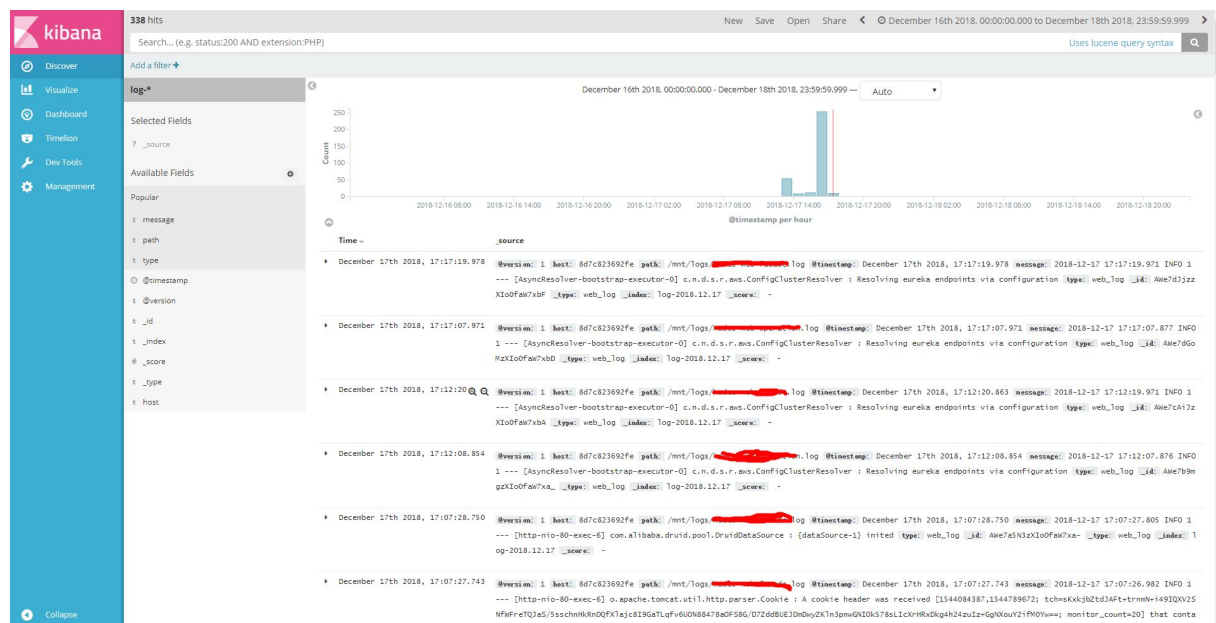


补充：微服务可观测性模式

- **问题：** 如何理解应用程序的行为并解决问题？
- **需求：** 任何解决方案都应该具有最小的运行时开销
- **模式：** 日志聚合模式
 - 使用集中式日志记录服务聚合来自每个服务实例的日志
 - 用户可搜索和分析日志
 - 可配置当某些消息出现在日志中时触发的警报
 - 实例： AWS Cloud Watch, Logstash (ELK)
- **缺点：**
 - 处理大量日志需要大量的基础设施
- **相关模式：**
 - 分布式追踪、异常跟踪



日志聚合模式





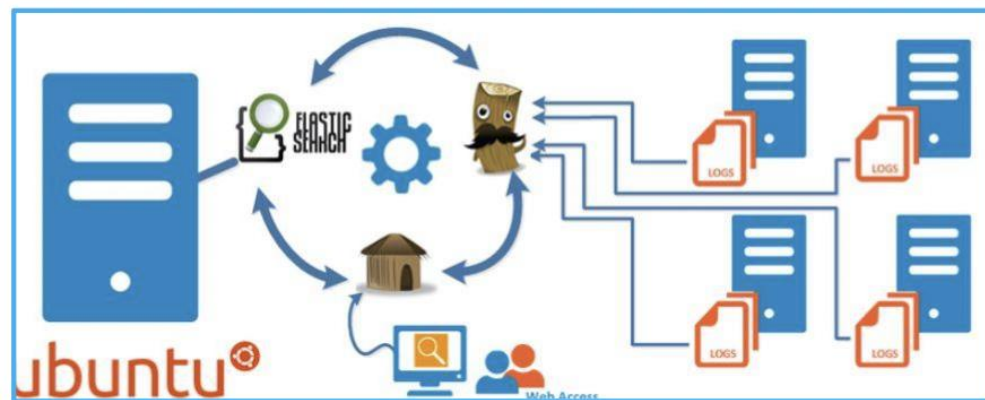
南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

日志记录的基础设施：

- ELK
 - Elasticsearch：面向文本搜索的NoSQL数据库，用作日志记录服务器
 - Logstash：聚合服务日志并将其写入Elasticsearch的日志流水线
 - Kibana：Elasticsearch的可视化工具
- 开源日志流水线包括Fluentd和Apache Flume
- 商用如AWS Cloud Watch Logs等





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **问题**：如何理解用户和应用程序的行为并检测定位问题？
- **需求**：了解用户最近执行了哪些操作，帮助支持、确保合规性、安全性和可疑行为等
- **模式**：审计日志模式
 - 向业务逻辑中添加审计日志代码
 - 创建审核日志条目并保存在数据库中
- **优点**：
 - 提供用户操作的记录
- **缺点**：
 - 审计代码与业务逻辑交织，使业务逻辑复杂化
- **相关模式**：
 - 后续模式：事件溯源（实施审计的可靠方式）

Login ID	User Name	Application	Login Date	Logout Date	Logged In	Failed Logins
CSDAdmin	Cherwell Admin	Cherwell Administrator	11/2/2015 11:24 AM		True	0
gina	Gina Mehra	Cherwell Web Portal	11/2/2015 10:38 AM		True	0
gina	Gina Mehra	Cherwell Web Portal	11/2/2015 10:38 AM		False	1
CSDAdmin	Cherwell Admin	Cherwell Administrator	11/2/2015 10:29 AM	11/2/2015 10:43 AM	False	0
CSDAdmin	Cherwell Admin	Cherwell Administrator	11/2/2015 10:29 AM		False	1
andrew	Andrew Simms	Cherwell Administrator	10/29/2015 10:58 ...	10/29/2015 11:47 ...	False	0
andrew	Andrew Simms	Cherwell Administrator	10/29/2015 10:36 ...	10/29/2015 10:41 ...	False	0
CSDAdmin	Cherwell Admin	Cherwell Administrator	10/27/2015 8:40 AM	10/27/2015 8:55 AM	False	0
henri	Henri Bryce	Cherwell Service Managem...	10/23/2015 3:12 PM	10/23/2015 3:13 PM	False	0
henri	Henri Bryce	Cherwell Administrator	10/23/2015 8:16 AM	10/23/2015 8:24 AM	False	0
henri	Henri Bryce	Cherwell Administrator	10/21/2015 2:16 PM	10/21/2015 2:17 PM	False	0
henri	Henri Bryce	Cherwell Administrator	10/20/2015 2:11 PM	10/20/2015 2:13 PM	False	0
henri	Henri Bryce	Cherwell Administrator	10/15/2015 10:19 ...	10/15/2015 10:49 ...	False	0
jbucknall		Cherwell Administrator	10/14/2015 3:43 PM		False	1
andrew	Andrew Simms	Cherwell Administrator	10/14/2015 3:42 PM	10/14/2015 4:06 PM	False	0
henri	Henri Bryce	Cherwell Administrator	10/14/2015 8:24 AM	10/14/2015 8:26 AM	False	0
henri	Henri Bryce	Cherwell Administrator	10/14/2015 8:22 AM	10/14/2015 8:23 AM	False	0
CSDAdmin	Cherwell Admin	Scheduling Server	10/14/2015 3:15 AM		True	0
CSDAdmin	Cherwell Admin	Configuration Scheduling S...	10/14/2015 3:15 AM		True	0
CSDAdmin	Cherwell Admin	Automation Process Server	10/14/2015 3:15 AM		True	0
andrew	Andrew Simms	Cherwell Administrator	10/13/2015 3:26 PM	10/13/2015 3:29 PM	False	0
andrew	Andrew Simms	Cherwell Service Managem...	10/13/2015 2:56 PM	10/13/2015 3:36 PM	False	0

审计日志模式

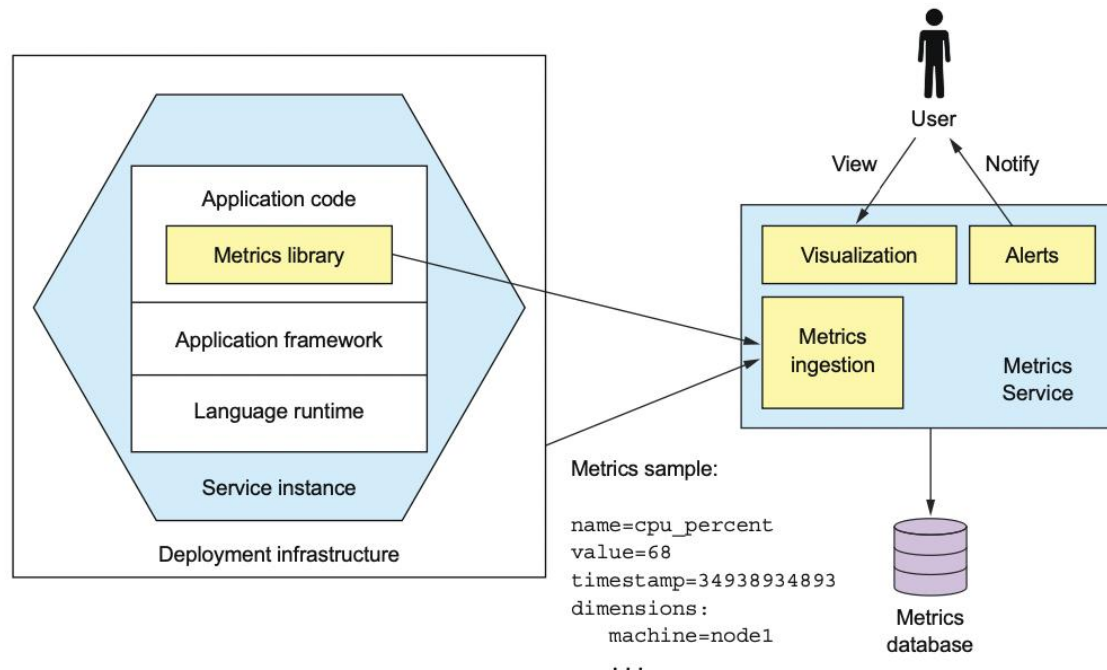


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **问题：** 如何理解应用程序的行为并检测定位问题？
- **需求：** 任何解决方案都应该具有最小的运行时开销
- **模式：** 应用程序指标模式
 - 检测服务以收集有关各个操作的统计信息，在集中式指标服务中聚合指标，提供报告和警报。聚合指标两种模型：
 - push - 服务将指标推送到指标服务
 - pull - 指标服务从服务中提取指标
 - 实例：Coda Hale/Yammer Java 指标库、Prometheus（普罗米修斯）、AWS Cloud Watch等
- **优点：**
 - 提供对应用程序行为的深入洞察
- **缺点：**
 - 指标代码与业务逻辑交织在一起，使其更加复杂
 - 聚合指标可能需要大量的基础设施
- **相关模式：**
 - 其他可观测性模式



应用程序指标模式

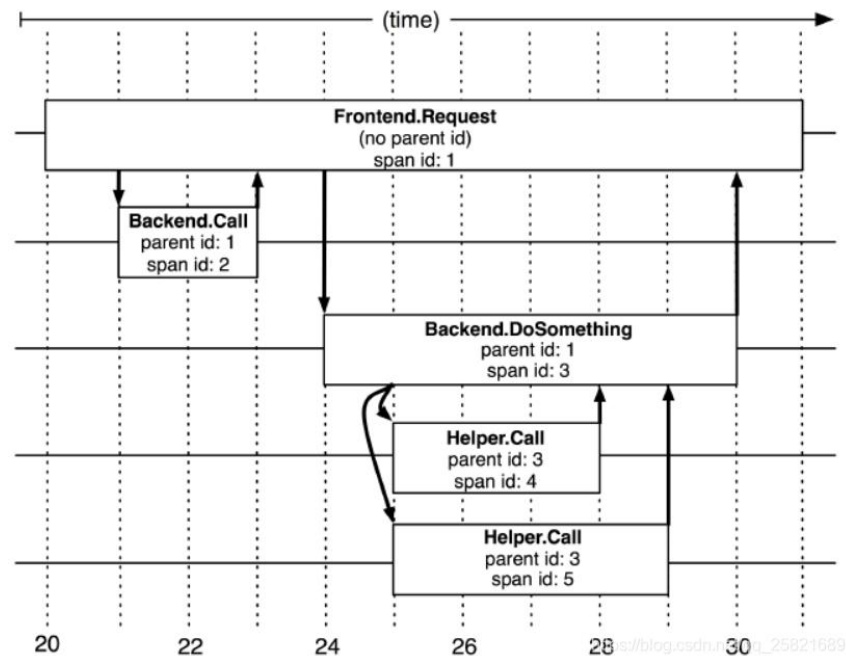
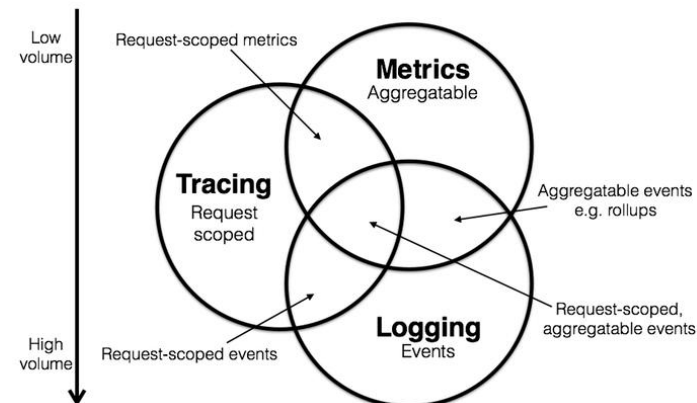


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **问题：** 如何理解应用程序的行为并解决问题？
- **需求：**
 - 外部监控只报告总体响应时间和调用次数，无法深入了解各个操作
 - 任何解决方案都应该具有最小的运行时开销
 - 请求的日志条目分散在许多日志中
- **模式：** 分布式追踪模式
 - 记录单次请求范围以内的信息
 - 为每个外部请求分配一个唯一的外部请求 ID
 - 并在提供可视化和分析的集中式服务器中记录请求如何从一个服务流向下一个服务
 - 在所有日志消息中包含外部请求 ID
 - 记录在集中服务中处理外部请求时执行的请求和操作的信息（例如开始时间、结束时间）



分布式追踪模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **模式：** 分布式追踪模式

- Spring Cloud Sleuth - Spring Cloud 应用程序的分布式跟踪
- Open Zipkin - 用于记录和显示跟踪信息的服务
- Open Tracing - 用于分布式跟踪的标准化 API

- **优点：**

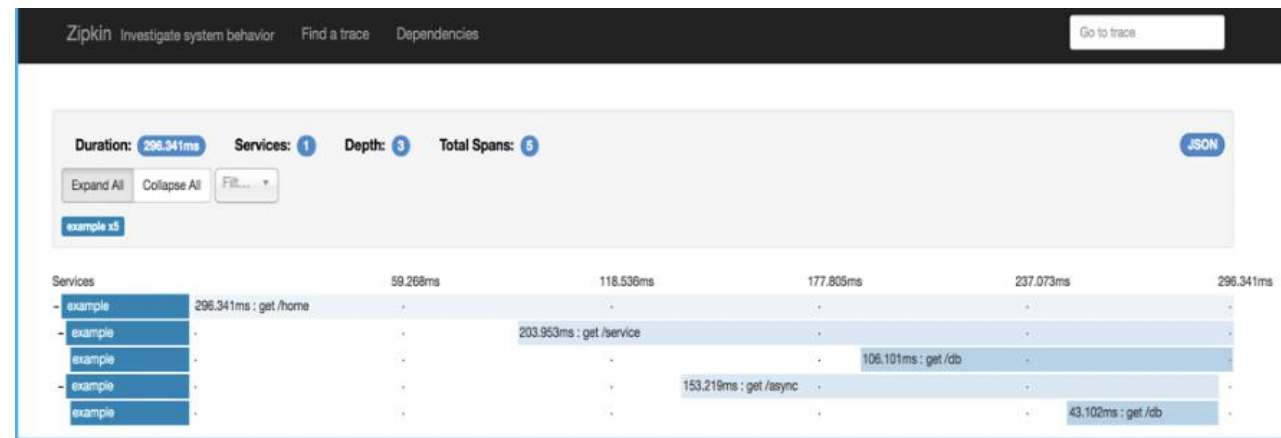
- 提供了对系统行为的有用洞察，包括延迟的来源
- 使开发人员能够通过聚合日志中搜索其外部请求 ID 来查看单个请求是如何处理的

- **缺点：**

- 聚合和存储追踪数据可能需要大量的基础设施

- **相关模式：**

- 日志聚合 - 外部请求 ID 包含在每个日志消息中



分布式追踪模式

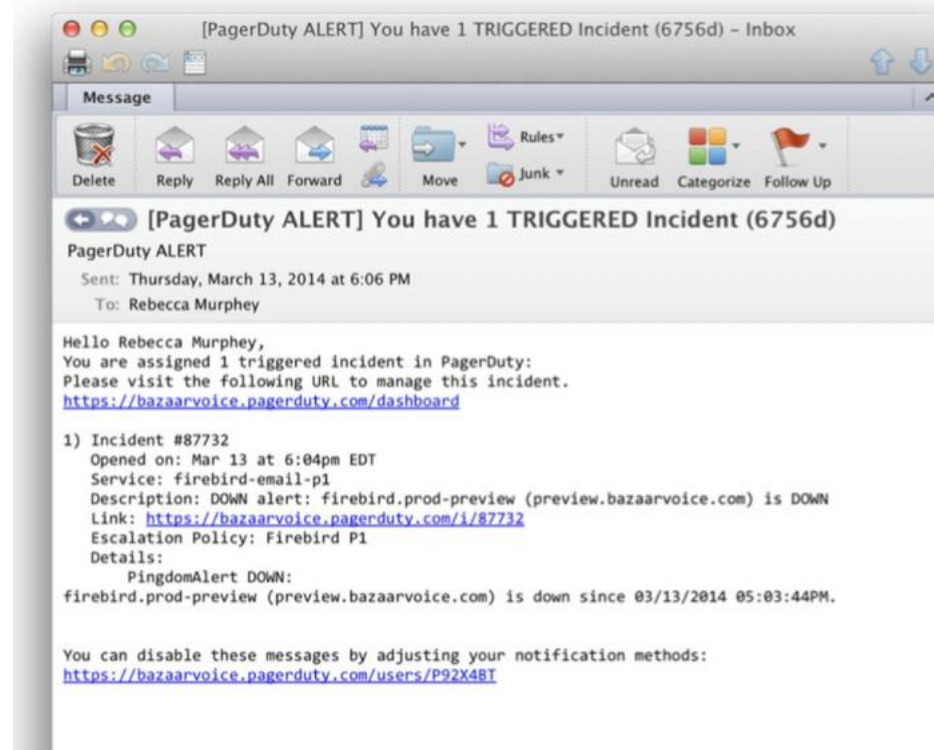


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **上下文：** 处理请求时有时会发生错误。发生错误时，服务实例会抛出异常，其中包含错误消息和堆栈跟踪
- **问题：** 如何理解应用程序的行为并解决问题？
- **需求：**
 - 异常必须由开发人员去重、记录、调查并解决潜在问题
 - 任何解决方案都应该具有最小的运行时开销
- **模式：** 异常跟踪模式
 - 向集中式异常跟踪服务报告所有异常，该服务聚合和跟踪异常并通知开发人员。
 - 实例：Sentry Datadog、PagerDuty
- **优点：**
 - 更容易查看异常并跟踪其解决方案
- **缺点：**
 - 异常跟踪服务是额外的基础设施
- **相关模式：**
 - 日志聚合-应记录异常并报告给跟踪服务



异常追踪模式

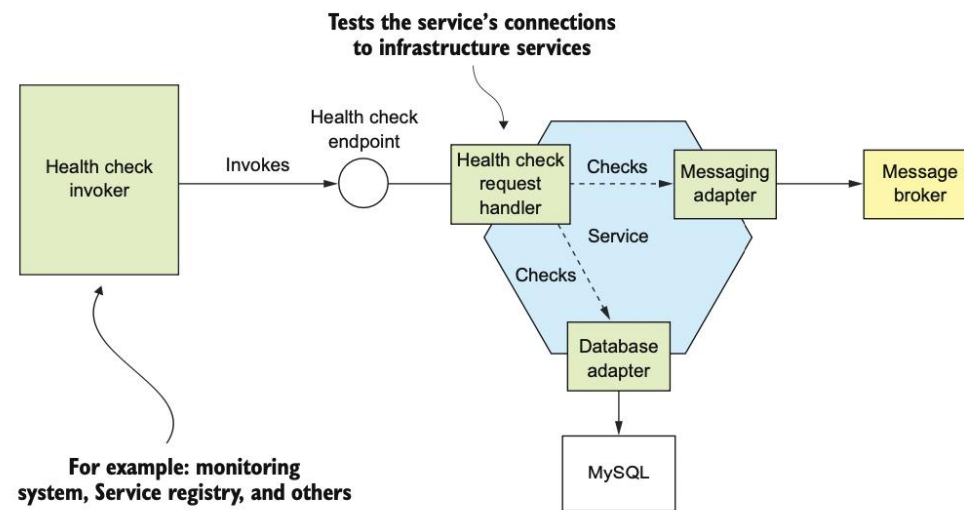


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **问题**：如何检测正在运行的服务实例无法处理请求？
- **需求**：当服务实例失败时应生成警报，请求应该被路由到正常工作的服务实例
- **模式**：健康检查API模式
 - 服务具有/health返回服务健康状况的健康检查 API 端点（实例[Spring Boot Actuator](#)），执行检查：
 - 服务实例使用的基础设施服务的连接状态
 - 主机的状态，例如磁盘空间
 - 应用程序特定逻辑
 - 监控服务、服务注册表或负载均衡可以定期“ping”调用端点来检查服务实例的健康状况



健康检查API模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



补充：微服务可观测性模式

- **模式：** 健康检查API模式

- 使用 Spring Boot 和 Spring Cloud 作为微服务框架
- 提供健康检查端点，由Spring Boot Actuator模块实现
- 配置调用/health可扩展健康检查逻辑的 HTTP 端点。

- **优点：**

- 定期测试服务实例的健康状况

- **缺点：**

- 不够全面
- 服务实例可能在健康检查之间失败

- **相关模式：**

- 前置模式：服务注册与发现模式、部署相关模式

```
@Component
public class HealthCheck implements HealthIndicator {

    @Override
    public Health health() {
        int errorCode = check(); // perform some specific health check
        if (errorCode != 0) {
            return Health.down()
                .withDetail("Error Code", errorCode).build();
        }
        return Health.up().build();
    }

    public int check() {
        // Our logic to check health
        return 0;
    }
}
```



南京大学120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022

03 课程回顾

- 微服务架构基础知识
- 微服务架构核心设计模式



南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构设计

➤ 微服务架构基础知识

- 发展历程、单体架构优劣势
- 微服务架构定义、主要特性、架构对比

➤ 微服务架构核心设计模式

- 挑战问题、模式和模式语言（模式组集合）
- 微服务的拆分和定义（粒度问题），如何拆、结果优劣
- 进程间通信机制复杂性高于方法调用、局部故障
- 部署复杂性（技术异构、相互隔离、经济高效）
- 运维复杂性（自动化部署工具、产品化PaaS平台、Docker容器编排平台）
-

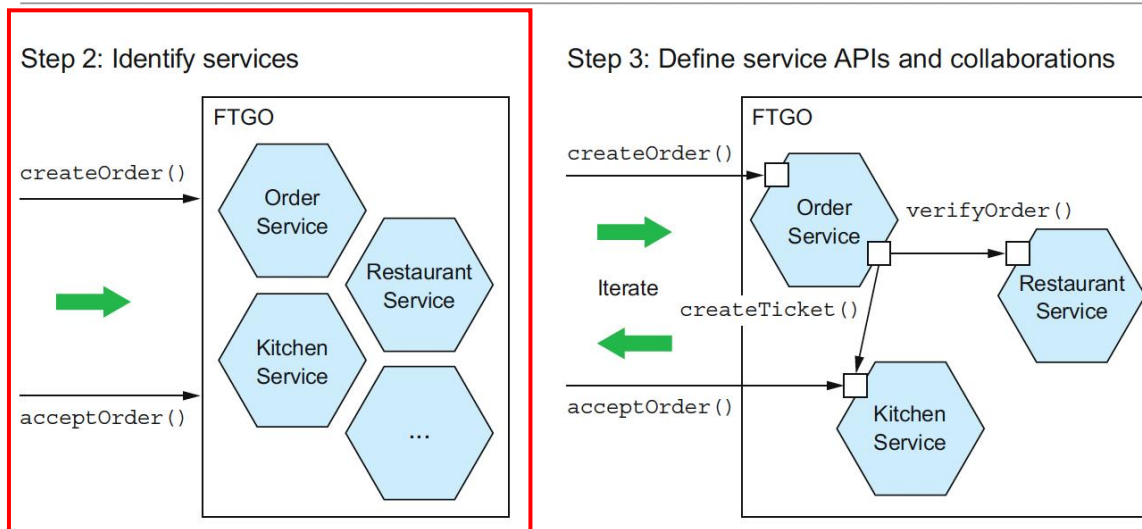
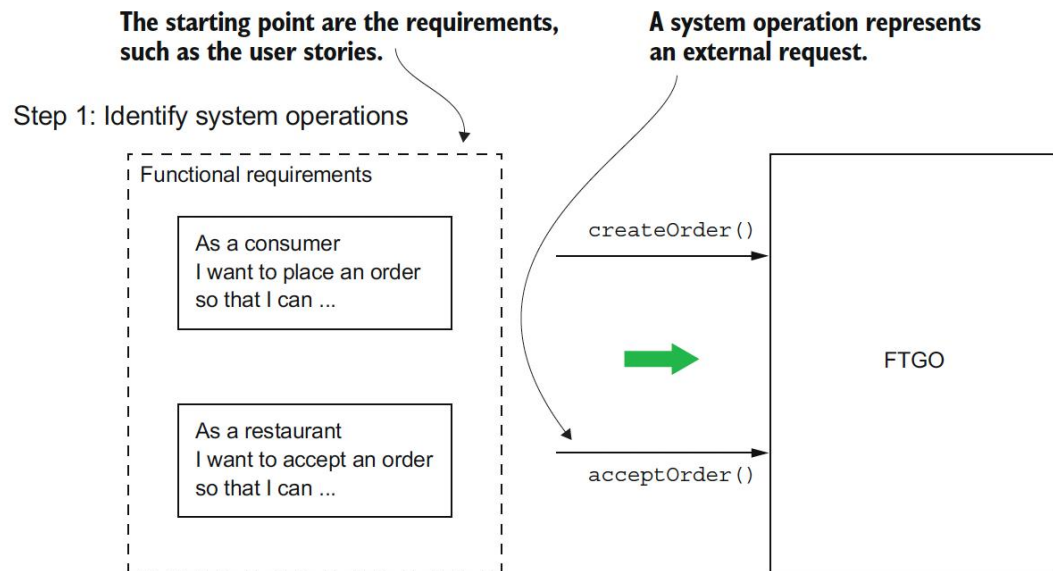


南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构拆分模式

- **问题：** 如何将应用拆分为微服务？
- **需求：**
 - 高内聚：实现一组密切相关的功能
 - 松耦合：封装内部细节，API交互
 - 单一职责原则（SRP）
 - 共同封闭原则（CCP）
 - 双披萨团队开发
 - 团队自治
 -
- **模式列表：**
 - 根据业务能力进行服务拆分
 - 根据子域进行服务拆分





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



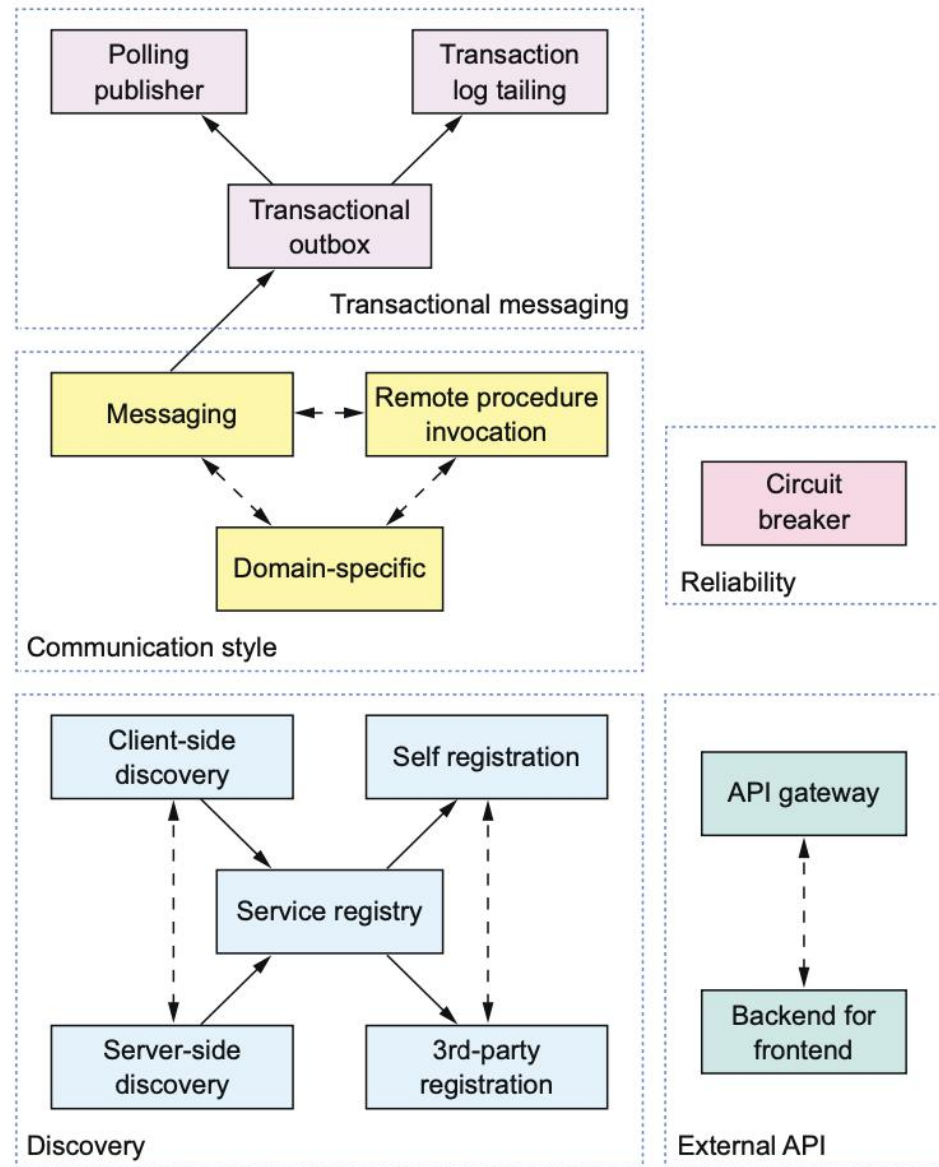
微服务架构通信模式

• 问题：

- 如何避免由于服务故障或网络中断所引起的故障蔓延到其他服务？
- 客户端如何在网络上发现服务实例的位置？
- 如何处理外部客户端与服务之间的通讯？
-

• 模式列表：

- 断路器（Circuit Breaker）模式
- 应用层服务发现模式
- 平台层服务发现模式
- API Gateway模式
-
-





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构部署模式

• 上下文

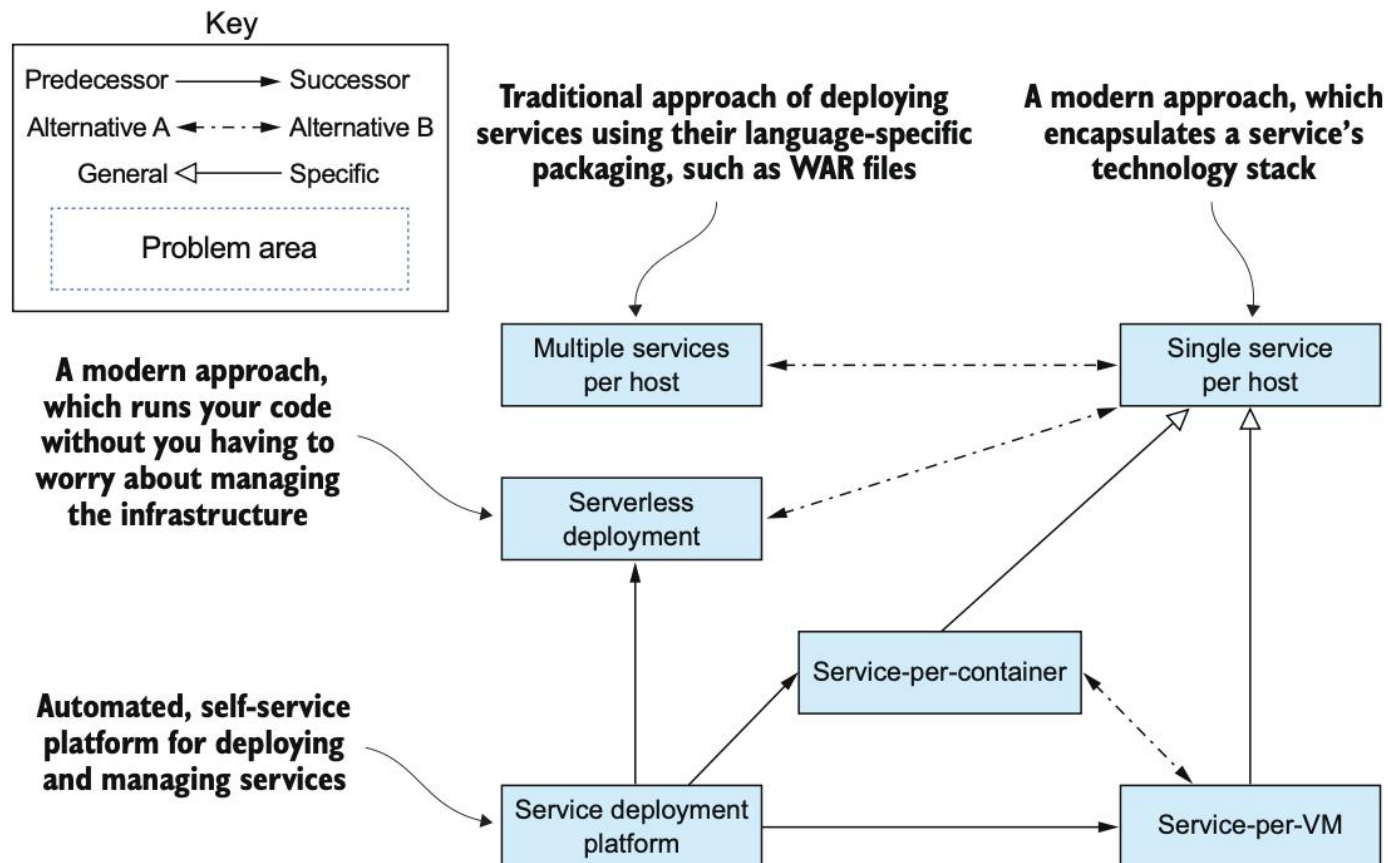
- 微服务架构包含一组服务
- 每个服务都部署为一组服务实例，以实现吞吐量和可用性

• 问题：如何部署以解决以下需求？

- 服务使用各种语言、框架和框架版本编写
- 需要快速构建、独立部署和扩展服务
- 服务实例需相互隔离
- 需限制服务消耗的资源（CPU 和内存）
- 尽可能经济高效地部署应用程序

• 模式列表：

- 单主机部署多个服务实例
- 单主机部署多个服务实例
- 将服务部署到虚拟机
- 将服务部署到容器
- 服务部署平台
- 无服务器部署





南京大學120周年校庆
120th ANNIVERSARY
NANJING UNIVERSITY
1902 - 2022



微服务架构可观测性模式

• 上下文

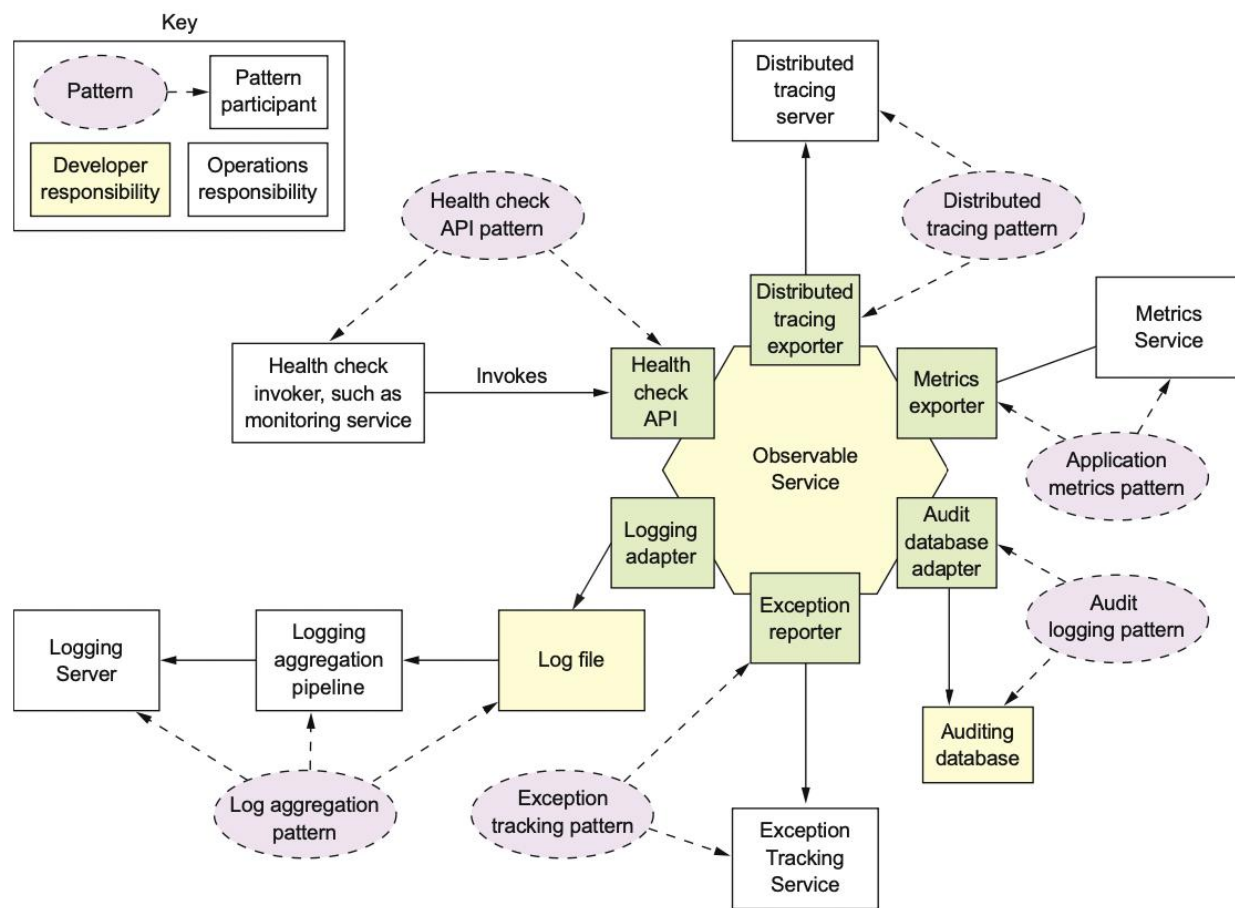
- 多台机器上、多个服务和多个服务实例
- 请求跨越多个服务实例，每个服务通过执行一个或多个操作来处理请求
- 以标准化格式将操作信息写入日志文件，跟踪用户行为和代码异常
- 服务实例可能无法处理请求但仍在运行

• 问题：

- 如何理解用户和应用程序的行为并解决问题？
- 如何检测正在运行的服务实例无法处理请求？

• 模式列表：

- 日志聚合
- 审计日志
- 应用程序指标
- 分布式跟踪
- 异常跟踪
- 健康检查API





谢 谢

诚耀百廿 雄创一流

